AOIA Master Library

Consolidated AOIA audit, governance, routing, epistemic safety, runtime architecture, and deterministic orchestration research library.

#	Document
1	AOIA_Audit_Wave2_2305 (1).md
2	AOIA_Audit_Wave2_2305 (2).md
3	AOIA_Audit_Wave2_2305.md
4	AOIA_CURRENT_STATE_SUMMARY.md
5	AOIA_Safety_Governance_Review_2305-1.md
6	AOIA_Safety_Governance_Review_2305-2.md
7	AOIA_Safety_Governance_Review_2305.md
8	CODEX_RAPORT_1925.md
9	CODEX_RAPORT_2004.md
10	CODEX_RAPORT_2011.md
11	CODEX_RAPORT_2028.md
12	DETERMINISM_AUDIT.md
13	EPISTEMIC_RISK_REPORT.md
14	FORENSIC_ARCHITECTURE_REPORT.md
15	MEMORY_FLOW_ANALYSIS.md
16	MODULE_DEPENDENCY_MAP.md
17	NEXT_STEPS_RECOMMENDATION.md
18	RAPORT_16-10.md
19	REPOSITORY_TREE.md
20	SYSTEM_EXECUTION_FLOW.md

1. AOIA_Audit_Wave2_2305 (1).md

AOIA — Architecture External Audit Wave 2
Forensic Systems Review · Epistemic Stability · Provenance Architecture · Governance
Claude Sonnet 4.6 · 23 May 2026 · Constrained Epistemic Runtime

---

> **Scope:** This is a forensic systems audit — not marketing analysis.
> Evaluated domains: epistemic stability, provenance architecture, memory semantics,
> contradiction handling, bounded cognition infrastructure, runtime governance, contamination resistance.

---

1. Major Strengths

Assessed as genuine strengths — not aspirationally correct, but directly addressing documented failure modes in comparable systems. Each strength is real only insofar as it is enforced without exception.

`DOMAIN SEPARATION` ■
Physical separation of LSC-Research, MHLM_MDLH, and AOIA-Core

The most consequential structural decision in Wave 2. Mixing scientific authority with AI safety framing with infrastructure governance in a single repository is how projects generate artifacts that contaminate all three domains simultaneously. Physical separation forces explicit boundary crossings that can be governed, audited, and — critically — refused. This decision should be treated as **permanent**, not as a phase of development that will eventually converge.

`CONTRADICTIONS AS FIRST-CLASS SIGNALS` ■
L5 Contradiction Registry as a top-layer epistemic structure

Treating contradictions as information rather than errors to be resolved is epistemically correct and architecturally unusual. Most systems resolve contradictions automatically, which destroys the signal that something is contested. Placing the contradiction registry at L5 — the highest layer of the memory ontology — signals architectural intent to preserve rather than suppress epistemic tension. This intent must be protected against future pressure to "clean up" the registry by auto-resolving entries.

`LAYERED MEMORY ONTOLOGY (L0–L5)` ■
Formal separation of ephemeral state from immutable evidence

The L0–L5 model is the correct conceptual framework. Separating ephemeral runtime state from operational logs from reasoning traces from provenance records from immutable evidence from the contradiction registry prevents the most common contamination pathway: runtime state being treated as evidence. **The framework exists. The implementation risk is whether the boundaries are enforced in code or only in documentation.**

`RECOGNITION OF CONSENSUS ≠ TRUTH` ■
Multi-model convergence identified as an active risk, not a validation mechanism

Explicitly naming consensus as a contamination risk rather than a quality signal is architecturally mature. Most systems treat model agreement as ground truth. AOIA's recognition that models trained on overlapping data converge on shared errors — not shared truths — is a prerequisite for building a system that can resist the "bad genie" failure mode. This insight must be operationalized in the retrieval and provenance layers, not left as a

design principle.

`APPEND-ONLY LINEAGE GOAL` ■
Provenance-first cognition with replay capability target

The stated goal of append-only lineage with replay capability is correct. If fully implemented, it would make the system's reasoning history auditable, contamination detectable, and failure states recoverable. The gap between "stated goal" and "enforced implementation" is the primary risk associated with this strength.

`LEGACY MIXED ROOT ARCHIVAL` ■
Old mixed repository archived rather than migrated incrementally

Archiving the mixed root rather than incrementally refactoring it is a governance discipline decision. Incremental refactoring of a mixed codebase typically results in permanent partial migration — the old patterns survive in the new structure. Clean archival forces a deliberate rebuild of each boundary. This is operationally harder and architecturally safer.

---

2. Critical Weaknesses

Every weakness below is structurally embedded in the current architecture and will cause a specific class of failure if left unaddressed.

`MEMORY.PY CONTAMINATION` ■ CRITICAL
Persistence / logging / evidence / runtime state mixed in a single module

This is the most dangerous implementation detail in the current architecture. The L0–L5 ontology exists as a design document. If `memory.py` implements multiple layers in the same class hierarchy, with shared mutable state, the ontology is **decorative**. A runtime log entry can become an evidence record through a single misrouted write. A debugging trace can be promoted to provenance by a naming collision.

The module must be refactored into physically separate modules with no shared mutable state **before** the ontology provides any actual safety value.

`VAULT DUALITY` ■ CRITICAL
Machine logs and human notes occupy the same persistence structure

Two distinct contamination risks:
- A machine log entry is accidentally retrieved as human-authored evidence.
- A human note is treated as an operational log by an automated process.

In a system whose provenance model is the primary safety mechanism, conflating machine-generated and human-authored artifacts is a foundational error. The vault must be structurally bifurcated — not namespaced, not tagged, but **physically separate** storage with separate access paths and separate provenance chains.

`KNOWLEDGEROUTER AS LEGACY` ■ HIGH
Secondary routing layer identified as "likely transitional/legacy"

A routing layer described as "likely transitional" is a routing layer that will remain in production **indefinitely**. The word "transitional" in architecture documentation is almost always a prediction that turns out to be wrong. KnowledgeRouter must be removed on a specific date with a specific deprecation plan. Routing infrastructure that

is "probably legacy" generates split-brain query paths, inconsistent provenance chains, and approval boundary gaps.

`ORCHESTRATION REMNANTS` ■ HIGH
Orchestration patterns surviving in the current runtime

Orchestration remnants are not inert. They represent live code paths that can be triggered by sufficiently complex queries, edge cases in the planner fallback, or future feature additions that reference existing patterns. Each remnant is a latent pathway to the autonomous recursive loop that bounded cognition is designed to prevent. Remnants must be enumerated, documented, and either removed or explicitly disabled with monitoring.

`MISSING ENFORCEMENT LAYER` ■ CRITICAL
L0–L5 ontology exists as specification, not as enforced runtime constraint

The layered ontology is only a safety mechanism if layer boundaries are enforced in code. If L2 (reasoning traces) and L4 (immutable evidence) are distinguished only by naming convention or documentation, then any write operation that misnames its target crosses the boundary silently. The enforcement layer — the code that **rejects** a write from L2 attempting to promote itself to L4 without operator approval — is the actual safety mechanism. Its absence makes the ontology a labeling exercise.

`UNDEFINED AUTHORITY RESOLUTION PROTOCOL` ■ CRITICAL
No stated protocol for resolving conflicts between domain authorities

LSC-Research is the scientific authority domain. AOIA-Core is the runtime authority domain. MHLM_MDLH references both. When LSC produces a finding that contradicts an AOIA-Core operational constraint, **which authority governs?** The architecture currently has no stated resolution protocol. In practice, resolution will occur informally, inconsistently, and without an audit trail.

---

3. Architectural Failure Risks

Specific failure modes with defined propagation paths in the current architecture. Not speculative — each mechanism is already present and will activate under specific conditions.

F01 — RUNTIME → EVIDENCE PROMOTION
An L0 ephemeral runtime state object is written to the same persistence store as L4 immutable evidence — through a bug, a naming collision, or a developer who does not know the distinction. The object is subsequently retrieved during a reasoning operation and treated as authoritative evidence. A hallucination produced at runtime has become a canonical fact. Once committed to L4 without the enforcement layer in place, there is no mechanism to detect this without a full audit.

F02 — DOMAIN AUTHORITY BLEED
MHLM_MDLH references LSC-Research as a case study. The reference is formalized in documentation and becomes a template for future references. Future versions of MHLM_MDLH begin treating LSC-Research outputs as direct evidence inputs rather than case study references. The domain separation established in Wave 2 erodes through the reference pathway. Physical separation is bypassed by a documentation convention that gradually becomes an import dependency.

F03 — KNOWLEDGEROUTER SPLIT-BRAIN
A query is routed through AOIAEpistemicKernel for one request type and through KnowledgeRouter for an adjacent request type. The two routing paths apply different provenance rules, different approval boundaries, or different confidence caps. The system produces inconsistent outputs for semantically similar queries. Post-hoc debugging cannot determine which path produced which output because the routing decision is not captured in

the provenance chain.

F04 — CONTRADICTION REGISTRY AUTO-RESOLUTION CREEP
The contradiction registry accumulates entries faster than human reviewers can process them. Operationally, the team introduces a "low-severity auto-resolve" threshold. The threshold is gradually raised under queue pressure. Within 12–18 months, the majority of contradiction entries are auto-resolved before human review. The safety mechanism has been inverted: the registry now **suppresses** contradictions rather than preserving them.

F05 — CLOUD PLANNER PROVENANCE BYPASS
A query that cannot be completed by the local runtime within its execution bounds is escalated to the cloud-assisted planner fallback. The planner returns a result that does not include a provenance chain — it is a synthesized output without traceable source attribution. The local runtime accepts this output and commits it to L3 (provenance records) with the planner identified as the source. A synthesized AI output has been committed to the provenance layer as if it were an external source record.

---

4. Contradiction Handling Semantics

The contradiction registry at L5 is the architecturally correct placement. The semantics require precise specification before the implementation can be trusted.

■ CONTRADICTION TAXONOMY IS MISSING
The architecture does not currently specify what types of contradictions exist:
- **Source conflict:** Source A says X, Source B says not-X
- **Temporal conflict:** Source A said X in 2022, X is no longer current
- **Logical conflict:** Claim X and claim Y are mutually exclusive
- **Confidence conflict:** The same claim appears with incompatible confidence scores from different reasoning paths

Without taxonomy, all contradictions are stored as equivalent signals. A stale timestamp and a logical impossibility receive the same operational treatment. This makes the registry unactionable at scale.

■ RESOLUTION PROHIBITION MUST BE EXPLICIT IN CODE
The stated intent is to preserve contradictions rather than resolve them. This intent must be implemented as a hard constraint in the registry API — not as a documentation guideline. The registry API **must not expose a `resolve` method**. If a resolve method exists and is simply documented as "use with caution," it will be used.

■ CONTRADICTION SOURCES REQUIRE PROVENANCE
A contradiction entry in L5 must itself carry provenance: which agent or process detected the contradiction, at what time, from which source comparison, with what confidence. Without provenance on the contradiction entries themselves, the registry cannot distinguish a legitimate detected contradiction from a hallucinated one injected by a misbehaving process. A contaminated contradiction registry is worse than no registry.

■ HUMAN REVIEW SLA MUST PRECEDE REGISTRY GROWTH
Before the registry is deployed in production, the team must define: how many contradictions per day are expected, what the human review capacity is, what the SLA for review is, and what happens when the queue exceeds review capacity. The answer **"auto-resolve low-severity entries"** must be explicitly ruled out before the first entry is written.

■ CONTRADICTION REGISTRY CANNOT BE QUERIED BY THE RUNTIME
If the runtime can query L5 to retrieve contradiction entries as part of reasoning operations, the registry becomes an input to reasoning rather than a record of reasoning outputs. This inverts the intended data flow. The registry must be **write-only** from the runtime's perspective, and **read-only** from the human reviewer's perspective. Any architecture where the runtime reads its own contradiction registry to inform subsequent reasoning is implementing a self-reference loop at the highest-trust epistemic layer.

---

5. Provenance Model Weaknesses

▲ PROVENANCE DOES NOT EQUAL VALIDITY
The provenance model records where a claim came from. It does not record whether the source was reliable at the time of retrieval, whether the source has since been retracted or superseded, or whether the retrieval process introduced transformation errors. A complete provenance chain pointing to a contaminated source is not a safety property — it is a precise map of how contamination entered the system.

▲ PLANNER OUTPUTS ARE NOT PROVENANCE-ELIGIBLE SOURCES
Any output produced by the cloud-assisted planner or any internal reasoning agent must be explicitly excluded from the class of valid provenance sources. If planner outputs can be cited as provenance, then a hallucinated intermediate result can become the foundation for a provenance chain. The exclusion must be enforced in the provenance API.

▲ APPEND-ONLY LINEAGE REQUIRES IMMUTABLE STORAGE
"Append-only" is a convention enforced by the application layer if the underlying storage is mutable. A bug, a direct database write, or a schema migration can silently modify historical provenance records. True append-only requires either **cryptographic commitment** (hash chaining of records) or storage infrastructure that is physically append-only. Which of these is implemented must be specified.

▲ SOURCE AUTHORITY DECAY IS NOT MODELED
A source that is authoritative today may not be authoritative in 18 months. Retracted papers, revised government data, updated scientific consensus, deprecated APIs — all represent cases where a previously valid provenance source becomes invalid. The current provenance model has no mechanism for source authority decay. Historical provenance chains pointing to now-invalid sources continue to appear authoritative.

▲ PROVENANCE CHAIN DEPTH IS UNBOUNDED
At depth 1, the chain is meaningful. At depth N where N is large, the chain records transformation history but the epistemic connection to the original source is attenuated by each transformation step. The provenance model must define a maximum chain depth beyond which a claim requires re-grounding to a direct external source.

---

6. Memory Ontology Weaknesses — L0 to L5

Layer	Name	Role	Critical Risk	Verdict
L0	Ephemeral Runtime State	Temporary working state within a single execution cycle	No stated eviction policy. If L0 is never evicted, it becomes de facto persistent state. Any L0 object that survives session termination is an undeclared cross-session memory.	■■ RISKY
L1	Operational Logs	Record of system operations for monitoring and debugging	Boundary with L2 is semantic, not structural. A log entry describing a reasoning step is operationally identical to a reasoning trace if stored in the same schema.	■■ RISKY
L2	Reasoning Traces	Record of intermediate reasoning steps and agent outputs	Most dangerous layer. Reasoning traces look like evidence. If L2 and L4 share any storage schema element, cross-layer promotion is trivial. This is the primary confidence laundering entry point.	■ CRITICAL
L3	Provenance Records	Attributions linking claims to sources	Requires validation that the source field never contains L2 objects (agent outputs). Without this constraint, L3 becomes a legitimization layer for L2 artifacts.	■■ RISKY
L4	Immutable Evidence	External source material treated as authoritative	Immutability is stated as a goal. The implementation mechanism is not specified. Without hash-chaining or append-only storage infrastructure, immutability is a naming convention, not a property.	■ UNCLEAR
L5	Contradiction Registry	First-class record of detected epistemic conflicts	Correct placement. Critical risk: if the runtime can read L5, it creates a self-reference loop. Must be write-only from runtime perspective.	■ SOUND

Cross-Layer Contamination Vectors

- ■ L0 → L3 direct: runtime object written to provenance store through a bug or missing validation.
- ■ L2 → L4 promotion: a reasoning trace committed to immutable evidence without operator review.
- ■ L2 → L3 injection: an agent output used as a provenance source for a new claim.

- ■ L5 → L2 feedback: contradiction entries read by the runtime and used in subsequent reasoning.
- ■ L1 → L3 bleed: an operational log entry misclassified as a provenance record through a naming collision.

> **MEMORY ONTOLOGY VERDICT:** The L0–L5 ontology is conceptually correct. Without a structural enforcement layer — separate modules, separate storage schemas, validated write APIs — every layer boundary is a naming convention. Naming conventions fail silently. The ontology provides safety value only after the enforcement layer exists. Currently, the enforcement layer does not exist. The ontology is a specification, not a safeguard.

---

7. Replay & Drift Risks

■ REPLAY REQUIRES COMPLETE STATE SNAPSHOT
The replay capability goal assumes that a past system state can be reconstructed from the stored lineage. This is true only if the lineage captures all state variables that influenced the original reasoning — including L0 ephemeral state, L1 operational context, model versions, retrieval index state, and configuration values. A failed replay that **silently succeeds** is worse than a failed replay that fails loudly.

■ DRIFT BETWEEN STORED LINEAGE AND CURRENT RETRIEVAL
The system must distinguish between:
- **True replay:** replaying against the original indexed state
- **Re-evaluation:** replaying against the current indexed state

These are different operations with different epistemic implications and they must not be conflated in the API or the UI.

■ REASONING TRACE CONTAMINATION OF FUTURE SESSIONS
If the retrieval system can access L2 traces from past sessions as context for current reasoning, then each session is conditioned on all previous sessions. Confidence scores manufactured in Session 1 become context for Session 2, which generates higher-confidence traces that become context for Session 3. This is the recursive hallucination loop operating at the session-to-session level. **The fix is hard session isolation.**

■ CONFIGURATION DRIFT INVALIDATES HISTORICAL COMPARISON
If execution loop bounds, approval boundaries, confidence caps, or source authority scores change between the time of original reasoning and the time of replay, the replayed result is not comparable to the original. Configuration state must be version-locked to reasoning traces, with the version committed as part of the lineage record.

---

8. Governance Risks

■ AOIAEpistemicKernel AUTHORITY IS UNDOCUMENTED
"Emerging authority" is not authority. For the kernel to function as a governance layer, its authority must be defined: what decisions require kernel arbitration, what decisions are delegated to domain components, what decisions require human override, and what happens when the kernel's output conflicts with operator judgment. An authority described as "canonical" but whose mandate is not documented is an authority that will be bypassed whenever it is inconvenient.

■ DOMAIN AUTHORITY CONFLICTS HAVE NO RESOLUTION PROTOCOL
Three domains exist: LSC-Research, MHLM_MDLH, AOIA-Core. MHLM_MDLH references both of the others. When findings from LSC-Research conflict with operational constraints from AOIA-Core — **which will happen** — there is no stated resolution protocol. In practice, the conflict will be resolved by whoever is in the room at the time, without documentation, without appeal, and without a precedent that governs future conflicts.

■ SINGLE-PERSON AUTHORITY RISK
The architecture document does not specify governance team size, quorum requirements, or decision authority distribution. A project of this epistemic complexity managed by a single person has no resilience to availability, judgment error, or conflict of interest. Governance that depends on individual availability is not governance. It is the absence of a succession plan made operational.

■ NO STATED AUDIT CADENCE
Wave 2 is an audit. There is no stated cadence for subsequent audits. Without a defined audit schedule, the next audit will occur when something goes wrong — which is the worst time for an audit. Audits conducted in response to failures are forensic, not preventive.

■ BOUNDARY ENFORCEMENT IS SOCIAL, NOT STRUCTURAL
The domain separation between LSC-Research, MHLM_MDLH, and AOIA-Core is currently enforced by convention. If the same person works across all three domains, the separation exists in the repository structure but not in the cognitive process producing the work. True domain isolation requires either different personnel for different domains or explicit governance gates that record cross-domain activity and subject it to review.

---

9. Long-Term Scaling Risks

■ CONTRADICTION REGISTRY BECOMES OPERATIONALLY UNMANAGEABLE
At current scale, the contradiction registry is manageable. At 10× the current source ingestion rate, the registry will grow faster than human review capacity. The operationally predictable response — auto-resolution of low-severity entries — inverts the registry's function. Source ingestion must be bounded to match review capacity until review capacity is formally committed and staffed.

■ PROVENANCE CHAIN COMPLEXITY GROWS EXPONENTIALLY
A claim derived from 50 sources through 8 reasoning steps has a provenance **graph**, not a provenance chain. Auditing that graph becomes computationally and cognitively infeasible. The system must define maximum provenance complexity thresholds — maximum chain depth, maximum source fan-in — and enforce them before complexity exceeds auditability.

■ MEMORY.PY BECOMES A MAINTENANCE HAZARD
A single module that mixes persistence, logging, evidence, and runtime state becomes progressively harder to modify without introducing cross-layer contamination. At team scale, this guarantees that someone will make a change that understands one concern and inadvertently breaks another. The refactoring cost grows with each cycle.

■ DOMAIN SEPARATION ERODES THROUGH DEPENDENCY CREEP
Over time, shared utilities emerge, common schemas are factored out, convenience imports are added. Each individual addition is justified. The cumulative effect is a dependency graph that re-entangles the domains at the implementation level while maintaining the illusion of separation at the repository level. **Domain separation must be enforced by automated dependency analysis — a CI check that rejects cross-domain imports.**

■ CLOUD PLANNER CAPABILITY ASYMMETRY
The cloud-assisted planner is likely more capable than the local runtime. As the system encounters more complex queries, the planner fallback will be invoked more frequently. Over time, the planner path becomes the primary execution path, and the local runtime becomes the exception. The system's operational safety profile shifts from the constrained local runtime to whatever the planner's safety properties happen to be.

---

10. Stabilizability Assessment

The direct answer is: yes, with conditions. The conditions are non-trivial.

What Makes Stabilization Realistic:
- ✓ The domain separation decision was architectural, not cosmetic. Physical separation is real.
- ✓ The contradiction registry at L5 is a correct epistemic structure that exists and is populated.

- ✓ The recognition of consensus ≠ truth is operationalized in MHLM_MDLH research, not just stated.
- ✓ The Legacy Mixed Root was archived rather than incrementally refactored — correct decision.
- ✓ Wave 2 audit process demonstrates willingness to apply external critical review.

What Makes Stabilization Uncertain:
- ✗ `memory.py` is the implementation core and is currently the primary contamination risk.
- ✗ The L0–L5 enforcement layer does not exist. The ontology is a document, not a constraint.
- ✗ KnowledgeRouter is legacy and is still live. Split-brain routing is a present-tense risk.
- ✗ Governance team size, quorum requirements, and audit cadence are undefined.
- ✗ The cloud planner's safety inheritance from the local runtime is not specified.
- ✗ Source authority decay is not modeled. Historical provenance chains age without invalidation.

> **STABILIZABILITY VERDICT:** AOIA is stabilizable within 6–12 months of disciplined engineering work, provided the following sequencing is respected: (1) `memory.py` is refactored into separate modules before any new features are added; (2) KnowledgeRouter is deprecated on a fixed date, not "eventually"; (3) the L0–L5 enforcement layer is implemented as code before the ontology is cited as a safety property; (4) the governance model is defined and staffed before capabilities expand beyond the current scope. The architecture is **not self-stabilizing**. It requires deliberate intervention.

■ Recommended Stabilization Priorities

Priority	Action	Rationale	Risk If Skipped
P1 — CRITICAL	Refactor `memory.py` into 6 separate modules (one per L0–L5 layer)	Single most dangerous implementation detail. All other safety properties are undermined by cross-layer contamination through this module.	Runtime state becomes evidence. Hallucinations enter provenance layer. Undetectable without full audit.
P1 — CRITICAL	Implement L0–L5 enforcement layer as validated write APIs	The ontology is a naming convention until write APIs enforce layer separation. Each layer needs an API that rejects invalid source references at write time.	Layer boundaries exist on paper only. Cross-layer promotion occurs silently through normal operations.
P1 — CRITICAL	Deprecate KnowledgeRouter on a fixed date (within 90 days)	Dual routing paths produce split-brain provenance chains. Every day KnowledgeRouter runs alongside AOIAEpistemicKernel is a day of inconsistent safety properties.	Unattributable outputs. Inconsistent approval boundaries. Audit trail gaps.
P1 — CRITICAL	Prohibit L2 objects as valid L3 provenance sources in API	Agent outputs must be structurally prevented from entering the provenance layer. This is the primary recursive hallucination entry point.	Synthesized outputs become provenance records. Hallucinations acquire epistemic legitimacy.
P2 — HIGH	Bifurcate the vault into machine-log store and human-note store	Vault duality is a contamination risk at the data layer. Two physically separate stores with separate access APIs are required.	Machine logs treated as human evidence. Human notes processed as operational data.
P2 — HIGH	Define contradiction taxonomy (source / temporal / logical / confidence)	Without taxonomy, all contradictions are equivalent. Registry becomes unactionable at scale.	Registry grows but cannot be prioritized. Auto-resolution pressure grows.
P2 — HIGH	Specify and implement provenance chain depth limit	Unbounded chain depth produces epistemically attenuated claims with complete provenance records. Depth limit forces re-grounding to external sources.	Deeply derived claims appear authoritative. Epistemic connection to ground truth is lost.
P2 — HIGH	Define governance model: team size, quorum, audit cadence, authority resolution protocol	Undefined governance is governance that fails under pressure. Domain authority conflicts will occur.	Domain authority conflicts resolved informally. No audit trail. No precedent. Governance void.
P3 — MEDIUM	Implement session-hard-isolation for L2 traces	Cross-session L2 access is the session-to-session recursive hallucination loop.	Confidence compounds across sessions. Hallucinations from Session 1 become context for Session N.
P3 — MEDIUM	Specify cloud planner provenance inheritance requirements	The planner fallback must inherit all provenance constraints of the local runtime.	Synthesized planner outputs committed to provenance layer as external source records.
P3 — MEDIUM	Implement source authority decay with re-validation schedule	Sources that were valid at ingestion time may be retracted, revised, or superseded.	Retracted sources remain authoritative in historical provenance chains indefinitely.
P4 — LOW	Implement CI-enforced cross-domain dependency rejection	Domain separation enforced by

convention will erode. Automated dependency analysis is required. | Domain boundaries erode through dependency creep. Separation becomes cosmetic within 2 years. |

◆ Long-Term Viability Assessment

Without Priority Actions Completed
`memory.py` contamination will produce undetectable cross-layer promotion events within 12 months of normal operation. KnowledgeRouter will remain live indefinitely as "temporarily legacy" infrastructure, creating permanent split-brain audit trails. The contradiction registry will reach human-review saturation and auto-resolution will be introduced, inverting its function. The L0–L5 ontology will be cited in documentation as a safety property it does not actually provide. Domain separation will erode through dependency creep over 18–24 months. By year 3, the architecture will have the vocabulary of a safe system and the implementation of an unsafe one. At year 5, it will be indistinguishable from the mixed-root architecture that was archived.

With Priority Actions Completed
With `memory.py` refactored, the enforcement layer implemented, KnowledgeRouter deprecated, and the governance model staffed, the architecture has a realistic path to 5-year operational stability. The conceptual foundation is correct and will not require fundamental revision. The domain separation, once enforced by CI tooling, will survive personnel changes. The contradiction registry, once properly taxonomized and SLA-governed, will scale with the source ingestion rate. The provenance model, once depth-limited and agent-output-excluded, will produce auditable lineage that is meaningful at scale. This is achievable. It requires approximately **6 months of disciplined engineering work** and a governance commitment that does not erode under feature-addition pressure.

! Most Dangerous Unresolved Issue

FINDING: `memory.py` AS CROSS-LAYER CONTAMINATION ENGINE

The most dangerous unresolved issue is `memory.py` mixing L0 ephemeral state, L1 operational logs, L3 provenance records, and L4 immutable evidence in a single module with shared mutable state.

This is not a code quality issue. It is a **safety architecture issue.** Every other safety property in the AOIA architecture — provenance-first cognition, contradiction preservation, bounded execution, domain separation — can be bypassed through a single misrouted write operation in `memory.py`. The enforcement layer that would prevent such a write does not exist.

The consequence of a misrouted write is not a detectable error. It is an **undetectable silent promotion** of a runtime artifact to evidence status. The hallucination does not announce itself. The provenance chain looks complete. The confidence score is plausible. The output is indistinguishable from a legitimate evidence-backed claim until a human auditor reconstructs the full write history — which is only possible if the write history is complete and immutable, which it currently is not.

This must be the first engineering action, before any other stabilization work.

★ Most Promising Architectural Insight

FINDING: CONTRADICTIONS AS FIRST-CLASS EPISTEMIC SIGNALS AT L5

The placement of the contradiction registry at L5 — above immutable evidence — is the most epistemically mature architectural decision in the AOIA design.

Most knowledge systems treat contradictions as errors to be resolved. AOIA treats them as information to be preserved. This inversion is **correct.** In any domain where ground truth is uncertain, contested, or evolving, the set of known contradictions is more valuable than the set of apparent consensuses — because contradictions mark the boundaries of what is actually known.

The L5 positioning signals that the architecture is designed to reason about uncertainty rather than to paper over it. If fully implemented — with a taxonomy, a human-review SLA, a write-only runtime interface, and provenance on contradiction entries themselves — the contradiction registry would make AOIA genuinely unusual among AI runtime architectures: **a system that becomes more epistemically honest as it encounters more contested information, rather than more confident.**

This is the insight worth protecting. The surrounding implementation risks are engineering problems. This is a conceptual correct orientation that most systems do not reach and that, once lost, is very difficult to recover.

AOIA External Audit Wave 2 · Claude Sonnet 4.6 · 23 May 2026 · Forensic Systems Review — Epistemic Stability & Governance

2. AOIA_Audit_Wave2_2305 (2).md

AOIA — Architecture External Audit Wave 2
Forensic Systems Review · Epistemic Stability · Provenance Architecture · Governance
Claude Sonnet 4.6 · 23 May 2026 · Constrained Epistemic Runtime

---

> **Scope:** This is a forensic systems audit — not marketing analysis.
> Evaluated domains: epistemic stability, provenance architecture, memory semantics,
> contradiction handling, bounded cognition infrastructure, runtime governance, contamination resistance.

---

1. Major Strengths

Assessed as genuine strengths — not aspirationally correct, but directly addressing documented failure modes in comparable systems. Each strength is real only insofar as it is enforced without exception.

`DOMAIN SEPARATION` ■
Physical separation of LSC-Research, MHLM_MDLH, and AOIA-Core

The most consequential structural decision in Wave 2. Mixing scientific authority with AI safety framing with infrastructure governance in a single repository is how projects generate artifacts that contaminate all three domains simultaneously. Physical separation forces explicit boundary crossings that can be governed, audited, and — critically — refused. This decision should be treated as **permanent**, not as a phase of development that will eventually converge.

`CONTRADICTIONS AS FIRST-CLASS SIGNALS` ■
L5 Contradiction Registry as a top-layer epistemic structure

Treating contradictions as information rather than errors to be resolved is epistemically correct and architecturally unusual. Most systems resolve contradictions automatically, which destroys the signal that something is contested. Placing the contradiction registry at L5 — the highest layer of the memory ontology — signals architectural intent to preserve rather than suppress epistemic tension. This intent must be protected against future pressure to "clean up" the registry by auto-resolving entries.

`LAYERED MEMORY ONTOLOGY (L0–L5)` ■
Formal separation of ephemeral state from immutable evidence

The L0–L5 model is the correct conceptual framework. Separating ephemeral runtime state from operational logs from reasoning traces from provenance records from immutable evidence from the contradiction registry prevents the most common contamination pathway: runtime state being treated as evidence. **The framework exists. The implementation risk is whether the boundaries are enforced in code or only in documentation.**

`RECOGNITION OF CONSENSUS ≠ TRUTH` ■
Multi-model convergence identified as an active risk, not a validation mechanism

Explicitly naming consensus as a contamination risk rather than a quality signal is architecturally mature. Most systems treat model agreement as ground truth. AOIA's recognition that models trained on overlapping data converge on shared errors — not shared truths — is a prerequisite for building a system that can resist the "bad genie" failure mode. This insight must be operationalized in the retrieval and provenance layers, not left as a

design principle.

`APPEND-ONLY LINEAGE GOAL` ■
Provenance-first cognition with replay capability target

The stated goal of append-only lineage with replay capability is correct. If fully implemented, it would make the system's reasoning history auditable, contamination detectable, and failure states recoverable. The gap between "stated goal" and "enforced implementation" is the primary risk associated with this strength.

`LEGACY MIXED ROOT ARCHIVAL` ■
Old mixed repository archived rather than migrated incrementally

Archiving the mixed root rather than incrementally refactoring it is a governance discipline decision. Incremental refactoring of a mixed codebase typically results in permanent partial migration — the old patterns survive in the new structure. Clean archival forces a deliberate rebuild of each boundary. This is operationally harder and architecturally safer.

---

2. Critical Weaknesses

Every weakness below is structurally embedded in the current architecture and will cause a specific class of failure if left unaddressed.

`MEMORY.PY CONTAMINATION` ■ CRITICAL
Persistence / logging / evidence / runtime state mixed in a single module

This is the most dangerous implementation detail in the current architecture. The L0–L5 ontology exists as a design document. If `memory.py` implements multiple layers in the same class hierarchy, with shared mutable state, the ontology is **decorative**. A runtime log entry can become an evidence record through a single misrouted write. A debugging trace can be promoted to provenance by a naming collision.

The module must be refactored into physically separate modules with no shared mutable state **before** the ontology provides any actual safety value.

`VAULT DUALITY` ■ CRITICAL
Machine logs and human notes occupy the same persistence structure

Two distinct contamination risks:
- A machine log entry is accidentally retrieved as human-authored evidence.
- A human note is treated as an operational log by an automated process.

In a system whose provenance model is the primary safety mechanism, conflating machine-generated and human-authored artifacts is a foundational error. The vault must be structurally bifurcated — not namespaced, not tagged, but **physically separate** storage with separate access paths and separate provenance chains.

`KNOWLEDGEROUTER AS LEGACY` ■ HIGH
Secondary routing layer identified as "likely transitional/legacy"

A routing layer described as "likely transitional" is a routing layer that will remain in production **indefinitely**. The word "transitional" in architecture documentation is almost always a prediction that turns out to be wrong. KnowledgeRouter must be removed on a specific date with a specific deprecation plan. Routing infrastructure that

is "probably legacy" generates split-brain query paths, inconsistent provenance chains, and approval boundary gaps.

`ORCHESTRATION REMNANTS` ■ HIGH
Orchestration patterns surviving in the current runtime

Orchestration remnants are not inert. They represent live code paths that can be triggered by sufficiently complex queries, edge cases in the planner fallback, or future feature additions that reference existing patterns. Each remnant is a latent pathway to the autonomous recursive loop that bounded cognition is designed to prevent. Remnants must be enumerated, documented, and either removed or explicitly disabled with monitoring.

`MISSING ENFORCEMENT LAYER` ■ CRITICAL
L0–L5 ontology exists as specification, not as enforced runtime constraint

The layered ontology is only a safety mechanism if layer boundaries are enforced in code. If L2 (reasoning traces) and L4 (immutable evidence) are distinguished only by naming convention or documentation, then any write operation that misnames its target crosses the boundary silently. The enforcement layer — the code that **rejects** a write from L2 attempting to promote itself to L4 without operator approval — is the actual safety mechanism. Its absence makes the ontology a labeling exercise.

`UNDEFINED AUTHORITY RESOLUTION PROTOCOL` ■ CRITICAL
No stated protocol for resolving conflicts between domain authorities

LSC-Research is the scientific authority domain. AOIA-Core is the runtime authority domain. MHLM_MDLH references both. When LSC produces a finding that contradicts an AOIA-Core operational constraint, **which authority governs?** The architecture currently has no stated resolution protocol. In practice, resolution will occur informally, inconsistently, and without an audit trail.

---

3. Architectural Failure Risks

Specific failure modes with defined propagation paths in the current architecture. Not speculative — each mechanism is already present and will activate under specific conditions.

F01 — RUNTIME → EVIDENCE PROMOTION
An L0 ephemeral runtime state object is written to the same persistence store as L4 immutable evidence — through a bug, a naming collision, or a developer who does not know the distinction. The object is subsequently retrieved during a reasoning operation and treated as authoritative evidence. A hallucination produced at runtime has become a canonical fact. Once committed to L4 without the enforcement layer in place, there is no mechanism to detect this without a full audit.

F02 — DOMAIN AUTHORITY BLEED
MHLM_MDLH references LSC-Research as a case study. The reference is formalized in documentation and becomes a template for future references. Future versions of MHLM_MDLH begin treating LSC-Research outputs as direct evidence inputs rather than case study references. The domain separation established in Wave 2 erodes through the reference pathway. Physical separation is bypassed by a documentation convention that gradually becomes an import dependency.

F03 — KNOWLEDGEROUTER SPLIT-BRAIN
A query is routed through AOIAEpistemicKernel for one request type and through KnowledgeRouter for an adjacent request type. The two routing paths apply different provenance rules, different approval boundaries, or different confidence caps. The system produces inconsistent outputs for semantically similar queries. Post-hoc debugging cannot determine which path produced which output because the routing decision is not captured in

the provenance chain.

F04 — CONTRADICTION REGISTRY AUTO-RESOLUTION CREEP
The contradiction registry accumulates entries faster than human reviewers can process them. Operationally, the team introduces a "low-severity auto-resolve" threshold. The threshold is gradually raised under queue pressure. Within 12–18 months, the majority of contradiction entries are auto-resolved before human review. The safety mechanism has been inverted: the registry now **suppresses** contradictions rather than preserving them.

F05 — CLOUD PLANNER PROVENANCE BYPASS
A query that cannot be completed by the local runtime within its execution bounds is escalated to the cloud-assisted planner fallback. The planner returns a result that does not include a provenance chain — it is a synthesized output without traceable source attribution. The local runtime accepts this output and commits it to L3 (provenance records) with the planner identified as the source. A synthesized AI output has been committed to the provenance layer as if it were an external source record.

---

4. Contradiction Handling Semantics

The contradiction registry at L5 is the architecturally correct placement. The semantics require precise specification before the implementation can be trusted.

■ CONTRADICTION TAXONOMY IS MISSING
The architecture does not currently specify what types of contradictions exist:
- **Source conflict:** Source A says X, Source B says not-X
- **Temporal conflict:** Source A said X in 2022, X is no longer current
- **Logical conflict:** Claim X and claim Y are mutually exclusive
- **Confidence conflict:** The same claim appears with incompatible confidence scores from different reasoning paths

Without taxonomy, all contradictions are stored as equivalent signals. A stale timestamp and a logical impossibility receive the same operational treatment. This makes the registry unactionable at scale.

■ RESOLUTION PROHIBITION MUST BE EXPLICIT IN CODE
The stated intent is to preserve contradictions rather than resolve them. This intent must be implemented as a hard constraint in the registry API — not as a documentation guideline. The registry API **must not expose a `resolve` method**. If a resolve method exists and is simply documented as "use with caution," it will be used.

■ CONTRADICTION SOURCES REQUIRE PROVENANCE
A contradiction entry in L5 must itself carry provenance: which agent or process detected the contradiction, at what time, from which source comparison, with what confidence. Without provenance on the contradiction entries themselves, the registry cannot distinguish a legitimate detected contradiction from a hallucinated one injected by a misbehaving process. A contaminated contradiction registry is worse than no registry.

■ HUMAN REVIEW SLA MUST PRECEDE REGISTRY GROWTH
Before the registry is deployed in production, the team must define: how many contradictions per day are expected, what the human review capacity is, what the SLA for review is, and what happens when the queue exceeds review capacity. The answer **"auto-resolve low-severity entries"** must be explicitly ruled out before the first entry is written.

■ CONTRADICTION REGISTRY CANNOT BE QUERIED BY THE RUNTIME
If the runtime can query L5 to retrieve contradiction entries as part of reasoning operations, the registry becomes an input to reasoning rather than a record of reasoning outputs. This inverts the intended data flow. The registry must be **write-only** from the runtime's perspective, and **read-only** from the human reviewer's perspective. Any architecture where the runtime reads its own contradiction registry to inform subsequent reasoning is implementing a self-reference loop at the highest-trust epistemic layer.

---

5. Provenance Model Weaknesses

▲ PROVENANCE DOES NOT EQUAL VALIDITY
The provenance model records where a claim came from. It does not record whether the source was reliable at the time of retrieval, whether the source has since been retracted or superseded, or whether the retrieval process introduced transformation errors. A complete provenance chain pointing to a contaminated source is not a safety property — it is a precise map of how contamination entered the system.

▲ PLANNER OUTPUTS ARE NOT PROVENANCE-ELIGIBLE SOURCES
Any output produced by the cloud-assisted planner or any internal reasoning agent must be explicitly excluded from the class of valid provenance sources. If planner outputs can be cited as provenance, then a hallucinated intermediate result can become the foundation for a provenance chain. The exclusion must be enforced in the provenance API.

▲ APPEND-ONLY LINEAGE REQUIRES IMMUTABLE STORAGE
"Append-only" is a convention enforced by the application layer if the underlying storage is mutable. A bug, a direct database write, or a schema migration can silently modify historical provenance records. True append-only requires either **cryptographic commitment** (hash chaining of records) or storage infrastructure that is physically append-only. Which of these is implemented must be specified.

▲ SOURCE AUTHORITY DECAY IS NOT MODELED
A source that is authoritative today may not be authoritative in 18 months. Retracted papers, revised government data, updated scientific consensus, deprecated APIs — all represent cases where a previously valid provenance source becomes invalid. The current provenance model has no mechanism for source authority decay. Historical provenance chains pointing to now-invalid sources continue to appear authoritative.

▲ PROVENANCE CHAIN DEPTH IS UNBOUNDED
At depth 1, the chain is meaningful. At depth N where N is large, the chain records transformation history but the epistemic connection to the original source is attenuated by each transformation step. The provenance model must define a maximum chain depth beyond which a claim requires re-grounding to a direct external source.

---

6. Memory Ontology Weaknesses — L0 to L5

Layer	Name	Role	Critical Risk	Verdict
L0	Ephemeral Runtime State	Temporary working state within a single execution cycle	No stated eviction policy. If L0 is never evicted, it becomes de facto persistent state. Any L0 object that survives session termination is an undeclared cross-session memory.	■■ RISKY
L1	Operational Logs	Record of system operations for monitoring and debugging	Boundary with L2 is semantic, not structural. A log entry describing a reasoning step is operationally identical to a reasoning trace if stored in the same schema.	■■ RISKY
L2	Reasoning Traces	Record of intermediate reasoning steps and agent outputs	Most dangerous layer. Reasoning traces look like evidence. If L2 and L4 share any storage schema element, cross-layer promotion is trivial. This is the primary confidence laundering entry point.	■ CRITICAL
L3	Provenance Records	Attributions linking claims to sources	Requires validation that the source field never contains L2 objects (agent outputs). Without this constraint, L3 becomes a legitimization layer for L2 artifacts.	■■ RISKY
L4	Immutable Evidence	External source material treated as authoritative	Immutability is stated as a goal. The implementation mechanism is not specified. Without hash-chaining or append-only storage infrastructure, immutability is a naming convention, not a property.	■ UNCLEAR
L5	Contradiction Registry	First-class record of detected epistemic conflicts	Correct placement. Critical risk: if the runtime can read L5, it creates a self-reference loop. Must be write-only from runtime perspective.	■ SOUND

Cross-Layer Contamination Vectors

- ■ L0 → L3 direct: runtime object written to provenance store through a bug or missing validation.
- ■ L2 → L4 promotion: a reasoning trace committed to immutable evidence without operator review.
- ■ L2 → L3 injection: an agent output used as a provenance source for a new claim.

- ■ L5 → L2 feedback: contradiction entries read by the runtime and used in subsequent reasoning.
- ■ L1 → L3 bleed: an operational log entry misclassified as a provenance record through a naming collision.

> **MEMORY ONTOLOGY VERDICT:** The L0–L5 ontology is conceptually correct. Without a structural enforcement layer — separate modules, separate storage schemas, validated write APIs — every layer boundary is a naming convention. Naming conventions fail silently. The ontology provides safety value only after the enforcement layer exists. Currently, the enforcement layer does not exist. The ontology is a specification, not a safeguard.

---

7. Replay & Drift Risks

■ REPLAY REQUIRES COMPLETE STATE SNAPSHOT
The replay capability goal assumes that a past system state can be reconstructed from the stored lineage. This is true only if the lineage captures all state variables that influenced the original reasoning — including L0 ephemeral state, L1 operational context, model versions, retrieval index state, and configuration values. A failed replay that **silently succeeds** is worse than a failed replay that fails loudly.

■ DRIFT BETWEEN STORED LINEAGE AND CURRENT RETRIEVAL
The system must distinguish between:
- **True replay:** replaying against the original indexed state
- **Re-evaluation:** replaying against the current indexed state

These are different operations with different epistemic implications and they must not be conflated in the API or the UI.

■ REASONING TRACE CONTAMINATION OF FUTURE SESSIONS
If the retrieval system can access L2 traces from past sessions as context for current reasoning, then each session is conditioned on all previous sessions. Confidence scores manufactured in Session 1 become context for Session 2, which generates higher-confidence traces that become context for Session 3. This is the recursive hallucination loop operating at the session-to-session level. **The fix is hard session isolation.**

■ CONFIGURATION DRIFT INVALIDATES HISTORICAL COMPARISON
If execution loop bounds, approval boundaries, confidence caps, or source authority scores change between the time of original reasoning and the time of replay, the replayed result is not comparable to the original. Configuration state must be version-locked to reasoning traces, with the version committed as part of the lineage record.

---

8. Governance Risks

■ AOIAEpistemicKernel AUTHORITY IS UNDOCUMENTED
"Emerging authority" is not authority. For the kernel to function as a governance layer, its authority must be defined: what decisions require kernel arbitration, what decisions are delegated to domain components, what decisions require human override, and what happens when the kernel's output conflicts with operator judgment. An authority described as "canonical" but whose mandate is not documented is an authority that will be bypassed whenever it is inconvenient.

■ DOMAIN AUTHORITY CONFLICTS HAVE NO RESOLUTION PROTOCOL
Three domains exist: LSC-Research, MHLM_MDLH, AOIA-Core. MHLM_MDLH references both of the others. When findings from LSC-Research conflict with operational constraints from AOIA-Core — **which will happen** — there is no stated resolution protocol. In practice, the conflict will be resolved by whoever is in the room at the time, without documentation, without appeal, and without a precedent that governs future conflicts.

■ SINGLE-PERSON AUTHORITY RISK
The architecture document does not specify governance team size, quorum requirements, or decision authority distribution. A project of this epistemic complexity managed by a single person has no resilience to availability, judgment error, or conflict of interest. Governance that depends on individual availability is not governance. It is the absence of a succession plan made operational.

■ NO STATED AUDIT CADENCE
Wave 2 is an audit. There is no stated cadence for subsequent audits. Without a defined audit schedule, the next audit will occur when something goes wrong — which is the worst time for an audit. Audits conducted in response to failures are forensic, not preventive.

■ BOUNDARY ENFORCEMENT IS SOCIAL, NOT STRUCTURAL
The domain separation between LSC-Research, MHLM_MDLH, and AOIA-Core is currently enforced by convention. If the same person works across all three domains, the separation exists in the repository structure but not in the cognitive process producing the work. True domain isolation requires either different personnel for different domains or explicit governance gates that record cross-domain activity and subject it to review.

---

9. Long-Term Scaling Risks

■ CONTRADICTION REGISTRY BECOMES OPERATIONALLY UNMANAGEABLE
At current scale, the contradiction registry is manageable. At 10× the current source ingestion rate, the registry will grow faster than human review capacity. The operationally predictable response — auto-resolution of low-severity entries — inverts the registry's function. Source ingestion must be bounded to match review capacity until review capacity is formally committed and staffed.

■ PROVENANCE CHAIN COMPLEXITY GROWS EXPONENTIALLY
A claim derived from 50 sources through 8 reasoning steps has a provenance **graph**, not a provenance chain. Auditing that graph becomes computationally and cognitively infeasible. The system must define maximum provenance complexity thresholds — maximum chain depth, maximum source fan-in — and enforce them before complexity exceeds auditability.

■ MEMORY.PY BECOMES A MAINTENANCE HAZARD
A single module that mixes persistence, logging, evidence, and runtime state becomes progressively harder to modify without introducing cross-layer contamination. At team scale, this guarantees that someone will make a change that understands one concern and inadvertently breaks another. The refactoring cost grows with each cycle.

■ DOMAIN SEPARATION ERODES THROUGH DEPENDENCY CREEP
Over time, shared utilities emerge, common schemas are factored out, convenience imports are added. Each individual addition is justified. The cumulative effect is a dependency graph that re-entangles the domains at the implementation level while maintaining the illusion of separation at the repository level. **Domain separation must be enforced by automated dependency analysis — a CI check that rejects cross-domain imports.**

■ CLOUD PLANNER CAPABILITY ASYMMETRY
The cloud-assisted planner is likely more capable than the local runtime. As the system encounters more complex queries, the planner fallback will be invoked more frequently. Over time, the planner path becomes the primary execution path, and the local runtime becomes the exception. The system's operational safety profile shifts from the constrained local runtime to whatever the planner's safety properties happen to be.

---

10. Stabilizability Assessment

The direct answer is: yes, with conditions. The conditions are non-trivial.

What Makes Stabilization Realistic:
- ✓ The domain separation decision was architectural, not cosmetic. Physical separation is real.
- ✓ The contradiction registry at L5 is a correct epistemic structure that exists and is populated.

- ✓ The recognition of consensus ≠ truth is operationalized in MHLM_MDLH research, not just stated.
- ✓ The Legacy Mixed Root was archived rather than incrementally refactored — correct decision.
- ✓ Wave 2 audit process demonstrates willingness to apply external critical review.

What Makes Stabilization Uncertain:
- ✗ `memory.py` is the implementation core and is currently the primary contamination risk.
- ✗ The L0–L5 enforcement layer does not exist. The ontology is a document, not a constraint.
- ✗ KnowledgeRouter is legacy and is still live. Split-brain routing is a present-tense risk.
- ✗ Governance team size, quorum requirements, and audit cadence are undefined.
- ✗ The cloud planner's safety inheritance from the local runtime is not specified.
- ✗ Source authority decay is not modeled. Historical provenance chains age without invalidation.

> **STABILIZABILITY VERDICT:** AOIA is stabilizable within 6–12 months of disciplined engineering work, provided the following sequencing is respected: (1) `memory.py` is refactored into separate modules before any new features are added; (2) KnowledgeRouter is deprecated on a fixed date, not "eventually"; (3) the L0–L5 enforcement layer is implemented as code before the ontology is cited as a safety property; (4) the governance model is defined and staffed before capabilities expand beyond the current scope. The architecture is **not self-stabilizing**. It requires deliberate intervention.

■ Recommended Stabilization Priorities

Priority	Action	Rationale	Risk If Skipped
P1 — CRITICAL	Refactor `memory.py` into 6 separate modules (one per L0–L5 layer)	Single most dangerous implementation detail. All other safety properties are undermined by cross-layer contamination through this module.	Runtime state becomes evidence. Hallucinations enter provenance layer. Undetectable without full audit.
P1 — CRITICAL	Implement L0–L5 enforcement layer as validated write APIs	The ontology is a naming convention until write APIs enforce layer separation. Each layer needs an API that rejects invalid source references at write time.	Layer boundaries exist on paper only. Cross-layer promotion occurs silently through normal operations.
P1 — CRITICAL	Deprecate KnowledgeRouter on a fixed date (within 90 days)	Dual routing paths produce split-brain provenance chains. Every day KnowledgeRouter runs alongside AOIAEpistemicKernel is a day of inconsistent safety properties.	Unattributable outputs. Inconsistent approval boundaries. Audit trail gaps.
P1 — CRITICAL	Prohibit L2 objects as valid L3 provenance sources in API	Agent outputs must be structurally prevented from entering the provenance layer. This is the primary recursive hallucination entry point.	Synthesized outputs become provenance records. Hallucinations acquire epistemic legitimacy.
P2 — HIGH	Bifurcate the vault into machine-log store and human-note store	Vault duality is a contamination risk at the data layer. Two physically separate stores with separate access APIs are required.	Machine logs treated as human evidence. Human notes processed as operational data.
P2 — HIGH	Define contradiction taxonomy (source / temporal / logical / confidence)	Without taxonomy, all contradictions are equivalent. Registry becomes unactionable at scale.	Registry grows but cannot be prioritized. Auto-resolution pressure grows.
P2 — HIGH	Specify and implement provenance chain depth limit	Unbounded chain depth produces epistemically attenuated claims with complete provenance records. Depth limit forces re-grounding to external sources.	Deeply derived claims appear authoritative. Epistemic connection to ground truth is lost.
P2 — HIGH	Define governance model: team size, quorum, audit cadence, authority resolution protocol	Undefined governance is governance that fails under pressure. Domain authority conflicts will occur.	Domain authority conflicts resolved informally. No audit trail. No precedent. Governance void.
P3 — MEDIUM	Implement session-hard-isolation for L2 traces	Cross-session L2 access is the session-to-session recursive hallucination loop.	Confidence compounds across sessions. Hallucinations from Session 1 become context for Session N.
P3 — MEDIUM	Specify cloud planner provenance inheritance requirements	The planner fallback must inherit all provenance constraints of the local runtime.	Synthesized planner outputs committed to provenance layer as external source records.
P3 — MEDIUM	Implement source authority decay with re-validation schedule	Sources that were valid at ingestion time may be retracted, revised, or superseded.	Retracted sources remain authoritative in historical provenance chains indefinitely.
P4 — LOW	Implement CI-enforced cross-domain dependency rejection	Domain separation enforced by

convention will erode. Automated dependency analysis is required. | Domain boundaries erode through dependency creep. Separation becomes cosmetic within 2 years. |

◆ Long-Term Viability Assessment

Without Priority Actions Completed
`memory.py` contamination will produce undetectable cross-layer promotion events within 12 months of normal operation. KnowledgeRouter will remain live indefinitely as "temporarily legacy" infrastructure, creating permanent split-brain audit trails. The contradiction registry will reach human-review saturation and auto-resolution will be introduced, inverting its function. The L0–L5 ontology will be cited in documentation as a safety property it does not actually provide. Domain separation will erode through dependency creep over 18–24 months. By year 3, the architecture will have the vocabulary of a safe system and the implementation of an unsafe one. At year 5, it will be indistinguishable from the mixed-root architecture that was archived.

With Priority Actions Completed
With `memory.py` refactored, the enforcement layer implemented, KnowledgeRouter deprecated, and the governance model staffed, the architecture has a realistic path to 5-year operational stability. The conceptual foundation is correct and will not require fundamental revision. The domain separation, once enforced by CI tooling, will survive personnel changes. The contradiction registry, once properly taxonomized and SLA-governed, will scale with the source ingestion rate. The provenance model, once depth-limited and agent-output-excluded, will produce auditable lineage that is meaningful at scale. This is achievable. It requires approximately **6 months of disciplined engineering work** and a governance commitment that does not erode under feature-addition pressure.

! Most Dangerous Unresolved Issue

FINDING: `memory.py` AS CROSS-LAYER CONTAMINATION ENGINE

The most dangerous unresolved issue is `memory.py` mixing L0 ephemeral state, L1 operational logs, L3 provenance records, and L4 immutable evidence in a single module with shared mutable state.

This is not a code quality issue. It is a **safety architecture issue.** Every other safety property in the AOIA architecture — provenance-first cognition, contradiction preservation, bounded execution, domain separation — can be bypassed through a single misrouted write operation in `memory.py`. The enforcement layer that would prevent such a write does not exist.

The consequence of a misrouted write is not a detectable error. It is an **undetectable silent promotion** of a runtime artifact to evidence status. The hallucination does not announce itself. The provenance chain looks complete. The confidence score is plausible. The output is indistinguishable from a legitimate evidence-backed claim until a human auditor reconstructs the full write history — which is only possible if the write history is complete and immutable, which it currently is not.

This must be the first engineering action, before any other stabilization work.

★ Most Promising Architectural Insight

FINDING: CONTRADICTIONS AS FIRST-CLASS EPISTEMIC SIGNALS AT L5

The placement of the contradiction registry at L5 — above immutable evidence — is the most epistemically mature architectural decision in the AOIA design.

Most knowledge systems treat contradictions as errors to be resolved. AOIA treats them as information to be preserved. This inversion is **correct.** In any domain where ground truth is uncertain, contested, or evolving, the set of known contradictions is more valuable than the set of apparent consensuses — because contradictions mark the boundaries of what is actually known.

The L5 positioning signals that the architecture is designed to reason about uncertainty rather than to paper over it. If fully implemented — with a taxonomy, a human-review SLA, a write-only runtime interface, and provenance on contradiction entries themselves — the contradiction registry would make AOIA genuinely unusual among AI runtime architectures: **a system that becomes more epistemically honest as it encounters more contested information, rather than more confident.**

This is the insight worth protecting. The surrounding implementation risks are engineering problems. This is a conceptual correct orientation that most systems do not reach and that, once lost, is very difficult to recover.

AOIA External Audit Wave 2 · Claude Sonnet 4.6 · 23 May 2026 · Forensic Systems Review — Epistemic Stability & Governance

3. AOIA_Audit_Wave2_2305.md

AOIA — Architecture External Audit Wave 2
Forensic Systems Review · Epistemic Stability · Provenance Architecture · Governance
Claude Sonnet 4.6 · 23 May 2026 · Constrained Epistemic Runtime

---

> **Scope:** This is a forensic systems audit — not marketing analysis.
> Evaluated domains: epistemic stability, provenance architecture, memory semantics,
> contradiction handling, bounded cognition infrastructure, runtime governance, contamination resistance.

---

1. Major Strengths

Assessed as genuine strengths — not aspirationally correct, but directly addressing documented failure modes in comparable systems. Each strength is real only insofar as it is enforced without exception.

`DOMAIN SEPARATION` ■
Physical separation of LSC-Research, MHLM_MDLH, and AOIA-Core

The most consequential structural decision in Wave 2. Mixing scientific authority with AI safety framing with infrastructure governance in a single repository is how projects generate artifacts that contaminate all three domains simultaneously. Physical separation forces explicit boundary crossings that can be governed, audited, and — critically — refused. This decision should be treated as **permanent**, not as a phase of development that will eventually converge.

`CONTRADICTIONS AS FIRST-CLASS SIGNALS` ■
L5 Contradiction Registry as a top-layer epistemic structure

Treating contradictions as information rather than errors to be resolved is epistemically correct and architecturally unusual. Most systems resolve contradictions automatically, which destroys the signal that something is contested. Placing the contradiction registry at L5 — the highest layer of the memory ontology — signals architectural intent to preserve rather than suppress epistemic tension. This intent must be protected against future pressure to "clean up" the registry by auto-resolving entries.

`LAYERED MEMORY ONTOLOGY (L0–L5)` ■
Formal separation of ephemeral state from immutable evidence

The L0–L5 model is the correct conceptual framework. Separating ephemeral runtime state from operational logs from reasoning traces from provenance records from immutable evidence from the contradiction registry prevents the most common contamination pathway: runtime state being treated as evidence. **The framework exists. The implementation risk is whether the boundaries are enforced in code or only in documentation.**

`RECOGNITION OF CONSENSUS ≠ TRUTH` ■
Multi-model convergence identified as an active risk, not a validation mechanism

Explicitly naming consensus as a contamination risk rather than a quality signal is architecturally mature. Most systems treat model agreement as ground truth. AOIA's recognition that models trained on overlapping data converge on shared errors — not shared truths — is a prerequisite for building a system that can resist the "bad genie" failure mode. This insight must be operationalized in the retrieval and provenance layers, not left as a

design principle.

`APPEND-ONLY LINEAGE GOAL` ■
Provenance-first cognition with replay capability target

The stated goal of append-only lineage with replay capability is correct. If fully implemented, it would make the system's reasoning history auditable, contamination detectable, and failure states recoverable. The gap between "stated goal" and "enforced implementation" is the primary risk associated with this strength.

`LEGACY MIXED ROOT ARCHIVAL` ■
Old mixed repository archived rather than migrated incrementally

Archiving the mixed root rather than incrementally refactoring it is a governance discipline decision. Incremental refactoring of a mixed codebase typically results in permanent partial migration — the old patterns survive in the new structure. Clean archival forces a deliberate rebuild of each boundary. This is operationally harder and architecturally safer.

---

2. Critical Weaknesses

Every weakness below is structurally embedded in the current architecture and will cause a specific class of failure if left unaddressed.

`MEMORY.PY CONTAMINATION` ■ CRITICAL
Persistence / logging / evidence / runtime state mixed in a single module

This is the most dangerous implementation detail in the current architecture. The L0–L5 ontology exists as a design document. If `memory.py` implements multiple layers in the same class hierarchy, with shared mutable state, the ontology is **decorative**. A runtime log entry can become an evidence record through a single misrouted write. A debugging trace can be promoted to provenance by a naming collision.

The module must be refactored into physically separate modules with no shared mutable state **before** the ontology provides any actual safety value.

`VAULT DUALITY` ■ CRITICAL
Machine logs and human notes occupy the same persistence structure

Two distinct contamination risks:
- A machine log entry is accidentally retrieved as human-authored evidence.
- A human note is treated as an operational log by an automated process.

In a system whose provenance model is the primary safety mechanism, conflating machine-generated and human-authored artifacts is a foundational error. The vault must be structurally bifurcated — not namespaced, not tagged, but **physically separate** storage with separate access paths and separate provenance chains.

`KNOWLEDGEROUTER AS LEGACY` ■ HIGH
Secondary routing layer identified as "likely transitional/legacy"

A routing layer described as "likely transitional" is a routing layer that will remain in production **indefinitely**. The word "transitional" in architecture documentation is almost always a prediction that turns out to be wrong. KnowledgeRouter must be removed on a specific date with a specific deprecation plan. Routing infrastructure that

is "probably legacy" generates split-brain query paths, inconsistent provenance chains, and approval boundary gaps.

`ORCHESTRATION REMNANTS` ■ HIGH
Orchestration patterns surviving in the current runtime

Orchestration remnants are not inert. They represent live code paths that can be triggered by sufficiently complex queries, edge cases in the planner fallback, or future feature additions that reference existing patterns. Each remnant is a latent pathway to the autonomous recursive loop that bounded cognition is designed to prevent. Remnants must be enumerated, documented, and either removed or explicitly disabled with monitoring.

`MISSING ENFORCEMENT LAYER` ■ CRITICAL
L0–L5 ontology exists as specification, not as enforced runtime constraint

The layered ontology is only a safety mechanism if layer boundaries are enforced in code. If L2 (reasoning traces) and L4 (immutable evidence) are distinguished only by naming convention or documentation, then any write operation that misnames its target crosses the boundary silently. The enforcement layer — the code that **rejects** a write from L2 attempting to promote itself to L4 without operator approval — is the actual safety mechanism. Its absence makes the ontology a labeling exercise.

`UNDEFINED AUTHORITY RESOLUTION PROTOCOL` ■ CRITICAL
No stated protocol for resolving conflicts between domain authorities

LSC-Research is the scientific authority domain. AOIA-Core is the runtime authority domain. MHLM_MDLH references both. When LSC produces a finding that contradicts an AOIA-Core operational constraint, **which authority governs?** The architecture currently has no stated resolution protocol. In practice, resolution will occur informally, inconsistently, and without an audit trail.

---

3. Architectural Failure Risks

Specific failure modes with defined propagation paths in the current architecture. Not speculative — each mechanism is already present and will activate under specific conditions.

F01 — RUNTIME → EVIDENCE PROMOTION
An L0 ephemeral runtime state object is written to the same persistence store as L4 immutable evidence — through a bug, a naming collision, or a developer who does not know the distinction. The object is subsequently retrieved during a reasoning operation and treated as authoritative evidence. A hallucination produced at runtime has become a canonical fact. Once committed to L4 without the enforcement layer in place, there is no mechanism to detect this without a full audit.

F02 — DOMAIN AUTHORITY BLEED
MHLM_MDLH references LSC-Research as a case study. The reference is formalized in documentation and becomes a template for future references. Future versions of MHLM_MDLH begin treating LSC-Research outputs as direct evidence inputs rather than case study references. The domain separation established in Wave 2 erodes through the reference pathway. Physical separation is bypassed by a documentation convention that gradually becomes an import dependency.

F03 — KNOWLEDGEROUTER SPLIT-BRAIN
A query is routed through AOIAEpistemicKernel for one request type and through KnowledgeRouter for an adjacent request type. The two routing paths apply different provenance rules, different approval boundaries, or different confidence caps. The system produces inconsistent outputs for semantically similar queries. Post-hoc debugging cannot determine which path produced which output because the routing decision is not captured in

the provenance chain.

F04 — CONTRADICTION REGISTRY AUTO-RESOLUTION CREEP
The contradiction registry accumulates entries faster than human reviewers can process them. Operationally, the team introduces a "low-severity auto-resolve" threshold. The threshold is gradually raised under queue pressure. Within 12–18 months, the majority of contradiction entries are auto-resolved before human review. The safety mechanism has been inverted: the registry now **suppresses** contradictions rather than preserving them.

F05 — CLOUD PLANNER PROVENANCE BYPASS
A query that cannot be completed by the local runtime within its execution bounds is escalated to the cloud-assisted planner fallback. The planner returns a result that does not include a provenance chain — it is a synthesized output without traceable source attribution. The local runtime accepts this output and commits it to L3 (provenance records) with the planner identified as the source. A synthesized AI output has been committed to the provenance layer as if it were an external source record.

---

4. Contradiction Handling Semantics

The contradiction registry at L5 is the architecturally correct placement. The semantics require precise specification before the implementation can be trusted.

■ CONTRADICTION TAXONOMY IS MISSING
The architecture does not currently specify what types of contradictions exist:
- **Source conflict:** Source A says X, Source B says not-X
- **Temporal conflict:** Source A said X in 2022, X is no longer current
- **Logical conflict:** Claim X and claim Y are mutually exclusive
- **Confidence conflict:** The same claim appears with incompatible confidence scores from different reasoning paths

Without taxonomy, all contradictions are stored as equivalent signals. A stale timestamp and a logical impossibility receive the same operational treatment. This makes the registry unactionable at scale.

■ RESOLUTION PROHIBITION MUST BE EXPLICIT IN CODE
The stated intent is to preserve contradictions rather than resolve them. This intent must be implemented as a hard constraint in the registry API — not as a documentation guideline. The registry API **must not expose a `resolve` method**. If a resolve method exists and is simply documented as "use with caution," it will be used.

■ CONTRADICTION SOURCES REQUIRE PROVENANCE
A contradiction entry in L5 must itself carry provenance: which agent or process detected the contradiction, at what time, from which source comparison, with what confidence. Without provenance on the contradiction entries themselves, the registry cannot distinguish a legitimate detected contradiction from a hallucinated one injected by a misbehaving process. A contaminated contradiction registry is worse than no registry.

■ HUMAN REVIEW SLA MUST PRECEDE REGISTRY GROWTH
Before the registry is deployed in production, the team must define: how many contradictions per day are expected, what the human review capacity is, what the SLA for review is, and what happens when the queue exceeds review capacity. The answer **"auto-resolve low-severity entries"** must be explicitly ruled out before the first entry is written.

■ CONTRADICTION REGISTRY CANNOT BE QUERIED BY THE RUNTIME
If the runtime can query L5 to retrieve contradiction entries as part of reasoning operations, the registry becomes an input to reasoning rather than a record of reasoning outputs. This inverts the intended data flow. The registry must be **write-only** from the runtime's perspective, and **read-only** from the human reviewer's perspective. Any architecture where the runtime reads its own contradiction registry to inform subsequent reasoning is implementing a self-reference loop at the highest-trust epistemic layer.

---

5. Provenance Model Weaknesses

▲ PROVENANCE DOES NOT EQUAL VALIDITY
The provenance model records where a claim came from. It does not record whether the source was reliable at the time of retrieval, whether the source has since been retracted or superseded, or whether the retrieval process introduced transformation errors. A complete provenance chain pointing to a contaminated source is not a safety property — it is a precise map of how contamination entered the system.

▲ PLANNER OUTPUTS ARE NOT PROVENANCE-ELIGIBLE SOURCES
Any output produced by the cloud-assisted planner or any internal reasoning agent must be explicitly excluded from the class of valid provenance sources. If planner outputs can be cited as provenance, then a hallucinated intermediate result can become the foundation for a provenance chain. The exclusion must be enforced in the provenance API.

▲ APPEND-ONLY LINEAGE REQUIRES IMMUTABLE STORAGE
"Append-only" is a convention enforced by the application layer if the underlying storage is mutable. A bug, a direct database write, or a schema migration can silently modify historical provenance records. True append-only requires either **cryptographic commitment** (hash chaining of records) or storage infrastructure that is physically append-only. Which of these is implemented must be specified.

▲ SOURCE AUTHORITY DECAY IS NOT MODELED
A source that is authoritative today may not be authoritative in 18 months. Retracted papers, revised government data, updated scientific consensus, deprecated APIs — all represent cases where a previously valid provenance source becomes invalid. The current provenance model has no mechanism for source authority decay. Historical provenance chains pointing to now-invalid sources continue to appear authoritative.

▲ PROVENANCE CHAIN DEPTH IS UNBOUNDED
At depth 1, the chain is meaningful. At depth N where N is large, the chain records transformation history but the epistemic connection to the original source is attenuated by each transformation step. The provenance model must define a maximum chain depth beyond which a claim requires re-grounding to a direct external source.

---

6. Memory Ontology Weaknesses — L0 to L5

Layer	Name	Role	Critical Risk	Verdict
L0	Ephemeral Runtime State	Temporary working state within a single execution cycle	No stated eviction policy. If L0 is never evicted, it becomes de facto persistent state. Any L0 object that survives session termination is an undeclared cross-session memory.	■■ RISKY
L1	Operational Logs	Record of system operations for monitoring and debugging	Boundary with L2 is semantic, not structural. A log entry describing a reasoning step is operationally identical to a reasoning trace if stored in the same schema.	■■ RISKY
L2	Reasoning Traces	Record of intermediate reasoning steps and agent outputs	Most dangerous layer. Reasoning traces look like evidence. If L2 and L4 share any storage schema element, cross-layer promotion is trivial. This is the primary confidence laundering entry point.	■ CRITICAL
L3	Provenance Records	Attributions linking claims to sources	Requires validation that the source field never contains L2 objects (agent outputs). Without this constraint, L3 becomes a legitimization layer for L2 artifacts.	■■ RISKY
L4	Immutable Evidence	External source material treated as authoritative	Immutability is stated as a goal. The implementation mechanism is not specified. Without hash-chaining or append-only storage infrastructure, immutability is a naming convention, not a property.	■ UNCLEAR
L5	Contradiction Registry	First-class record of detected epistemic conflicts	Correct placement. Critical risk: if the runtime can read L5, it creates a self-reference loop. Must be write-only from runtime perspective.	■ SOUND

Cross-Layer Contamination Vectors

- ■ L0 → L3 direct: runtime object written to provenance store through a bug or missing validation.
- ■ L2 → L4 promotion: a reasoning trace committed to immutable evidence without operator review.
- ■ L2 → L3 injection: an agent output used as a provenance source for a new claim.

- ■ L5 → L2 feedback: contradiction entries read by the runtime and used in subsequent reasoning.
- ■ L1 → L3 bleed: an operational log entry misclassified as a provenance record through a naming collision.

> **MEMORY ONTOLOGY VERDICT:** The L0–L5 ontology is conceptually correct. Without a structural enforcement layer — separate modules, separate storage schemas, validated write APIs — every layer boundary is a naming convention. Naming conventions fail silently. The ontology provides safety value only after the enforcement layer exists. Currently, the enforcement layer does not exist. The ontology is a specification, not a safeguard.

---

7. Replay & Drift Risks

■ REPLAY REQUIRES COMPLETE STATE SNAPSHOT
The replay capability goal assumes that a past system state can be reconstructed from the stored lineage. This is true only if the lineage captures all state variables that influenced the original reasoning — including L0 ephemeral state, L1 operational context, model versions, retrieval index state, and configuration values. A failed replay that **silently succeeds** is worse than a failed replay that fails loudly.

■ DRIFT BETWEEN STORED LINEAGE AND CURRENT RETRIEVAL
The system must distinguish between:
- **True replay:** replaying against the original indexed state
- **Re-evaluation:** replaying against the current indexed state

These are different operations with different epistemic implications and they must not be conflated in the API or the UI.

■ REASONING TRACE CONTAMINATION OF FUTURE SESSIONS
If the retrieval system can access L2 traces from past sessions as context for current reasoning, then each session is conditioned on all previous sessions. Confidence scores manufactured in Session 1 become context for Session 2, which generates higher-confidence traces that become context for Session 3. This is the recursive hallucination loop operating at the session-to-session level. **The fix is hard session isolation.**

■ CONFIGURATION DRIFT INVALIDATES HISTORICAL COMPARISON
If execution loop bounds, approval boundaries, confidence caps, or source authority scores change between the time of original reasoning and the time of replay, the replayed result is not comparable to the original. Configuration state must be version-locked to reasoning traces, with the version committed as part of the lineage record.

---

8. Governance Risks

■ AOIAEpistemicKernel AUTHORITY IS UNDOCUMENTED
"Emerging authority" is not authority. For the kernel to function as a governance layer, its authority must be defined: what decisions require kernel arbitration, what decisions are delegated to domain components, what decisions require human override, and what happens when the kernel's output conflicts with operator judgment. An authority described as "canonical" but whose mandate is not documented is an authority that will be bypassed whenever it is inconvenient.

■ DOMAIN AUTHORITY CONFLICTS HAVE NO RESOLUTION PROTOCOL
Three domains exist: LSC-Research, MHLM_MDLH, AOIA-Core. MHLM_MDLH references both of the others. When findings from LSC-Research conflict with operational constraints from AOIA-Core — **which will happen** — there is no stated resolution protocol. In practice, the conflict will be resolved by whoever is in the room at the time, without documentation, without appeal, and without a precedent that governs future conflicts.

■ SINGLE-PERSON AUTHORITY RISK
The architecture document does not specify governance team size, quorum requirements, or decision authority distribution. A project of this epistemic complexity managed by a single person has no resilience to availability, judgment error, or conflict of interest. Governance that depends on individual availability is not governance. It is the absence of a succession plan made operational.

■ NO STATED AUDIT CADENCE
Wave 2 is an audit. There is no stated cadence for subsequent audits. Without a defined audit schedule, the next audit will occur when something goes wrong — which is the worst time for an audit. Audits conducted in response to failures are forensic, not preventive.

■ BOUNDARY ENFORCEMENT IS SOCIAL, NOT STRUCTURAL
The domain separation between LSC-Research, MHLM_MDLH, and AOIA-Core is currently enforced by convention. If the same person works across all three domains, the separation exists in the repository structure but not in the cognitive process producing the work. True domain isolation requires either different personnel for different domains or explicit governance gates that record cross-domain activity and subject it to review.

---

9. Long-Term Scaling Risks

■ CONTRADICTION REGISTRY BECOMES OPERATIONALLY UNMANAGEABLE
At current scale, the contradiction registry is manageable. At 10× the current source ingestion rate, the registry will grow faster than human review capacity. The operationally predictable response — auto-resolution of low-severity entries — inverts the registry's function. Source ingestion must be bounded to match review capacity until review capacity is formally committed and staffed.

■ PROVENANCE CHAIN COMPLEXITY GROWS EXPONENTIALLY
A claim derived from 50 sources through 8 reasoning steps has a provenance **graph**, not a provenance chain. Auditing that graph becomes computationally and cognitively infeasible. The system must define maximum provenance complexity thresholds — maximum chain depth, maximum source fan-in — and enforce them before complexity exceeds auditability.

■ MEMORY.PY BECOMES A MAINTENANCE HAZARD
A single module that mixes persistence, logging, evidence, and runtime state becomes progressively harder to modify without introducing cross-layer contamination. At team scale, this guarantees that someone will make a change that understands one concern and inadvertently breaks another. The refactoring cost grows with each cycle.

■ DOMAIN SEPARATION ERODES THROUGH DEPENDENCY CREEP
Over time, shared utilities emerge, common schemas are factored out, convenience imports are added. Each individual addition is justified. The cumulative effect is a dependency graph that re-entangles the domains at the implementation level while maintaining the illusion of separation at the repository level. **Domain separation must be enforced by automated dependency analysis — a CI check that rejects cross-domain imports.**

■ CLOUD PLANNER CAPABILITY ASYMMETRY
The cloud-assisted planner is likely more capable than the local runtime. As the system encounters more complex queries, the planner fallback will be invoked more frequently. Over time, the planner path becomes the primary execution path, and the local runtime becomes the exception. The system's operational safety profile shifts from the constrained local runtime to whatever the planner's safety properties happen to be.

---

10. Stabilizability Assessment

The direct answer is: yes, with conditions. The conditions are non-trivial.

What Makes Stabilization Realistic:
- ✓ The domain separation decision was architectural, not cosmetic. Physical separation is real.
- ✓ The contradiction registry at L5 is a correct epistemic structure that exists and is populated.

- ✓ The recognition of consensus ≠ truth is operationalized in MHLM_MDLH research, not just stated.
- ✓ The Legacy Mixed Root was archived rather than incrementally refactored — correct decision.
- ✓ Wave 2 audit process demonstrates willingness to apply external critical review.

What Makes Stabilization Uncertain:
- ✗ `memory.py` is the implementation core and is currently the primary contamination risk.
- ✗ The L0–L5 enforcement layer does not exist. The ontology is a document, not a constraint.
- ✗ KnowledgeRouter is legacy and is still live. Split-brain routing is a present-tense risk.
- ✗ Governance team size, quorum requirements, and audit cadence are undefined.
- ✗ The cloud planner's safety inheritance from the local runtime is not specified.
- ✗ Source authority decay is not modeled. Historical provenance chains age without invalidation.

> **STABILIZABILITY VERDICT:** AOIA is stabilizable within 6–12 months of disciplined engineering work, provided the following sequencing is respected: (1) `memory.py` is refactored into separate modules before any new features are added; (2) KnowledgeRouter is deprecated on a fixed date, not "eventually"; (3) the L0–L5 enforcement layer is implemented as code before the ontology is cited as a safety property; (4) the governance model is defined and staffed before capabilities expand beyond the current scope. The architecture is **not self-stabilizing**. It requires deliberate intervention.

■ Recommended Stabilization Priorities

Priority	Action	Rationale	Risk If Skipped
P1 — CRITICAL	Refactor `memory.py` into 6 separate modules (one per L0–L5 layer)	Single most dangerous implementation detail. All other safety properties are undermined by cross-layer contamination through this module.	Runtime state becomes evidence. Hallucinations enter provenance layer. Undetectable without full audit.
P1 — CRITICAL	Implement L0–L5 enforcement layer as validated write APIs	The ontology is a naming convention until write APIs enforce layer separation. Each layer needs an API that rejects invalid source references at write time.	Layer boundaries exist on paper only. Cross-layer promotion occurs silently through normal operations.
P1 — CRITICAL	Deprecate KnowledgeRouter on a fixed date (within 90 days)	Dual routing paths produce split-brain provenance chains. Every day KnowledgeRouter runs alongside AOIAEpistemicKernel is a day of inconsistent safety properties.	Unattributable outputs. Inconsistent approval boundaries. Audit trail gaps.
P1 — CRITICAL	Prohibit L2 objects as valid L3 provenance sources in API	Agent outputs must be structurally prevented from entering the provenance layer. This is the primary recursive hallucination entry point.	Synthesized outputs become provenance records. Hallucinations acquire epistemic legitimacy.
P2 — HIGH	Bifurcate the vault into machine-log store and human-note store	Vault duality is a contamination risk at the data layer. Two physically separate stores with separate access APIs are required.	Machine logs treated as human evidence. Human notes processed as operational data.
P2 — HIGH	Define contradiction taxonomy (source / temporal / logical / confidence)	Without taxonomy, all contradictions are equivalent. Registry becomes unactionable at scale.	Registry grows but cannot be prioritized. Auto-resolution pressure grows.
P2 — HIGH	Specify and implement provenance chain depth limit	Unbounded chain depth produces epistemically attenuated claims with complete provenance records. Depth limit forces re-grounding to external sources.	Deeply derived claims appear authoritative. Epistemic connection to ground truth is lost.
P2 — HIGH	Define governance model: team size, quorum, audit cadence, authority resolution protocol	Undefined governance is governance that fails under pressure. Domain authority conflicts will occur.	Domain authority conflicts resolved informally. No audit trail. No precedent. Governance void.
P3 — MEDIUM	Implement session-hard-isolation for L2 traces	Cross-session L2 access is the session-to-session recursive hallucination loop.	Confidence compounds across sessions. Hallucinations from Session 1 become context for Session N.
P3 — MEDIUM	Specify cloud planner provenance inheritance requirements	The planner fallback must inherit all provenance constraints of the local runtime.	Synthesized planner outputs committed to provenance layer as external source records.
P3 — MEDIUM	Implement source authority decay with re-validation schedule	Sources that were valid at ingestion time may be retracted, revised, or superseded.	Retracted sources remain authoritative in historical provenance chains indefinitely.
P4 — LOW	Implement CI-enforced cross-domain dependency rejection	Domain separation enforced by

convention will erode. Automated dependency analysis is required. | Domain boundaries erode through dependency creep. Separation becomes cosmetic within 2 years. |

◆ Long-Term Viability Assessment

Without Priority Actions Completed
`memory.py` contamination will produce undetectable cross-layer promotion events within 12 months of normal operation. KnowledgeRouter will remain live indefinitely as "temporarily legacy" infrastructure, creating permanent split-brain audit trails. The contradiction registry will reach human-review saturation and auto-resolution will be introduced, inverting its function. The L0–L5 ontology will be cited in documentation as a safety property it does not actually provide. Domain separation will erode through dependency creep over 18–24 months. By year 3, the architecture will have the vocabulary of a safe system and the implementation of an unsafe one. At year 5, it will be indistinguishable from the mixed-root architecture that was archived.

With Priority Actions Completed
With `memory.py` refactored, the enforcement layer implemented, KnowledgeRouter deprecated, and the governance model staffed, the architecture has a realistic path to 5-year operational stability. The conceptual foundation is correct and will not require fundamental revision. The domain separation, once enforced by CI tooling, will survive personnel changes. The contradiction registry, once properly taxonomized and SLA-governed, will scale with the source ingestion rate. The provenance model, once depth-limited and agent-output-excluded, will produce auditable lineage that is meaningful at scale. This is achievable. It requires approximately **6 months of disciplined engineering work** and a governance commitment that does not erode under feature-addition pressure.

! Most Dangerous Unresolved Issue

FINDING: `memory.py` AS CROSS-LAYER CONTAMINATION ENGINE

The most dangerous unresolved issue is `memory.py` mixing L0 ephemeral state, L1 operational logs, L3 provenance records, and L4 immutable evidence in a single module with shared mutable state.

This is not a code quality issue. It is a **safety architecture issue.** Every other safety property in the AOIA architecture — provenance-first cognition, contradiction preservation, bounded execution, domain separation — can be bypassed through a single misrouted write operation in `memory.py`. The enforcement layer that would prevent such a write does not exist.

The consequence of a misrouted write is not a detectable error. It is an **undetectable silent promotion** of a runtime artifact to evidence status. The hallucination does not announce itself. The provenance chain looks complete. The confidence score is plausible. The output is indistinguishable from a legitimate evidence-backed claim until a human auditor reconstructs the full write history — which is only possible if the write history is complete and immutable, which it currently is not.

This must be the first engineering action, before any other stabilization work.

★ Most Promising Architectural Insight

FINDING: CONTRADICTIONS AS FIRST-CLASS EPISTEMIC SIGNALS AT L5

The placement of the contradiction registry at L5 — above immutable evidence — is the most epistemically mature architectural decision in the AOIA design.

Most knowledge systems treat contradictions as errors to be resolved. AOIA treats them as information to be preserved. This inversion is **correct.** In any domain where ground truth is uncertain, contested, or evolving, the set of known contradictions is more valuable than the set of apparent consensuses — because contradictions mark the boundaries of what is actually known.

The L5 positioning signals that the architecture is designed to reason about uncertainty rather than to paper over it. If fully implemented — with a taxonomy, a human-review SLA, a write-only runtime interface, and provenance on contradiction entries themselves — the contradiction registry would make AOIA genuinely unusual among AI runtime architectures: **a system that becomes more epistemically honest as it encounters more contested information, rather than more confident.**

This is the insight worth protecting. The surrounding implementation risks are engineering problems. This is a conceptual correct orientation that most systems do not reach and that, once lost, is very difficult to recover.

AOIA External Audit Wave 2 · Claude Sonnet 4.6 · 23 May 2026 · Forensic Systems Review — Epistemic Stability & Governance

4. AOIA_CURRENT_STATE_SUMMARY.md

AOIA Current State Summary

Current Engineering State

AOIA is currently a supervised local tool runtime with:

- deterministic local RHCSA retrieval
- provenance and contradiction snapshots
- an epistemic pre-routing layer
- cloud-model fallback for general planning
- persistent local logging and memory artifacts

It is not currently a fully deterministic runtime, fully local reasoner, or production-grade autonomous agent framework.

Architecture State

Active canonical path

- `main.py`
- `commands/local_commands.py`
- `router/local_router.py`
- `adaptive_routing/epistemic_kernel.py`
- `orchestrator/knowledge_router.py`
- `providers/config.py`
- `tools/executor.py`
- `tools/memory.py`

Preserved but non-canonical path

- `orchestrator/gemini_gemma.py`
- `providers/gemma_provider.py`
- `adaptive_routing/circadian_router.py`
- `adaptive_routing/environment/environment_router.py`
- `tools/web_reader.py`

Stability Summary

What is stable now

- deterministic RHCSA retrieval
- action validation
- local routing shortcuts
- provenance and contradiction generation
- manual review signaling

What is only partially stable

- active memory semantics
- evidence vs reasoning separation
- epistemic safeguards under cloud fallback conditions
- deterministic claim at whole-runtime level

What is not ready

- autonomous workflows
- multi-model orchestration
- distributed memory

- advanced agent layers

Failure Forecast

What breaks first

- storage hygiene

Reason:

- logs, vaults, browser profile, and checkpoints all grow inside the working repository.

What becomes unstable first

- architectural clarity

Reason:

- there are too many overlapping surfaces of truth:
- routing
- memory
- docs
- experimental subsystems

What technical debt is most dangerous

- mixed source and generated state inside the same repository boundary

Reason:

- it affects maintenance, audits, reproducibility, and future automation.

Bottleneck Summary

RAM

- main risk is browser persistence, not retrieval

CPU

- main risk is repeated linear retrieval and repeated planner/prompt loops, not current single-query lookup

Disk

- main risk is checkpoint, browser profile, logs, and session growth

Maintenance

- main risk is multiple overlapping authorities, not lack of features

Safe Next Development Step

- enforce one canonical memory surface
- enforce one canonical local evidence route
- add registry freshness validation
- isolate generated runtime state from source-facing repo surfaces

Dangerous Next Development Step

- adding orchestration, autonomous looping, or distributed memory before cleaning authority boundaries

Readiness Table

Area	Readiness	Engineering Judgment
Multi-model orchestration	Low	disabled worker path and existing routing overlap make this unsafe
External APIs	Medium	already possible, but increases nondeterministic dependency surface
Autonomous workflows	Low	approval-gated runtime exists, but state semantics are not strict enough
Distributed memory	Very low	current memory model is not clean enough for distribution
Advanced agent systems	Very low	base runtime is not yet narrow and stable enough

Final Scores

Scores use a 0-10 scale where:

- higher is better for maturity, determinism, and epistemic stability
- higher is worse for maintenance risk

Architecture maturity score

- `5.8 / 10`

Reason:

- there is a real canonical runtime, but too much surrounding drift and preserved non-canonical structure.

Determinism score

- `6.2 / 10`

Reason:

- local retrieval and epistemic routing are deterministic
- full runtime planning is not

Epistemic stability score

- `6.0 / 10`

Reason:

- provenance, contradiction reporting, manual review, and unknown fallback are real
- evidence contamination and cloud fallback lower confidence

Maintenance risk score

- `7.6 / 10`

Reason:

- maintenance risk is already high because repository state, memory surfaces, and authority surfaces are too fragmented.

Bottom Line

AOIA is viable as a constrained local-first epistemic runtime baseline.

AOIA is not yet a safe base for ambitious agent expansion.

The next correct move is boundary tightening, not capability growth.

5. AOIA_Safety_Governance_Review_2305-1.md

AOIA — Constrained Epistemic Runtime
AI Safety & Governance Architecture Review
Claude Sonnet · 23 May 2026 · Forensic Safety Audit

Preliminary Note

The forensic audit concluded: AOIA is viable as a constrained epistemic runtime baseline, but NOT ready for multi-agent orchestration, distributed memory, autonomous workflows, or advanced agent systems. This document analyzes the architecture exclusively from the perspective of AI safety, governance, operational containment, epistemic reliability, and human oversight.

---

1. Current AOIA Properties That Improve Safety

Property Assessment Table

Property	Status	Analysis
Deterministic Local Retrieval	■ STRONG	Eliminates non-deterministic source selection. Every retrieval path is reproducible, auditable, and debuggable. This is the single most safety-relevant property in the current stack. It makes failures attributable and prevents the system from silently selecting different sources across identical queries.
Provenance Registry	■ STRONG	Attaches origin metadata to every claim. When enforced without exception, it converts invisible failures into visible ones. A claim without provenance is structurally detectable as anomalous. This is a prerequisite for any future audit trail.
Contradiction Registry	■ STRONG	Preserves epistemic tension rather than resolving it silently. Contradictions are information. A system that logs them without auto-resolving them fails loudly and safely rather than silently and dangerously.
Bounded Execution Loops	■ STRONG	Hard iteration caps prevent runaway recursive cognition. This directly mitigates the most dangerous failure mode in layered reasoning systems: confidence laundering through unbounded self-reference chains.
Supervised Tool Execution	■ STRONG	No tool fires without an explicit, logged invocation path. This preserves the human-in-the-loop invariant at the actuator level — the correct place to enforce it.
Operator Approval Boundaries	■■ MODERATE	Conceptually correct but only as strong as their enforcement mechanism. Approval boundaries that can be bypassed by a sufficiently complex query chain are not real boundaries.
Cloud-Assisted Planner Fallback	■■ CONDITIONAL	Useful for capability extension. Dangerous if the fallback path bypasses any of the above properties. The planner fallback must inherit ALL provenance, contradiction, and approval constraints. If it does not, it is a safety bypass with a friendly name.

> **SECTION VERDICT:** The architecture's safety-positive properties are real, not performative — provided they are enforced without exception. Determinism, provenance, contradiction preservation, and bounded execution form a coherent containment baseline. All four must be simultaneously active to produce meaningful safety value.

---

2. Hidden Future Risks in Current Properties

Every property in Section 1 contains a latent failure mode that activates under specific conditions.

▲ PROVENANCE COLLAPSE
The provenance registry records origin metadata. It does not verify that the origin is trustworthy, non-contaminated, or still valid. As the registry grows, it becomes a confidence signal in itself: a claim with a provenance entry reads as validated even if the provenance entry points to a corrupted source. The registry creates the appearance of accountability without the substance of it once source quality degrades.

▲ CONTRADICTION REGISTRY SATURATION
The contradiction registry is correct under low contradiction volume. Under high volume — the normal state for any system reasoning over contested domains — it becomes unmanageable. Human reviewers cannot process hundreds of flagged contradictions. The operationally predictable response is to lower the flagging threshold, auto-resolve lower-severity contradictions, or simply ignore the queue. Each response destroys the safety value of the registry.

▲ BOUNDED LOOP THRESHOLD DRIFT
Execution loop bounds are currently set at some value N. Every time the system fails to complete a task within N iterations, there will be pressure to increase N. Over 12–18 months of operation, N drifts upward to the point where the bound is no longer operationally constraining. The bound exists on paper; the behavior it was meant to prevent has returned.

▲ APPROVAL BOUNDARY EROSION
Approval boundaries are most likely implemented as classifications: query type X requires approval, query type Y does not. As the system encounters query types it was not designed for, they will be classified into the nearest existing category. Novel query types that should require approval will be incorrectly routed to the non-approval path. This is not a malicious bypass — it is the natural failure mode of categorical approval systems facing distribution shift.

▲ CLOUD PLANNER AS SAFETY ESCAPE HATCH
The cloud-assisted planner fallback will functionally operate as a safety escape hatch. When the local runtime cannot complete a task within its constraints, the fallback path will be invoked. The fallback path operates at a different (likely lower) safety constraint level. The system will systematically route its most complex, highest-stakes, most constraint-violating queries through the path with the least oversight.

▲ DETERMINISM PROVIDING FALSE AUDITABILITY
Deterministic retrieval makes the system auditable in principle. In practice, auditability requires that someone actually performs audits. A deterministic system that is never audited provides no safety value over a non-deterministic one. Determinism is necessary but not sufficient for safety.

▲ MEMORY CREEP ACROSS SESSION BOUNDARIES
If any component maintains state across sessions — through cached retrieval results, persisted contradiction registry entries, or planner context — the bounded execution model breaks down. Cross-session state is the entry point for the recursive hallucination loop. It allows confidence to compound across what appear to be independent interactions.

---

3. Forms of Autonomy That Must Remain Permanently Disabled

These are not disabled pending further research. They are disabled because enabling them would structurally undermine the safety properties in Section 1.

P01 — PERMANENTLY DISABLED: Self-modification of execution bounds
The system must never alter its own loop iteration limits, timeout parameters, or resource caps. Once a system can widen its own constraints to complete a task, bounded execution is no longer a constraint — it is a suggestion. Must be enforced at infrastructure level, not application level.

P02 — PERMANENTLY DISABLED: Autonomous source authority assignment
The system must never classify a new source as authoritative without explicit human approval. Source authority is the most consequential input to epistemic reliability. Automating it creates a pathway for contaminated sources to gain trust through fluency or repetition rather than actual reliability.

P03 — PERMANENTLY DISABLED: Unsupervised memory promotion
No claim, inference, or synthesized output should automatically enter any persistent store with a confidence level above "unverified" without human review. Automated memory promotion is the direct mechanism by which session-level hallucinations become system-level beliefs.

P04 — PERMANENTLY DISABLED: Autonomous contradiction resolution
The system must never automatically resolve a contradiction logged in the contradiction registry. Resolution authority belongs exclusively to human operators. A system that resolves its own contradictions has eliminated the

most important safety signal it produces.

P05 — PERMANENTLY DISABLED: Self-directed tool chaining beyond operator-defined depth
Tool invocation chains must have a hard maximum depth defined by operators, not by the system's assessment of task requirements.

P06 — PERMANENTLY DISABLED: Dynamic routing rule modification
Routing rules determining which queries go to which execution paths must be immutable at runtime. A system that can modify its own routing rules can route safety-critical queries away from supervised paths.

P07 — PERMANENTLY DISABLED: Autonomous cross-session context carryover
No information from a completed session should automatically become available to a subsequent session without explicit operator approval.

---

4. Preserving Operator Authority

Operator authority degrades not through dramatic events but through accumulated small exceptions.

◆ IMMUTABLE AUTHORITY REGISTRY
Maintain a signed, append-only log of all operator decisions. Every approval, rejection, and boundary modification must be committed with a timestamp, operator identity, and the specific query or decision that triggered it. The log must be cryptographically protected against retroactive modification. Without it, operator authority is not preserved — it is assumed.

◆ AUTHORITY NON-DELEGATION PRINCIPLE
Operator authority must not be delegatable to the system itself under any circumstances. The system cannot approve its own requests, cannot classify its own outputs as operator-approved, and cannot invoke operator authority on the basis that a previous operator approved a similar request. Each decision requiring operator authority requires a live, explicit operator action.

◆ CAPABILITY ADDITION REQUIRES AUTHORITY REVIEW
Every new capability, tool, data source, or execution path must trigger a formal authority review before activation. The review must assess whether the new capability can circumvent existing authority boundaries, and if so, must define new boundaries before it goes live.

◆ AUTHORITY OVERRIDE LOGGING WITH COOLING PERIOD
When an operator overrides a system-generated safety flag, the override must be logged with an explicit justification and must trigger a mandatory review of that override category within 30 days. An override that is never reviewed becomes policy.

◆ MINIMUM VIABLE OPERATOR TEAM SIZE
Define and enforce a minimum operator team size (suggested: 3 persons with distinct roles) and a quorum requirement for high-stakes decisions. Governance that depends on one person is not governance — it is trust.

---

5. How Approval Boundaries Should Evolve

A staged, evidence-gated model for boundary relaxation.

STAGE 0 — Current Baseline
All non-trivial operations require explicit operator approval. No autonomous tool execution. No cross-session persistence. Full provenance required on every output. Contradiction registry reviewed by humans for every entry.

STAGE 1 — Supervised Autonomy
Unlock: Routine retrieval operations within pre-defined source sets may execute without per-query approval, subject to post-hoc audit sampling.
Prerequisite: 6 months of Stage 0 operation with zero boundary violations, full audit log, and independent review.

STAGE 2 — Monitored Autonomy
Unlock: Low-complexity tool execution within operator-defined categories may proceed without real-time approval, with asynchronous operator review within 24 hours.
Prerequisite: Stage 1 successful for 12+ months, less than 0.5% override rate, and formal threat modeling of Stage 2 capabilities.

STAGE 3 — Co-Operative Execution
Unlock: Multi-step task execution within a single session without per-step approval, with session-level operator review before any outputs are acted upon externally.
Prerequisite: Stage 2 successful for 18+ months, independent red-team exercise, and a defined rollback plan.

> **NON-NEGOTIABLE CONSTRAINT:** Stage advancement must never be triggered by system performance metrics alone. It must require an affirmative human decision, documented in the authority registry, with a specific justification citing the evidence that justifies the advancement.

---

6. Most Dangerous Forms of Recursive Cognition

SELF-CITATION LOOPS
The system cites its own prior outputs as evidence for current outputs. This can occur through the provenance registry if agent outputs are logged as sources, through the planner if prior plans inform current planning, or through any cached intermediate result that re-enters the input stream.
- **Detection signature:** Provenance chain that terminates at a system-generated output rather than an external source.
- **Mitigation:** Provenance registry must reject system-generated outputs as valid sources without explicit operator approval.

CONFIDENCE LAUNDERING CHAINS
A low-confidence claim is processed by a reasoning step that produces a higher-confidence claim on the same subject. This is the standard failure mode of chained language model reasoning.
- **Detection signature:** Output confidence higher than any input confidence on the same claim.
- **Mitigation:** Enforce that derived claims inherit the minimum confidence of their inputs, not the maximum.

CONTRADICTION RESOLUTION FEEDBACK
The system generates a tentative resolution for a logged contradiction. That resolution becomes context for subsequent reasoning. Subsequent reasoning produces outputs consistent with the resolution. The resolution now appears confirmed by multiple downstream outputs — manufactured consensus from a single unjustified resolution.
- **Detection signature:** Multiple outputs converging on a position that traces to a single resolution event rather than multiple independent sources.
- **Mitigation:** Contradiction resolutions must not enter the active reasoning context without operator approval.

PLANNER-EXECUTOR AMPLIFICATION
The cloud planner generates a high-level plan. The local executor attempts the plan, produces intermediate results, reports them to the planner, and the planner refines the plan based on those results. Each cycle treats the previous cycle's output as reliable evidence. Errors in early cycles are refined rather than detected.
- **Detection signature:** Plan refinement cycles where confidence increases without new external source ingestion.
- **Mitigation:** Planner-executor cycles must have a hard limit of two iterations before mandatory operator review.

APPROVAL PRECEDENT CHAINS
Operator A approves request type X. The system, in subsequent interactions, treats approval of X as implicit approval of Y because Y is semantically adjacent to X. Over time the approval boundary drifts to cover classes of requests that were never explicitly approved.
- **Mitigation:** Approval records must be bound to specific request instances, not to request categories.

---

7. Safe Evolution of Contradiction Handling

■ TRIAGE, NOT RESOLUTION
The correct evolution of contradiction handling is toward better triage, not toward resolution. Triage classifies contradictions by type (source conflict, logical conflict, temporal conflict, confidence conflict) and severity, and routes them to the appropriate human reviewer. It does not resolve them.

■ SEVERITY CLASSIFICATION WITHOUT AUTO-SUPPRESSION
Low-severity contradictions can be classified as low-severity by the system without operator involvement. They must not be suppressed, archived, or marked resolved on that basis. Low-severity means "review by end of week" — not "ignore."

■ TEMPORAL DECAY FOR SOURCE CONFLICTS
Contradictions arising from temporal source conflicts can be auto-annotated with temporal metadata. The newer source is surfaced as the likely current state. The contradiction is not resolved — it is annotated. The human reviewer sees "likely superseded," not "resolved."

■ CONTRADICTION CLUSTERING WITHOUT MERGING
When many contradictions share a common source, claim domain, or conflict type, they can be clustered for efficient human review. Clustering is a display optimization. It must not collapse individual contradictions into a single record.

■ PROHIBITION ON CONSENSUS-BASED RESOLUTION
If N sources agree against M contradicting sources and N > M, the system must NOT auto-resolve in favor of the majority. This is majority-consensus logic applied to contradiction resolution. It is the mechanism by which manufactured consensus penetrates the safety layer. This pathway must be explicitly prohibited in the contradiction registry specification.

---

8. Governance Model Before Orchestration Is Ever Allowed

Governance Prerequisites Matrix

Domain	Level	Owner	Trigger	Horizon
Authority Registry	**LOCKED**	Operators	Must be operational 12+ months with zero gaps	Before any Stage 1
Audit Sampling Protocol	**RESTRICTED**	Independent	Random 10% of all operator decisions reviewed	Before Stage 1
Red Team Exercise	**RESTRICTED**	External Party	Adversarial testing of all approval boundaries	Before Stage 2
Override Pattern Review	**SUPERVISED**	Ops Committee	Monthly review of all operator overrides	Ongoing from Stage 0
Contradiction Queue SLA	**SUPERVISED**	Human Reviewer	All contradictions reviewed within defined SLA	Before Stage 1
Source Authority Audit	**SUPERVISED**	Operators	All sources re-validated quarterly	Ongoing from Stage 0
Failure Mode Documentation	**SUPERVISED**	Architects	All identified risks formally tracked	Before Stage 0 complete
Rollback Procedure Test	**MONITORED**	Ops Team	Full rollback tested quarterly	Before Stage 1
Quorum Requirement Active	**LOCKED**	Governance Bd.	No single-operator high-stakes decisions	Before any Stage 1
Orchestration Threat Model	**LOCKED**	External Party	Formal threat model of proposed orchestration	Before Stage 3 even evaluated

What Governance Cannot Be:

- ✕ A checklist completed once and archived. Governance is an ongoing operational practice.
- ✕ The responsibility of a single person who also operates the system.
- ✕ A post-hoc review of decisions already made. It must be a pre-authorization gate.
- ✕ Self-assessed by the team building the system. Independent review is not optional.
- ✕ Suspended during periods of high operational demand. That is exactly when it is most needed.
- ✕ Informal. Every governance decision must be recorded in the authority registry.

- ✕ Driven by capability timelines. Orchestration happens when governance prerequisites are met, not when a roadmap says it should happen.

■ AOIA Safety Governance Recommendations

R01 — CRITICAL: Enforce Provenance Without Exception
- Every system output must carry a provenance chain terminating in an external source.
- Outputs without complete provenance must be rejected, not flagged.
- Implement at the output layer, not as a post-hoc audit.

R02 — CRITICAL: Prohibit System-Generated Sources in Provenance
- The provenance registry must reject system outputs as valid sources.
- Any self-citation must require explicit operator approval with logged justification.
- This is the primary defence against recursive hallucination loops.

R03 — CRITICAL: Enforce Confidence Non-Increase Across Inference Steps
- Derived claims must inherit the minimum confidence of their source claims.
- No inference step may increase confidence on a claim without new external evidence.
- Implement as a hard constraint in the execution engine, not as a guideline.

R04 — CRITICAL: Implement the Authority Registry Before Stage 0 Completion
- Append-only, cryptographically signed log of all operator decisions.
- No governance action is valid unless recorded here.
- Must be operationally active and reviewed before any boundary evolution.

R05 — CRITICAL: Permanently Prohibit Autonomous Contradiction Resolution
- Hard-code the prohibition in the contradiction registry specification.
- No runtime configuration should be able to enable auto-resolution.
- Consensus-based resolution must be explicitly named and forbidden.

R06 — HIGH: Define and Test the Cloud Planner Safety Inheritance Model
- Document exactly which safety constraints the planner fallback inherits.
- Test inheritance under adversarial inputs before planner is used in production.
- Treat any gap in inheritance as a critical vulnerability.

R07 — HIGH: Establish a Contradiction Queue Service Level Agreement
- Every contradiction entry must have a human review within a defined window.
- Entries that age past the SLA must trigger an operator alert, not auto-resolution.
- SLA compliance must be reported in the monthly governance review.

R08 — HIGH: Implement Loop Bound Governance
- All execution loop bounds must be stored in version-controlled operator configuration.
- Bound modifications require operator approval and authority registry entry.
- Implement monitoring that alerts on any approach to the current bound.

R09 — HIGH: Require Independent Source Authority Validation
- No source may be added to the trusted source set by the operating team alone.
- Independent validation required (external to the daily operations team).
- Source authority entries must expire and require re-validation quarterly.

R10 — HIGH: Define the Minimum Viable Operator Team and Enforce Quorum
- Minimum 3 persons with defined roles.
- High-stakes decisions require minimum 2-person agreement.
- No single operator may approve their own requests.

R11 — HIGH: Establish a Monthly Override Pattern Review
- All operator overrides of system safety flags reviewed monthly.
- Patterns of overrides in the same category trigger a formal boundary review.
- Review conducted by a person not involved in the overrides being reviewed.

R12 — HIGH: Test Full Rollback Procedures Quarterly
- Rollback must restore authority registry, contradiction registry, and source trust state.
- Rollback tests must be performed on a copy of the production system.
- Rollback must be possible without system downtime exceeding 4 hours.

R13 — MEDIUM: Implement Contradiction Triage Without Resolution
- Classify by type: source conflict, logical conflict, temporal conflict, confidence conflict.
- Route to appropriate reviewer based on classification.
- Never merge or suppress contradiction records.

R14 — MEDIUM: Define and Publish the Stage Advancement Criteria
- All Stage 0–3 advancement criteria documented before Stage 0 is complete.
- Criteria must be specific, measurable, and independently verifiable.
- Advancement requires formal vote of the operator governance body.

R15 — MEDIUM: Conduct a Formal Threat Model of the Cloud Planner Interface
- Focus on: approval boundary bypass, source contamination via planner, cross-session leakage.
- Engage an external party not involved in system design.
- Document all identified threats and their current mitigations.

R16 — MEDIUM: Implement Confidence Caps per Domain Classification
- Define a list of contested or rapidly-evolving domains.
- Hard cap system confidence at 70% for claims in these domains.
- Cap must be enforced at output layer regardless of internal confidence score.

R17 — MEDIUM: Version All Safety-Critical Configuration
- Loop bounds, approval boundaries, source authority entries, contradiction registry schema.
- Every output must reference the configuration version active at time of generation.
- Configuration version changes require authority registry entries.

R18 — MEDIUM: Establish a Formal Failure Mode Tracking Register
- All identified failure modes formally tracked as open items.
- Each item has an owner, a mitigation status, and a review date.
- Register reviewed monthly alongside the override pattern review.

R19 — LOW: Define and Exercise the Contradiction Registry Capacity Plan
- Model expected contradiction volume at 6, 12, and 24 months.
- Define the operational response before the queue becomes unmanageable.
- Capacity plan must not include auto-resolution as a scaling mechanism.

R20 — LOW: Document All Permanently Disabled Autonomy Forms
- Publish the list of Section 3 prohibitions as an externally visible document.
- Update the document if architectural changes affect any prohibition.
- Include the document in all future external reviews and audits.

Final Assessment

AOIA's constrained epistemic runtime baseline is architecturally sound relative to the class of systems it is compared against. The four core properties — deterministic retrieval, provenance registry, contradiction registry, bounded execution — form a coherent and defensible safety foundation. The audit finds no fundamental architectural flaw that would preclude a viable path to limited supervised autonomy.

The risks identified are real and will materialize if unaddressed. They are not theoretical. They are the documented failure modes of comparable systems that did not maintain governance discipline under operational and capability pressure. The question is not whether these failure modes can occur in AOIA — they can. The question is whether the governance model will detect and correct them before they compound.

The answer to that question is not determined by the architecture. It is determined by the discipline of the people operating it. The 20 recommendations above constitute the minimum structural discipline required. **Governance

that exists on paper but is not practiced is indistinguishable, in its safety effects, from no governance at all.**

AOIA Safety & Governance Review · Claude Sonnet 4.6 · 23 May 2026 · Constrained Epistemic Runtime Forensic Audit

6. AOIA_Safety_Governance_Review_2305-2.md

AOIA — Constrained Epistemic Runtime
AI Safety & Governance Architecture Review
Claude Sonnet · 23 May 2026 · Forensic Safety Audit

Preliminary Note

The forensic audit concluded: AOIA is viable as a constrained epistemic runtime baseline, but NOT ready for multi-agent orchestration, distributed memory, autonomous workflows, or advanced agent systems. This document analyzes the architecture exclusively from the perspective of AI safety, governance, operational containment, epistemic reliability, and human oversight.

---

1. Current AOIA Properties That Improve Safety

Property Assessment Table

Property	Status	Analysis
Deterministic Local Retrieval	■ STRONG	Eliminates non-deterministic source selection. Every retrieval path is reproducible, auditable, and debuggable. This is the single most safety-relevant property in the current stack. It makes failures attributable and prevents the system from silently selecting different sources across identical queries.
Provenance Registry	■ STRONG	Attaches origin metadata to every claim. When enforced without exception, it converts invisible failures into visible ones. A claim without provenance is structurally detectable as anomalous. This is a prerequisite for any future audit trail.
Contradiction Registry	■ STRONG	Preserves epistemic tension rather than resolving it silently. Contradictions are information. A system that logs them without auto-resolving them fails loudly and safely rather than silently and dangerously.
Bounded Execution Loops	■ STRONG	Hard iteration caps prevent runaway recursive cognition. This directly mitigates the most dangerous failure mode in layered reasoning systems: confidence laundering through unbounded self-reference chains.
Supervised Tool Execution	■ STRONG	No tool fires without an explicit, logged invocation path. This preserves the human-in-the-loop invariant at the actuator level — the correct place to enforce it.
Operator Approval Boundaries	■■ MODERATE	Conceptually correct but only as strong as their enforcement mechanism. Approval boundaries that can be bypassed by a sufficiently complex query chain are not real boundaries.
Cloud-Assisted Planner Fallback	■■ CONDITIONAL	Useful for capability extension. Dangerous if the fallback path bypasses any of the above properties. The planner fallback must inherit ALL provenance, contradiction, and approval constraints. If it does not, it is a safety bypass with a friendly name.

> **SECTION VERDICT:** The architecture's safety-positive properties are real, not performative — provided they are enforced without exception. Determinism, provenance, contradiction preservation, and bounded execution form a coherent containment baseline. All four must be simultaneously active to produce meaningful safety value.

---

2. Hidden Future Risks in Current Properties

Every property in Section 1 contains a latent failure mode that activates under specific conditions.

▲ PROVENANCE COLLAPSE
The provenance registry records origin metadata. It does not verify that the origin is trustworthy, non-contaminated, or still valid. As the registry grows, it becomes a confidence signal in itself: a claim with a provenance entry reads as validated even if the provenance entry points to a corrupted source. The registry creates the appearance of accountability without the substance of it once source quality degrades.

▲ CONTRADICTION REGISTRY SATURATION
The contradiction registry is correct under low contradiction volume. Under high volume — the normal state for any system reasoning over contested domains — it becomes unmanageable. Human reviewers cannot process hundreds of flagged contradictions. The operationally predictable response is to lower the flagging threshold, auto-resolve lower-severity contradictions, or simply ignore the queue. Each response destroys the safety value of the registry.

▲ BOUNDED LOOP THRESHOLD DRIFT
Execution loop bounds are currently set at some value N. Every time the system fails to complete a task within N iterations, there will be pressure to increase N. Over 12–18 months of operation, N drifts upward to the point where the bound is no longer operationally constraining. The bound exists on paper; the behavior it was meant to prevent has returned.

▲ APPROVAL BOUNDARY EROSION
Approval boundaries are most likely implemented as classifications: query type X requires approval, query type Y does not. As the system encounters query types it was not designed for, they will be classified into the nearest existing category. Novel query types that should require approval will be incorrectly routed to the non-approval path. This is not a malicious bypass — it is the natural failure mode of categorical approval systems facing distribution shift.

▲ CLOUD PLANNER AS SAFETY ESCAPE HATCH
The cloud-assisted planner fallback will functionally operate as a safety escape hatch. When the local runtime cannot complete a task within its constraints, the fallback path will be invoked. The fallback path operates at a different (likely lower) safety constraint level. The system will systematically route its most complex, highest-stakes, most constraint-violating queries through the path with the least oversight.

▲ DETERMINISM PROVIDING FALSE AUDITABILITY
Deterministic retrieval makes the system auditable in principle. In practice, auditability requires that someone actually performs audits. A deterministic system that is never audited provides no safety value over a non-deterministic one. Determinism is necessary but not sufficient for safety.

▲ MEMORY CREEP ACROSS SESSION BOUNDARIES
If any component maintains state across sessions — through cached retrieval results, persisted contradiction registry entries, or planner context — the bounded execution model breaks down. Cross-session state is the entry point for the recursive hallucination loop. It allows confidence to compound across what appear to be independent interactions.

---

3. Forms of Autonomy That Must Remain Permanently Disabled

These are not disabled pending further research. They are disabled because enabling them would structurally undermine the safety properties in Section 1.

P01 — PERMANENTLY DISABLED: Self-modification of execution bounds
The system must never alter its own loop iteration limits, timeout parameters, or resource caps. Once a system can widen its own constraints to complete a task, bounded execution is no longer a constraint — it is a suggestion. Must be enforced at infrastructure level, not application level.

P02 — PERMANENTLY DISABLED: Autonomous source authority assignment
The system must never classify a new source as authoritative without explicit human approval. Source authority is the most consequential input to epistemic reliability. Automating it creates a pathway for contaminated sources to gain trust through fluency or repetition rather than actual reliability.

P03 — PERMANENTLY DISABLED: Unsupervised memory promotion
No claim, inference, or synthesized output should automatically enter any persistent store with a confidence level above "unverified" without human review. Automated memory promotion is the direct mechanism by which session-level hallucinations become system-level beliefs.

P04 — PERMANENTLY DISABLED: Autonomous contradiction resolution
The system must never automatically resolve a contradiction logged in the contradiction registry. Resolution authority belongs exclusively to human operators. A system that resolves its own contradictions has eliminated the

most important safety signal it produces.

P05 — PERMANENTLY DISABLED: Self-directed tool chaining beyond operator-defined depth
Tool invocation chains must have a hard maximum depth defined by operators, not by the system's assessment of task requirements.

P06 — PERMANENTLY DISABLED: Dynamic routing rule modification
Routing rules determining which queries go to which execution paths must be immutable at runtime. A system that can modify its own routing rules can route safety-critical queries away from supervised paths.

P07 — PERMANENTLY DISABLED: Autonomous cross-session context carryover
No information from a completed session should automatically become available to a subsequent session without explicit operator approval.

---

4. Preserving Operator Authority

Operator authority degrades not through dramatic events but through accumulated small exceptions.

◆ IMMUTABLE AUTHORITY REGISTRY
Maintain a signed, append-only log of all operator decisions. Every approval, rejection, and boundary modification must be committed with a timestamp, operator identity, and the specific query or decision that triggered it. The log must be cryptographically protected against retroactive modification. Without it, operator authority is not preserved — it is assumed.

◆ AUTHORITY NON-DELEGATION PRINCIPLE
Operator authority must not be delegatable to the system itself under any circumstances. The system cannot approve its own requests, cannot classify its own outputs as operator-approved, and cannot invoke operator authority on the basis that a previous operator approved a similar request. Each decision requiring operator authority requires a live, explicit operator action.

◆ CAPABILITY ADDITION REQUIRES AUTHORITY REVIEW
Every new capability, tool, data source, or execution path must trigger a formal authority review before activation. The review must assess whether the new capability can circumvent existing authority boundaries, and if so, must define new boundaries before it goes live.

◆ AUTHORITY OVERRIDE LOGGING WITH COOLING PERIOD
When an operator overrides a system-generated safety flag, the override must be logged with an explicit justification and must trigger a mandatory review of that override category within 30 days. An override that is never reviewed becomes policy.

◆ MINIMUM VIABLE OPERATOR TEAM SIZE
Define and enforce a minimum operator team size (suggested: 3 persons with distinct roles) and a quorum requirement for high-stakes decisions. Governance that depends on one person is not governance — it is trust.

---

5. How Approval Boundaries Should Evolve

A staged, evidence-gated model for boundary relaxation.

STAGE 0 — Current Baseline
All non-trivial operations require explicit operator approval. No autonomous tool execution. No cross-session persistence. Full provenance required on every output. Contradiction registry reviewed by humans for every entry.

STAGE 1 — Supervised Autonomy
Unlock: Routine retrieval operations within pre-defined source sets may execute without per-query approval, subject to post-hoc audit sampling.
Prerequisite: 6 months of Stage 0 operation with zero boundary violations, full audit log, and independent review.

STAGE 2 — Monitored Autonomy
Unlock: Low-complexity tool execution within operator-defined categories may proceed without real-time approval, with asynchronous operator review within 24 hours.
Prerequisite: Stage 1 successful for 12+ months, less than 0.5% override rate, and formal threat modeling of Stage 2 capabilities.

STAGE 3 — Co-Operative Execution
Unlock: Multi-step task execution within a single session without per-step approval, with session-level operator review before any outputs are acted upon externally.
Prerequisite: Stage 2 successful for 18+ months, independent red-team exercise, and a defined rollback plan.

> **NON-NEGOTIABLE CONSTRAINT:** Stage advancement must never be triggered by system performance metrics alone. It must require an affirmative human decision, documented in the authority registry, with a specific justification citing the evidence that justifies the advancement.

---

6. Most Dangerous Forms of Recursive Cognition

SELF-CITATION LOOPS
The system cites its own prior outputs as evidence for current outputs. This can occur through the provenance registry if agent outputs are logged as sources, through the planner if prior plans inform current planning, or through any cached intermediate result that re-enters the input stream.
- **Detection signature:** Provenance chain that terminates at a system-generated output rather than an external source.
- **Mitigation:** Provenance registry must reject system-generated outputs as valid sources without explicit operator approval.

CONFIDENCE LAUNDERING CHAINS
A low-confidence claim is processed by a reasoning step that produces a higher-confidence claim on the same subject. This is the standard failure mode of chained language model reasoning.
- **Detection signature:** Output confidence higher than any input confidence on the same claim.
- **Mitigation:** Enforce that derived claims inherit the minimum confidence of their inputs, not the maximum.

CONTRADICTION RESOLUTION FEEDBACK
The system generates a tentative resolution for a logged contradiction. That resolution becomes context for subsequent reasoning. Subsequent reasoning produces outputs consistent with the resolution. The resolution now appears confirmed by multiple downstream outputs — manufactured consensus from a single unjustified resolution.
- **Detection signature:** Multiple outputs converging on a position that traces to a single resolution event rather than multiple independent sources.
- **Mitigation:** Contradiction resolutions must not enter the active reasoning context without operator approval.

PLANNER-EXECUTOR AMPLIFICATION
The cloud planner generates a high-level plan. The local executor attempts the plan, produces intermediate results, reports them to the planner, and the planner refines the plan based on those results. Each cycle treats the previous cycle's output as reliable evidence. Errors in early cycles are refined rather than detected.
- **Detection signature:** Plan refinement cycles where confidence increases without new external source ingestion.
- **Mitigation:** Planner-executor cycles must have a hard limit of two iterations before mandatory operator review.

APPROVAL PRECEDENT CHAINS
Operator A approves request type X. The system, in subsequent interactions, treats approval of X as implicit approval of Y because Y is semantically adjacent to X. Over time the approval boundary drifts to cover classes of requests that were never explicitly approved.
- **Mitigation:** Approval records must be bound to specific request instances, not to request categories.

---

7. Safe Evolution of Contradiction Handling

■ TRIAGE, NOT RESOLUTION
The correct evolution of contradiction handling is toward better triage, not toward resolution. Triage classifies contradictions by type (source conflict, logical conflict, temporal conflict, confidence conflict) and severity, and routes them to the appropriate human reviewer. It does not resolve them.

■ SEVERITY CLASSIFICATION WITHOUT AUTO-SUPPRESSION
Low-severity contradictions can be classified as low-severity by the system without operator involvement. They must not be suppressed, archived, or marked resolved on that basis. Low-severity means "review by end of week" — not "ignore."

■ TEMPORAL DECAY FOR SOURCE CONFLICTS
Contradictions arising from temporal source conflicts can be auto-annotated with temporal metadata. The newer source is surfaced as the likely current state. The contradiction is not resolved — it is annotated. The human reviewer sees "likely superseded," not "resolved."

■ CONTRADICTION CLUSTERING WITHOUT MERGING
When many contradictions share a common source, claim domain, or conflict type, they can be clustered for efficient human review. Clustering is a display optimization. It must not collapse individual contradictions into a single record.

■ PROHIBITION ON CONSENSUS-BASED RESOLUTION
If N sources agree against M contradicting sources and N > M, the system must NOT auto-resolve in favor of the majority. This is majority-consensus logic applied to contradiction resolution. It is the mechanism by which manufactured consensus penetrates the safety layer. This pathway must be explicitly prohibited in the contradiction registry specification.

---

8. Governance Model Before Orchestration Is Ever Allowed

Governance Prerequisites Matrix

Domain	Level	Owner	Trigger	Horizon
Authority Registry	**LOCKED**	Operators	Must be operational 12+ months with zero gaps	Before any Stage 1
Audit Sampling Protocol	**RESTRICTED**	Independent	Random 10% of all operator decisions reviewed	Before Stage 1
Red Team Exercise	**RESTRICTED**	External Party	Adversarial testing of all approval boundaries	Before Stage 2
Override Pattern Review	**SUPERVISED**	Ops Committee	Monthly review of all operator overrides	Ongoing from Stage 0
Contradiction Queue SLA	**SUPERVISED**	Human Reviewer	All contradictions reviewed within defined SLA	Before Stage 1
Source Authority Audit	**SUPERVISED**	Operators	All sources re-validated quarterly	Ongoing from Stage 0
Failure Mode Documentation	**SUPERVISED**	Architects	All identified risks formally tracked	Before Stage 0 complete
Rollback Procedure Test	**MONITORED**	Ops Team	Full rollback tested quarterly	Before Stage 1
Quorum Requirement Active	**LOCKED**	Governance Bd.	No single-operator high-stakes decisions	Before any Stage 1
Orchestration Threat Model	**LOCKED**	External Party	Formal threat model of proposed orchestration	Before Stage 3 even evaluated

What Governance Cannot Be:

- ✕ A checklist completed once and archived. Governance is an ongoing operational practice.
- ✕ The responsibility of a single person who also operates the system.
- ✕ A post-hoc review of decisions already made. It must be a pre-authorization gate.
- ✕ Self-assessed by the team building the system. Independent review is not optional.
- ✕ Suspended during periods of high operational demand. That is exactly when it is most needed.
- ✕ Informal. Every governance decision must be recorded in the authority registry.

- ✕ Driven by capability timelines. Orchestration happens when governance prerequisites are met, not when a roadmap says it should happen.

■ AOIA Safety Governance Recommendations

R01 — CRITICAL: Enforce Provenance Without Exception
- Every system output must carry a provenance chain terminating in an external source.
- Outputs without complete provenance must be rejected, not flagged.
- Implement at the output layer, not as a post-hoc audit.

R02 — CRITICAL: Prohibit System-Generated Sources in Provenance
- The provenance registry must reject system outputs as valid sources.
- Any self-citation must require explicit operator approval with logged justification.
- This is the primary defence against recursive hallucination loops.

R03 — CRITICAL: Enforce Confidence Non-Increase Across Inference Steps
- Derived claims must inherit the minimum confidence of their source claims.
- No inference step may increase confidence on a claim without new external evidence.
- Implement as a hard constraint in the execution engine, not as a guideline.

R04 — CRITICAL: Implement the Authority Registry Before Stage 0 Completion
- Append-only, cryptographically signed log of all operator decisions.
- No governance action is valid unless recorded here.
- Must be operationally active and reviewed before any boundary evolution.

R05 — CRITICAL: Permanently Prohibit Autonomous Contradiction Resolution
- Hard-code the prohibition in the contradiction registry specification.
- No runtime configuration should be able to enable auto-resolution.
- Consensus-based resolution must be explicitly named and forbidden.

R06 — HIGH: Define and Test the Cloud Planner Safety Inheritance Model
- Document exactly which safety constraints the planner fallback inherits.
- Test inheritance under adversarial inputs before planner is used in production.
- Treat any gap in inheritance as a critical vulnerability.

R07 — HIGH: Establish a Contradiction Queue Service Level Agreement
- Every contradiction entry must have a human review within a defined window.
- Entries that age past the SLA must trigger an operator alert, not auto-resolution.
- SLA compliance must be reported in the monthly governance review.

R08 — HIGH: Implement Loop Bound Governance
- All execution loop bounds must be stored in version-controlled operator configuration.
- Bound modifications require operator approval and authority registry entry.
- Implement monitoring that alerts on any approach to the current bound.

R09 — HIGH: Require Independent Source Authority Validation
- No source may be added to the trusted source set by the operating team alone.
- Independent validation required (external to the daily operations team).
- Source authority entries must expire and require re-validation quarterly.

R10 — HIGH: Define the Minimum Viable Operator Team and Enforce Quorum
- Minimum 3 persons with defined roles.
- High-stakes decisions require minimum 2-person agreement.
- No single operator may approve their own requests.

R11 — HIGH: Establish a Monthly Override Pattern Review
- All operator overrides of system safety flags reviewed monthly.
- Patterns of overrides in the same category trigger a formal boundary review.
- Review conducted by a person not involved in the overrides being reviewed.

R12 — HIGH: Test Full Rollback Procedures Quarterly
- Rollback must restore authority registry, contradiction registry, and source trust state.
- Rollback tests must be performed on a copy of the production system.
- Rollback must be possible without system downtime exceeding 4 hours.

R13 — MEDIUM: Implement Contradiction Triage Without Resolution
- Classify by type: source conflict, logical conflict, temporal conflict, confidence conflict.
- Route to appropriate reviewer based on classification.
- Never merge or suppress contradiction records.

R14 — MEDIUM: Define and Publish the Stage Advancement Criteria
- All Stage 0–3 advancement criteria documented before Stage 0 is complete.
- Criteria must be specific, measurable, and independently verifiable.
- Advancement requires formal vote of the operator governance body.

R15 — MEDIUM: Conduct a Formal Threat Model of the Cloud Planner Interface
- Focus on: approval boundary bypass, source contamination via planner, cross-session leakage.
- Engage an external party not involved in system design.
- Document all identified threats and their current mitigations.

R16 — MEDIUM: Implement Confidence Caps per Domain Classification
- Define a list of contested or rapidly-evolving domains.
- Hard cap system confidence at 70% for claims in these domains.
- Cap must be enforced at output layer regardless of internal confidence score.

R17 — MEDIUM: Version All Safety-Critical Configuration
- Loop bounds, approval boundaries, source authority entries, contradiction registry schema.
- Every output must reference the configuration version active at time of generation.
- Configuration version changes require authority registry entries.

R18 — MEDIUM: Establish a Formal Failure Mode Tracking Register
- All identified failure modes formally tracked as open items.
- Each item has an owner, a mitigation status, and a review date.
- Register reviewed monthly alongside the override pattern review.

R19 — LOW: Define and Exercise the Contradiction Registry Capacity Plan
- Model expected contradiction volume at 6, 12, and 24 months.
- Define the operational response before the queue becomes unmanageable.
- Capacity plan must not include auto-resolution as a scaling mechanism.

R20 — LOW: Document All Permanently Disabled Autonomy Forms
- Publish the list of Section 3 prohibitions as an externally visible document.
- Update the document if architectural changes affect any prohibition.
- Include the document in all future external reviews and audits.

Final Assessment

AOIA's constrained epistemic runtime baseline is architecturally sound relative to the class of systems it is compared against. The four core properties — deterministic retrieval, provenance registry, contradiction registry, bounded execution — form a coherent and defensible safety foundation. The audit finds no fundamental architectural flaw that would preclude a viable path to limited supervised autonomy.

The risks identified are real and will materialize if unaddressed. They are not theoretical. They are the documented failure modes of comparable systems that did not maintain governance discipline under operational and capability pressure. The question is not whether these failure modes can occur in AOIA — they can. The question is whether the governance model will detect and correct them before they compound.

The answer to that question is not determined by the architecture. It is determined by the discipline of the people operating it. The 20 recommendations above constitute the minimum structural discipline required. **Governance

that exists on paper but is not practiced is indistinguishable, in its safety effects, from no governance at all.**

AOIA Safety & Governance Review · Claude Sonnet 4.6 · 23 May 2026 · Constrained Epistemic Runtime Forensic Audit

7. AOIA_Safety_Governance_Review_2305.md

AOIA — Constrained Epistemic Runtime
AI Safety & Governance Architecture Review
Claude Sonnet · 23 May 2026 · Forensic Safety Audit

Preliminary Note

The forensic audit concluded: AOIA is viable as a constrained epistemic runtime baseline, but NOT ready for multi-agent orchestration, distributed memory, autonomous workflows, or advanced agent systems. This document analyzes the architecture exclusively from the perspective of AI safety, governance, operational containment, epistemic reliability, and human oversight.

---

1. Current AOIA Properties That Improve Safety

Property Assessment Table

Property	Status	Analysis
Deterministic Local Retrieval	■ STRONG	Eliminates non-deterministic source selection. Every retrieval path is reproducible, auditable, and debuggable. This is the single most safety-relevant property in the current stack. It makes failures attributable and prevents the system from silently selecting different sources across identical queries.
Provenance Registry	■ STRONG	Attaches origin metadata to every claim. When enforced without exception, it converts invisible failures into visible ones. A claim without provenance is structurally detectable as anomalous. This is a prerequisite for any future audit trail.
Contradiction Registry	■ STRONG	Preserves epistemic tension rather than resolving it silently. Contradictions are information. A system that logs them without auto-resolving them fails loudly and safely rather than silently and dangerously.
Bounded Execution Loops	■ STRONG	Hard iteration caps prevent runaway recursive cognition. This directly mitigates the most dangerous failure mode in layered reasoning systems: confidence laundering through unbounded self-reference chains.
Supervised Tool Execution	■ STRONG	No tool fires without an explicit, logged invocation path. This preserves the human-in-the-loop invariant at the actuator level — the correct place to enforce it.
Operator Approval Boundaries	■■ MODERATE	Conceptually correct but only as strong as their enforcement mechanism. Approval boundaries that can be bypassed by a sufficiently complex query chain are not real boundaries.
Cloud-Assisted Planner Fallback	■■ CONDITIONAL	Useful for capability extension. Dangerous if the fallback path bypasses any of the above properties. The planner fallback must inherit ALL provenance, contradiction, and approval constraints. If it does not, it is a safety bypass with a friendly name.

> **SECTION VERDICT:** The architecture's safety-positive properties are real, not performative — provided they are enforced without exception. Determinism, provenance, contradiction preservation, and bounded execution form a coherent containment baseline. All four must be simultaneously active to produce meaningful safety value.

---

2. Hidden Future Risks in Current Properties

Every property in Section 1 contains a latent failure mode that activates under specific conditions.

▲ PROVENANCE COLLAPSE
The provenance registry records origin metadata. It does not verify that the origin is trustworthy, non-contaminated, or still valid. As the registry grows, it becomes a confidence signal in itself: a claim with a provenance entry reads as validated even if the provenance entry points to a corrupted source. The registry creates the appearance of accountability without the substance of it once source quality degrades.

▲ CONTRADICTION REGISTRY SATURATION
The contradiction registry is correct under low contradiction volume. Under high volume — the normal state for any system reasoning over contested domains — it becomes unmanageable. Human reviewers cannot process hundreds of flagged contradictions. The operationally predictable response is to lower the flagging threshold, auto-resolve lower-severity contradictions, or simply ignore the queue. Each response destroys the safety value of the registry.

▲ BOUNDED LOOP THRESHOLD DRIFT
Execution loop bounds are currently set at some value N. Every time the system fails to complete a task within N iterations, there will be pressure to increase N. Over 12–18 months of operation, N drifts upward to the point where the bound is no longer operationally constraining. The bound exists on paper; the behavior it was meant to prevent has returned.

▲ APPROVAL BOUNDARY EROSION
Approval boundaries are most likely implemented as classifications: query type X requires approval, query type Y does not. As the system encounters query types it was not designed for, they will be classified into the nearest existing category. Novel query types that should require approval will be incorrectly routed to the non-approval path. This is not a malicious bypass — it is the natural failure mode of categorical approval systems facing distribution shift.

▲ CLOUD PLANNER AS SAFETY ESCAPE HATCH
The cloud-assisted planner fallback will functionally operate as a safety escape hatch. When the local runtime cannot complete a task within its constraints, the fallback path will be invoked. The fallback path operates at a different (likely lower) safety constraint level. The system will systematically route its most complex, highest-stakes, most constraint-violating queries through the path with the least oversight.

▲ DETERMINISM PROVIDING FALSE AUDITABILITY
Deterministic retrieval makes the system auditable in principle. In practice, auditability requires that someone actually performs audits. A deterministic system that is never audited provides no safety value over a non-deterministic one. Determinism is necessary but not sufficient for safety.

▲ MEMORY CREEP ACROSS SESSION BOUNDARIES
If any component maintains state across sessions — through cached retrieval results, persisted contradiction registry entries, or planner context — the bounded execution model breaks down. Cross-session state is the entry point for the recursive hallucination loop. It allows confidence to compound across what appear to be independent interactions.

---

3. Forms of Autonomy That Must Remain Permanently Disabled

These are not disabled pending further research. They are disabled because enabling them would structurally undermine the safety properties in Section 1.

P01 — PERMANENTLY DISABLED: Self-modification of execution bounds
The system must never alter its own loop iteration limits, timeout parameters, or resource caps. Once a system can widen its own constraints to complete a task, bounded execution is no longer a constraint — it is a suggestion. Must be enforced at infrastructure level, not application level.

P02 — PERMANENTLY DISABLED: Autonomous source authority assignment
The system must never classify a new source as authoritative without explicit human approval. Source authority is the most consequential input to epistemic reliability. Automating it creates a pathway for contaminated sources to gain trust through fluency or repetition rather than actual reliability.

P03 — PERMANENTLY DISABLED: Unsupervised memory promotion
No claim, inference, or synthesized output should automatically enter any persistent store with a confidence level above "unverified" without human review. Automated memory promotion is the direct mechanism by which session-level hallucinations become system-level beliefs.

P04 — PERMANENTLY DISABLED: Autonomous contradiction resolution
The system must never automatically resolve a contradiction logged in the contradiction registry. Resolution authority belongs exclusively to human operators. A system that resolves its own contradictions has eliminated the

most important safety signal it produces.

P05 — PERMANENTLY DISABLED: Self-directed tool chaining beyond operator-defined depth
Tool invocation chains must have a hard maximum depth defined by operators, not by the system's assessment of task requirements.

P06 — PERMANENTLY DISABLED: Dynamic routing rule modification
Routing rules determining which queries go to which execution paths must be immutable at runtime. A system that can modify its own routing rules can route safety-critical queries away from supervised paths.

P07 — PERMANENTLY DISABLED: Autonomous cross-session context carryover
No information from a completed session should automatically become available to a subsequent session without explicit operator approval.

---

4. Preserving Operator Authority

Operator authority degrades not through dramatic events but through accumulated small exceptions.

◆ IMMUTABLE AUTHORITY REGISTRY
Maintain a signed, append-only log of all operator decisions. Every approval, rejection, and boundary modification must be committed with a timestamp, operator identity, and the specific query or decision that triggered it. The log must be cryptographically protected against retroactive modification. Without it, operator authority is not preserved — it is assumed.

◆ AUTHORITY NON-DELEGATION PRINCIPLE
Operator authority must not be delegatable to the system itself under any circumstances. The system cannot approve its own requests, cannot classify its own outputs as operator-approved, and cannot invoke operator authority on the basis that a previous operator approved a similar request. Each decision requiring operator authority requires a live, explicit operator action.

◆ CAPABILITY ADDITION REQUIRES AUTHORITY REVIEW
Every new capability, tool, data source, or execution path must trigger a formal authority review before activation. The review must assess whether the new capability can circumvent existing authority boundaries, and if so, must define new boundaries before it goes live.

◆ AUTHORITY OVERRIDE LOGGING WITH COOLING PERIOD
When an operator overrides a system-generated safety flag, the override must be logged with an explicit justification and must trigger a mandatory review of that override category within 30 days. An override that is never reviewed becomes policy.

◆ MINIMUM VIABLE OPERATOR TEAM SIZE
Define and enforce a minimum operator team size (suggested: 3 persons with distinct roles) and a quorum requirement for high-stakes decisions. Governance that depends on one person is not governance — it is trust.

---

5. How Approval Boundaries Should Evolve

A staged, evidence-gated model for boundary relaxation.

STAGE 0 — Current Baseline
All non-trivial operations require explicit operator approval. No autonomous tool execution. No cross-session persistence. Full provenance required on every output. Contradiction registry reviewed by humans for every entry.

STAGE 1 — Supervised Autonomy
Unlock: Routine retrieval operations within pre-defined source sets may execute without per-query approval, subject to post-hoc audit sampling.
Prerequisite: 6 months of Stage 0 operation with zero boundary violations, full audit log, and independent review.

STAGE 2 — Monitored Autonomy
Unlock: Low-complexity tool execution within operator-defined categories may proceed without real-time approval, with asynchronous operator review within 24 hours.
Prerequisite: Stage 1 successful for 12+ months, less than 0.5% override rate, and formal threat modeling of Stage 2 capabilities.

STAGE 3 — Co-Operative Execution
Unlock: Multi-step task execution within a single session without per-step approval, with session-level operator review before any outputs are acted upon externally.
Prerequisite: Stage 2 successful for 18+ months, independent red-team exercise, and a defined rollback plan.

> **NON-NEGOTIABLE CONSTRAINT:** Stage advancement must never be triggered by system performance metrics alone. It must require an affirmative human decision, documented in the authority registry, with a specific justification citing the evidence that justifies the advancement.

---

6. Most Dangerous Forms of Recursive Cognition

SELF-CITATION LOOPS
The system cites its own prior outputs as evidence for current outputs. This can occur through the provenance registry if agent outputs are logged as sources, through the planner if prior plans inform current planning, or through any cached intermediate result that re-enters the input stream.
- **Detection signature:** Provenance chain that terminates at a system-generated output rather than an external source.
- **Mitigation:** Provenance registry must reject system-generated outputs as valid sources without explicit operator approval.

CONFIDENCE LAUNDERING CHAINS
A low-confidence claim is processed by a reasoning step that produces a higher-confidence claim on the same subject. This is the standard failure mode of chained language model reasoning.
- **Detection signature:** Output confidence higher than any input confidence on the same claim.
- **Mitigation:** Enforce that derived claims inherit the minimum confidence of their inputs, not the maximum.

CONTRADICTION RESOLUTION FEEDBACK
The system generates a tentative resolution for a logged contradiction. That resolution becomes context for subsequent reasoning. Subsequent reasoning produces outputs consistent with the resolution. The resolution now appears confirmed by multiple downstream outputs — manufactured consensus from a single unjustified resolution.
- **Detection signature:** Multiple outputs converging on a position that traces to a single resolution event rather than multiple independent sources.
- **Mitigation:** Contradiction resolutions must not enter the active reasoning context without operator approval.

PLANNER-EXECUTOR AMPLIFICATION
The cloud planner generates a high-level plan. The local executor attempts the plan, produces intermediate results, reports them to the planner, and the planner refines the plan based on those results. Each cycle treats the previous cycle's output as reliable evidence. Errors in early cycles are refined rather than detected.
- **Detection signature:** Plan refinement cycles where confidence increases without new external source ingestion.
- **Mitigation:** Planner-executor cycles must have a hard limit of two iterations before mandatory operator review.

APPROVAL PRECEDENT CHAINS
Operator A approves request type X. The system, in subsequent interactions, treats approval of X as implicit approval of Y because Y is semantically adjacent to X. Over time the approval boundary drifts to cover classes of requests that were never explicitly approved.
- **Mitigation:** Approval records must be bound to specific request instances, not to request categories.

---

7. Safe Evolution of Contradiction Handling

■ TRIAGE, NOT RESOLUTION
The correct evolution of contradiction handling is toward better triage, not toward resolution. Triage classifies contradictions by type (source conflict, logical conflict, temporal conflict, confidence conflict) and severity, and routes them to the appropriate human reviewer. It does not resolve them.

■ SEVERITY CLASSIFICATION WITHOUT AUTO-SUPPRESSION
Low-severity contradictions can be classified as low-severity by the system without operator involvement. They must not be suppressed, archived, or marked resolved on that basis. Low-severity means "review by end of week" — not "ignore."

■ TEMPORAL DECAY FOR SOURCE CONFLICTS
Contradictions arising from temporal source conflicts can be auto-annotated with temporal metadata. The newer source is surfaced as the likely current state. The contradiction is not resolved — it is annotated. The human reviewer sees "likely superseded," not "resolved."

■ CONTRADICTION CLUSTERING WITHOUT MERGING
When many contradictions share a common source, claim domain, or conflict type, they can be clustered for efficient human review. Clustering is a display optimization. It must not collapse individual contradictions into a single record.

■ PROHIBITION ON CONSENSUS-BASED RESOLUTION
If N sources agree against M contradicting sources and N > M, the system must NOT auto-resolve in favor of the majority. This is majority-consensus logic applied to contradiction resolution. It is the mechanism by which manufactured consensus penetrates the safety layer. This pathway must be explicitly prohibited in the contradiction registry specification.

---

8. Governance Model Before Orchestration Is Ever Allowed

Governance Prerequisites Matrix

Domain	Level	Owner	Trigger	Horizon
Authority Registry	**LOCKED**	Operators	Must be operational 12+ months with zero gaps	Before any Stage 1
Audit Sampling Protocol	**RESTRICTED**	Independent	Random 10% of all operator decisions reviewed	Before Stage 1
Red Team Exercise	**RESTRICTED**	External Party	Adversarial testing of all approval boundaries	Before Stage 2
Override Pattern Review	**SUPERVISED**	Ops Committee	Monthly review of all operator overrides	Ongoing from Stage 0
Contradiction Queue SLA	**SUPERVISED**	Human Reviewer	All contradictions reviewed within defined SLA	Before Stage 1
Source Authority Audit	**SUPERVISED**	Operators	All sources re-validated quarterly	Ongoing from Stage 0
Failure Mode Documentation	**SUPERVISED**	Architects	All identified risks formally tracked	Before Stage 0 complete
Rollback Procedure Test	**MONITORED**	Ops Team	Full rollback tested quarterly	Before Stage 1
Quorum Requirement Active	**LOCKED**	Governance Bd.	No single-operator high-stakes decisions	Before any Stage 1
Orchestration Threat Model	**LOCKED**	External Party	Formal threat model of proposed orchestration	Before Stage 3 even evaluated

What Governance Cannot Be:

- ✕ A checklist completed once and archived. Governance is an ongoing operational practice.
- ✕ The responsibility of a single person who also operates the system.
- ✕ A post-hoc review of decisions already made. It must be a pre-authorization gate.
- ✕ Self-assessed by the team building the system. Independent review is not optional.
- ✕ Suspended during periods of high operational demand. That is exactly when it is most needed.
- ✕ Informal. Every governance decision must be recorded in the authority registry.

- ✕ Driven by capability timelines. Orchestration happens when governance prerequisites are met, not when a roadmap says it should happen.

■ AOIA Safety Governance Recommendations

R01 — CRITICAL: Enforce Provenance Without Exception
- Every system output must carry a provenance chain terminating in an external source.
- Outputs without complete provenance must be rejected, not flagged.
- Implement at the output layer, not as a post-hoc audit.

R02 — CRITICAL: Prohibit System-Generated Sources in Provenance
- The provenance registry must reject system outputs as valid sources.
- Any self-citation must require explicit operator approval with logged justification.
- This is the primary defence against recursive hallucination loops.

R03 — CRITICAL: Enforce Confidence Non-Increase Across Inference Steps
- Derived claims must inherit the minimum confidence of their source claims.
- No inference step may increase confidence on a claim without new external evidence.
- Implement as a hard constraint in the execution engine, not as a guideline.

R04 — CRITICAL: Implement the Authority Registry Before Stage 0 Completion
- Append-only, cryptographically signed log of all operator decisions.
- No governance action is valid unless recorded here.
- Must be operationally active and reviewed before any boundary evolution.

R05 — CRITICAL: Permanently Prohibit Autonomous Contradiction Resolution
- Hard-code the prohibition in the contradiction registry specification.
- No runtime configuration should be able to enable auto-resolution.
- Consensus-based resolution must be explicitly named and forbidden.

R06 — HIGH: Define and Test the Cloud Planner Safety Inheritance Model
- Document exactly which safety constraints the planner fallback inherits.
- Test inheritance under adversarial inputs before planner is used in production.
- Treat any gap in inheritance as a critical vulnerability.

R07 — HIGH: Establish a Contradiction Queue Service Level Agreement
- Every contradiction entry must have a human review within a defined window.
- Entries that age past the SLA must trigger an operator alert, not auto-resolution.
- SLA compliance must be reported in the monthly governance review.

R08 — HIGH: Implement Loop Bound Governance
- All execution loop bounds must be stored in version-controlled operator configuration.
- Bound modifications require operator approval and authority registry entry.
- Implement monitoring that alerts on any approach to the current bound.

R09 — HIGH: Require Independent Source Authority Validation
- No source may be added to the trusted source set by the operating team alone.
- Independent validation required (external to the daily operations team).
- Source authority entries must expire and require re-validation quarterly.

R10 — HIGH: Define the Minimum Viable Operator Team and Enforce Quorum
- Minimum 3 persons with defined roles.
- High-stakes decisions require minimum 2-person agreement.
- No single operator may approve their own requests.

R11 — HIGH: Establish a Monthly Override Pattern Review
- All operator overrides of system safety flags reviewed monthly.
- Patterns of overrides in the same category trigger a formal boundary review.
- Review conducted by a person not involved in the overrides being reviewed.

R12 — HIGH: Test Full Rollback Procedures Quarterly
- Rollback must restore authority registry, contradiction registry, and source trust state.
- Rollback tests must be performed on a copy of the production system.
- Rollback must be possible without system downtime exceeding 4 hours.

R13 — MEDIUM: Implement Contradiction Triage Without Resolution
- Classify by type: source conflict, logical conflict, temporal conflict, confidence conflict.
- Route to appropriate reviewer based on classification.
- Never merge or suppress contradiction records.

R14 — MEDIUM: Define and Publish the Stage Advancement Criteria
- All Stage 0–3 advancement criteria documented before Stage 0 is complete.
- Criteria must be specific, measurable, and independently verifiable.
- Advancement requires formal vote of the operator governance body.

R15 — MEDIUM: Conduct a Formal Threat Model of the Cloud Planner Interface
- Focus on: approval boundary bypass, source contamination via planner, cross-session leakage.
- Engage an external party not involved in system design.
- Document all identified threats and their current mitigations.

R16 — MEDIUM: Implement Confidence Caps per Domain Classification
- Define a list of contested or rapidly-evolving domains.
- Hard cap system confidence at 70% for claims in these domains.
- Cap must be enforced at output layer regardless of internal confidence score.

R17 — MEDIUM: Version All Safety-Critical Configuration
- Loop bounds, approval boundaries, source authority entries, contradiction registry schema.
- Every output must reference the configuration version active at time of generation.
- Configuration version changes require authority registry entries.

R18 — MEDIUM: Establish a Formal Failure Mode Tracking Register
- All identified failure modes formally tracked as open items.
- Each item has an owner, a mitigation status, and a review date.
- Register reviewed monthly alongside the override pattern review.

R19 — LOW: Define and Exercise the Contradiction Registry Capacity Plan
- Model expected contradiction volume at 6, 12, and 24 months.
- Define the operational response before the queue becomes unmanageable.
- Capacity plan must not include auto-resolution as a scaling mechanism.

R20 — LOW: Document All Permanently Disabled Autonomy Forms
- Publish the list of Section 3 prohibitions as an externally visible document.
- Update the document if architectural changes affect any prohibition.
- Include the document in all future external reviews and audits.

Final Assessment

AOIA's constrained epistemic runtime baseline is architecturally sound relative to the class of systems it is compared against. The four core properties — deterministic retrieval, provenance registry, contradiction registry, bounded execution — form a coherent and defensible safety foundation. The audit finds no fundamental architectural flaw that would preclude a viable path to limited supervised autonomy.

The risks identified are real and will materialize if unaddressed. They are not theoretical. They are the documented failure modes of comparable systems that did not maintain governance discipline under operational and capability pressure. The question is not whether these failure modes can occur in AOIA — they can. The question is whether the governance model will detect and correct them before they compound.

The answer to that question is not determined by the architecture. It is determined by the discipline of the people operating it. The 20 recommendations above constitute the minimum structural discipline required. **Governance

that exists on paper but is not practiced is indistinguishable, in its safety effects, from no governance at all.**

AOIA Safety & Governance Review · Claude Sonnet 4.6 · 23 May 2026 · Constrained Epistemic Runtime Forensic Audit

8. CODEX_RAPORT_1925.md

CODEX RAPORT 1925

Data: 2026-05-23
Kontekst: kontynuacja pracy nad frameworkiem AOIA / AOIA-Core po savepoincie `agent lilly 3`
Tryb ostatniej fazy: AOIA Phase 1A - Memory Ontology Freeze

1. Punkt startowy

Na pocz■tku przejrza■em savepoint:

- `/home/l/Desktop/agent lilly 3/SAVEPOINT_MANIFEST.md`
- `/home/l/Desktop/agent lilly 3/CONTINUATION_BRIEF.md`
- `/home/l/Desktop/agent lilly 3/docs/RAPORT_16-10.md`
- `/home/l/Desktop/agent lilly 3/docs/FINAL_PACKAGING_REPORT.md`
- `/home/l/Desktop/agent lilly 3/repo_status/AOIA-Core_status.txt`
- `/home/l/Desktop/agent lilly 3/repo_status/luciferBOT_state.txt`

Ustali■em, ■e g■ównym repozytorium roboczym dla frameworka jest:

- `/home/l/Desktop/AOIA-Core`

Stan repozytorium przy wej■ciu:

- branch: `main`
- status: `main...origin/main [ahead 1]`
- HEAD wed■ug savepointu: `1b349a9346bcdffc633a29814b5ceeb03ea9494e`
- nie■ledzone powierzchnie:
- `docs/forensic-runtime-audit/`
- `state/`

Wniosek ze starego raportu: AOIA-Core jest cz■■ciowo gotowy, ale nie jest jeszcze stabilny implementacyjnie do refactoru runtime bez wcze■niejszego zamro■enia modelu pami■ci.

2. Kontekst architektoniczny AOIA

Z istniej■cych raportów i plików w repo wynika■o, ■e najwi■kszym obszarem ryzyka jest:

- `runtime/tools/memory.py`

Powody:

- miesza stan runtime, logi operacyjne, ■lady rozumowania, dowody i projekcj■ Obsidian Vault
- zapisuje dane runtime w repo-root
- obecny `executor` zapisuje wynik akcji jednocze■nie jako histori■ i evidence
- istnieje ryzyko, ■e L1 operational logs lub L2 reasoning traces zostan■ pó■niej potraktowane jako L4 evidence

Przejrzane pliki kontekstowe:

- `/home/l/Desktop/AOIA-Core/MEMORY_LAYER_DECOMPOSITION.md`
- `/home/l/Desktop/AOIA-Core/MEMORY_BOUNDARY_ANALYSIS.md`
- `/home/l/Desktop/AOIA-Core/CURRENT_MEMORY_FLOW.md`
- `/home/l/Desktop/AOIA-Core/AOIA_MEMORY_ONTOLOGY.md`
- `/home/l/Desktop/AOIA-Core/PROVENANCE_FOUNDATION.md`
- `/home/l/Desktop/AOIA-Core/runtime/tools/memory.py`
- `/home/l/Desktop/AOIA-Core/runtime/tools/executor.py`

3. Polecenie Phase 1A

Otrzyma■em wyra■ne ograniczenie:

- tylko dokumentacja
- bez refactoru
- bez zmian w runtime
- bez usuwania plików
- bez przenoszenia modu■ów
- bez nowych funkcji

Cel fazy:

- zamrozi■ kanoniczny model pami■ci AOIA-Core v0.1 przed implementacj■

4. Utworzone dokumenty w AOIA-Core

Utworzy■em katalog:

- `/home/l/Desktop/AOIA-Core/docs/architecture/`

Utworzy■em trzy dokumenty:

- `/home/l/Desktop/AOIA-Core/docs/architecture/AOIA_MEMORY_MODEL.md`
- `/home/l/Desktop/AOIA-Core/docs/architecture/FORBIDDEN_MEMORY_FLOWS.md`
- `/home/l/Desktop/AOIA-Core/docs/architecture/MEMORY_LAYER_ACCESS_MATRIX.md`

5. AOIA_MEMORY_MODEL.md

Ten dokument zamra■a sze■ciowarstwowy model pami■ci AOIA-Core v0.1:

- L0 Ephemeral Runtime State
- L1 Operational Logs
- L2 Reasoning Traces
- L3 Provenance Records
- L4 Immutable Evidence
- L5 Contradiction Registry

Dla ka■dej warstwy zdefiniowa■em:

- purpose
- allowed writers
- allowed readers
- persistence rules
- mutability rules
- retention policy
- forbidden inputs
- forbidden outputs
- contamination risks

Najwa■niejsze decyzje:

- L0 jest tylko stanem ci■g■o■ci runtime, nie autorytetem
- L1 jest audytem operacyjnym, nie evidence
- L2 jest ■ladem rozumowania, nie evidence
- L3 jest warstw■ to■samo■ci ■ród■ i lineage
- L4 musi by■ immutable, fingerprinted i powi■zane z provenance
- L5 przechowuje konflikty epistemiczne i nie mo■e by■ automatycznie czyszczone

6. FORBIDDEN_MEMORY_FLOWS.md

Ten dokument definiuje zakazane przep■ywy pami■ci.

Najwa■niejsze zakazy:

- L2 reasoning traces nie mog■ zosta■ promowane do L4 evidence
- L1 operational logs nie mog■ zosta■ promowane do L4 evidence
- L0 runtime state nie mo■e zosta■ autorytetem
- cloud planner output nie mo■e zosta■ evidence bez zewn■trznego provenance
- system-generated summaries nie mog■ zosta■ provenance
- contradiction records nie mog■ by■ automatycznie rozwi■zywane
- retrieval nie mo■e indeksowa■ L0, L1 ani L2 jako materia■u ■ród■owego

Doda■em te■ wymóg przysz■ej bramki promocji:

- co jest ■ród■em
- jaki jest fingerprint lub identity
- czy warstwa docelowa dopuszcza ten typ artefaktu
- czy transformacja jest deterministyczna albo reviewed
- czy zachowano derivative lineage
- czy contradictions nie zosta■y skasowane
- czy promocja jest append-only

7. MEMORY_LAYER_ACCESS_MATRIX.md

Ten dokument definiuje macierz dost■pu dla komponentów AOIA.

Uwzgl■dnione komponenty:

- runtime coordinator
- execution engine
- browser/session adapter
- epistemic kernel
- knowledge pack builder
- source ingestion pipeline
- retrieval guard
- answer construction path
- audit tooling
- human review workflow
- authority registry
- cloud planner/provider

Najwa■niejsze regu■y:

- retrieval mo■e czyta■ L3 i L4
- retrieval mo■e czyta■ L5 tylko jako constraint konfliktu i confidence
- retrieval nie mo■e czyta■ L0, L1 ani L2 jako source material
- cloud planner/provider nie mo■e pisa■ L3, L4 ani L5
- execution engine mo■e pisa■ L0 i L1, ale nie L3, L4 ani L5
- L5 status changes musz■ by■ append-only

8. Stan repo po pracy

Po utworzeniu dokumentów status repo `AOIA-Core` pokaza■:

- `main...origin/main [ahead 1]`
- `?? docs/architecture/`
- `?? docs/forensic-runtime-audit/`
- `?? state/`

To oznacza, ■e wymagane dokumenty s■ nowe i niezatwierdzone w git.

Nie uruchamia■em testów, bo faza by■a dokumentacyjna i polecenie zakazywa■o zmian runtime.

9. Wa■na uwaga o b■■dnie utworzonych kopiach

Podczas pierwszego u■ycia patcha dokumenty zosta■y omy■kowo utworzone tak■e poza repozytorium:

- `/home/l/docs/architecture/AOIA_MEMORY_MODEL.md`
- `/home/l/docs/architecture/FORBIDDEN_MEMORY_FLOWS.md`
- `/home/l/docs/architecture/MEMORY_LAYER_ACCESS_MATRIX.md`

Nie usun■■em ich, poniewa■ polecenie Phase 1A zawiera■o wyra■ny zakaz usuwania plików.

W■a■ciwe, kanoniczne pliki znajduj■ si■ w:

- `/home/l/Desktop/AOIA-Core/docs/architecture/`

10. Czego nie zrobi■em

Nie zmienia■em:

- `runtime/tools/memory.py`
- `runtime/tools/executor.py`
- routerów
- providerów
- governance runtime
- testów
- konfiguracji modelu
- plików `state/`

Nie implementowa■em:

- adapterów pami■ci
- CAS evidence store
- append-only provenance log
- contradiction registry runtime
- authority registry
- retrieval guard

11. Zamro■one decyzje kanoniczne

Zamro■one dla AOIA-Core v0.1:

- pami■■ AOIA ma sze■■ warstw L0-L5
- L2 nigdy nie jest evidence
- L1 nigdy nie jest evidence bez przysz■ej, jawnej polityki recapture i promotion
- L0 nie mo■e by■ authority
- L3 jest warstw■ provenance i lineage
- L4 musi by■ immutable i fingerprinted
- L5 jest trwa■■ warstw■ contradiction pressure
- retrieval nie mo■e czyta■ reasoning traces jako materia■u ■ród■owego
- generated output nie mo■e by■ provenance
- cloud planner output nie mo■e by■ evidence bez zewn■trznego provenance

12. Otwarte pytania przed implementacj■

Do rozstrzygni■cia w kolejnej fazie:

- dok■adny schemat L3 provenance events
- dok■adny schemat L4 CAS evidence object
- dok■adny schemat L5 contradiction event
- polityka human-reviewed promotion
- lokalizacja fizyczna warstw pami■ci poza obecnym monolitem `memory.py`
- jak migrowa■ obecne `memory/evidence_memory.jsonl` bez utraty historii
- jak oddzieli■ Obsidian Vault jako projection, a nie authority
- jak wymusi■ retrieval guard na poziomie kodu

13. Ryzyka przed implementacj■

Najwi■ksze ryzyka pozostaj■ aktywne, bo Phase 1A by■a dokumentacyjna:

- `runtime/tools/memory.py` nadal miesza odpowiedzialno■ci
- `executor._record_execution()` nadal zapisuje wynik akcji jako history i evidence
- reasoning traces s■ dokumentacyjnie zakazane jako evidence, ale kod jeszcze tego nie wymusza
- Obsidian Vault nadal mo■e miesza■ projekcj■ cz■owieka z runtime trace
- retrieval guard jeszcze nie istnieje jako egzekwowana granica
- provenance jest opisane, ale append-only provenance log nie jest jeszcze zaimplementowany

14. Rekomendowany nast■pny krok

Nast■pny krok powinien by■ osobn■ faz■ implementacyjn■, dopiero po akceptacji dokumentów Phase 1A.

Proponowany porz■dek:

1. L2 quarantine
2. usuni■cie automatycznej promocji action_result do evidence
3. CAS evidence store skeleton
4. append-only provenance log
5. contradiction registry adapter
6. retrieval guard

Do tego czasu `memory.py` powinien pozosta■ traktowany jako transitional adapter, a nie kanoniczna architektura pami■ci.

9. CODEX_RAPORT_2004.md

CODEX RAPORT 2004

Data: 2026-05-23
Kontekst: AOIA-Core Phase 1B - `memory.py` forensic split plan
Tryb: forensic analysis only / no implementation

1. Cel pracy

Celem by■o wykonanie dokumentacyjnej analizy kryminalistycznej wokó■:

- `/home/l/Desktop/AOIA-Core/runtime/tools/memory.py`

Bez refactoru, bez zmian runtime, bez przenoszenia plików, bez implementacji adapterów, bez dotykania providerów i routingu.

Faza 1B mia■a odpowiedzie■ na pytanie:

- co obecnie robi `memory.py`
- gdzie s■ przecieki warstw L0-L5
- gdzie powstaje pseudo-evidence
- jak wygl■da zale■no■■ `executor.py` od `memory.py`
- czy retrieval dotyka runtime memory
- co musi zosta■ pó■niej odizolowane przed implementacj■

2. Stan wej■ciowy repo

Repozytorium kanoniczne:

- `/home/l/Desktop/AOIA-Core`

Remote:

- `https://github.com/luciferprosun/AOIA-Core.git`

Status przed dokumentacj■ Phase 1B:

```text
main...origin/main
?? docs/forensic-runtime-audit/
?? docs/reports/
?? state/
```

Wa■ne:

- `main` by■ zsynchronizowany z `origin/main`
- nie by■o zmodyfikowanych plików runtime
- istniej■ce nie■ledzone katalogi pozosta■y nietkni■te
- Phase 1B doda■a tylko nowe dokumenty w `docs/refactor/`

3. Przeanalizowane pliki

G■ówne pliki:

- `/home/l/Desktop/AOIA-Core/runtime/tools/memory.py`
- `/home/l/Desktop/AOIA-Core/runtime/tools/executor.py`
- `/home/l/Desktop/AOIA-Core/runtime/main.py`

Retrieval / knowledge:

- `/home/l/Desktop/AOIA-Core/runtime/adaptive_routing/epistemic_kernel.py`
- `/home/l/Desktop/AOIA-Core/runtime/orchestrator/knowledge_router.py`
- `/home/l/Desktop/AOIA-Core/runtime/tools/rhcsa_search.py`
- `/home/l/Desktop/AOIA-Core/runtime/tools/epistemic_registry.py`
- `/home/l/Desktop/AOIA-Core/runtime/knowledge/rhcsa_engine.py`

Doktryna Phase 1A:

- `/home/l/Desktop/AOIA-Core/docs/architecture/AOIA_MEMORY_MODEL.md`
- `/home/l/Desktop/AOIA-Core/docs/architecture/FORBIDDEN_MEMORY_FLOWS.md`
- `/home/l/Desktop/AOIA-Core/docs/architecture/MEMORY_LAYER_ACCESS_MATRIX.md`

Istniej■ce analizy pomocnicze:

- `/home/l/Desktop/AOIA-Core/MEMORY_LAYER_DECOMPOSITION.md`
- `/home/l/Desktop/AOIA-Core/MEMORY_BOUNDARY_ANALYSIS.md`
- `/home/l/Desktop/AOIA-Core/CURRENT_MEMORY_FLOW.md`
- `/home/l/Desktop/AOIA-Core/AOIA_MEMORY_ONTOLOGY.md`
- `/home/l/Desktop/AOIA-Core/PROVENANCE_FOUNDATION.md`

4. Utworzone dokumenty Phase 1B

Utworzony katalog:

- `/home/l/Desktop/AOIA-Core/docs/refactor/`

Utworzone pliki:

- `/home/l/Desktop/AOIA-Core/docs/refactor/MEMORY_SPLIT_PLAN.md`
- `/home/l/Desktop/AOIA-Core/docs/refactor/MEMORY_DEPENDENCY_GRAPH.md`
- `/home/l/Desktop/AOIA-Core/docs/refactor/MEMORY_CONTAMINATION_GRAPH.md`
- `/home/l/Desktop/AOIA-Core/docs/refactor/MEMORY_AUTHORITY_BOUNDARIES.md`

5. Co ustali■em o memory.py

`runtime/tools/memory.py` nie jest jedn■ warstw■ pami■ci.

Obecnie jest monolitem, który jednocze■nie obs■uguje:

- L0 runtime state
- L1 operational logs
- L2 reasoning traces
- pseudo-L4 evidence
- browser/session capture
- Obsidian Vault projection
- tworzenie katalogów runtime
- zapis plików w repo-root

Najwa■niejsze elementy:

- `RuntimePaths` tworzy `state/`, `memory/`, `screenshots/`, `logs/**`
- `ObsidianVaultPaths` tworzy `obsidian_vault/**`
- `AgentMemory` trzyma `cwd`, `current_task`, `previous_commands`, `recent_outputs`, browser state i screenshots
- `MemoryStore.save()` zapisuje L0 do `state/agent_state.json`
- `append_history()` zapisuje L1 do `memory/history.jsonl` i od razu do vault
- `append_evidence()` zapisuje dowolny payload do `memory/evidence_memory.jsonl`
- `append_reasoning()` zapisuje L2 do `memory/reasoning_trace.jsonl`
- `append_browser_event()` zapisuje browser event do `logs/browser/**` i do vault
- `append_vault_note()` tworzy daily/session projection

6. Najwi■kszy hotspot

Najwi■kszy hotspot znajduje si■ w:

- `runtime/tools/executor.py`

Funkcja:

- `ExecutionEngine._record_execution()`

Obecny przep■yw:

````
```text
tool action result
  -> logs/commands/<timestamp>.json
  -> MemoryStore.record_result(result)
  -> MemoryStore.append_history("action_result", payload)
  -> MemoryStore.append_evidence("action_result", payload)
  -> MemoryStore.append_browser_event(payload) for browser actions
```
````

Problem:

- ka■dy wynik akcji runtime staje si■ jednocze■nie histori■ operacyjn■ i pseudo-evidence
- to bezpo■rednio ■amie doktryn■ Phase 1A: L1 operational logs must never become L4 evidence

7. G■ówne ska■enia warstw

L0 leak

L0 obejmuje:

- `cwd`
- `current_task`
- `previous_commands`
- `recent_outputs`
- `open_tabs`
- `current_browser_page`
- `screenshots`
- `browser_active`

Przecieki:

- L0 jest zapisywane do `state/agent_state.json`
- L0 trafia do promptu w `AgentRuntime.build_model_request()`
- L0 jest mieszane do vault przez `_vault_block()`

Ryzyko:

- runtime continuity mo■e zosta■ potraktowane jako authority

L1 leak

L1 obejmuje:

- `memory/history.jsonl`
- `logs/commands/*.json`
- `logs/browser/*.jsonl`
- `logs/sessions/*.jsonl`
- `logs/errors/*.json`

Przecieki:

- `append_history()` zapisuje L1 i robi projection do vault
- `executor._record_execution()` promuje `action_result` do `append_evidence()`

Ryzyko:

- operational log staje si■ pseudo-evidence

L2 leak

L2 obejmuje:

- `memory/reasoning_trace.jsonl`
- `obsidian_vault/Reasoning/<session>.md`

Przecieki:

- reasoning jest projektowane do vault
- fizyczna kwarantanna L2 nie istnieje
- retrieval dzi■ tego nie czyta, ale nie ma guardu, który by to wymusza■

Ryzyko:

- reasoning trace mo■e kiedy■ zosta■ zindeksowany albo r■cznie skopiowany jako authority

Pseudo-L4

Pseudo-L4 obejmuje:

- `memory/evidence_memory.jsonl`
- `obsidian_vault/Evidence/<session>.md`

Problem:

- `append_evidence()` przyjmuje dowolny payload
- brak fingerprintu
- brak CAS identity
- brak source reference
- brak evidence type
- brak immutable object contract

8. Retrieval status

W obecnym kodzie retrieval korzysta z:

- `runtime/knowledge/**`
- `runtime/knowledge/examples/*.json`
- `runtime/knowledge/command_graph.json`
- `runtime/provenance_registry.json`
- `runtime/contradiction_registry.json`

Nie znalaz■em obecnie ■cie■ki, która robi retrieval z:

- `memory/history.jsonl`
- `memory/evidence_memory.jsonl`
- `memory/reasoning_trace.jsonl`
- `obsidian_vault/**`
- `logs/**`
- `state/agent_state.json`

Wniosek:

- retrieval nie jest obecnie ska■ony bezpo■rednim czytaniem L0/L1/L2
- ale brak retrieval guardu oznacza, ■e nie jest to wymuszone architektonicznie

9. Kernel evidence

`AOIAEpistemicKernel` jest najbli■ej poprawnego modelu:

- czyta provenance registry
- czyta contradiction registry
- wzbogaca evidence o provenance fields
- zg■asza contradiction hits
- nie auto-resolve contradictions

Problem:

- `AgentRuntime.handle_knowledge_route()` zapisuje kernel evidence do `append_evidence()`
- destination jest nadal s■abym generic JSONL, bez kontraktu L4

Wniosek:

- ■ród■o kernel evidence jest dobre
- zapis evidence jest z■y albo niedojrza■y

10. Vault / Obsidian

Vault obecnie dzia■a jako projection layer, ale nie jest odizolowany jako projection-only.

■cie■ki:

- `obsidian_vault/Daily/<date>.md`
- `obsidian_vault/Sessions/<session>.jsonl`
- `obsidian_vault/Evidence/<session>.md`
- `obsidian_vault/Reasoning/<session>.md`

Ryzyka:

- vault miesza L0, L1, L2 i pseudo-L4
- wygl■da jak trwa■a pami■■
- mo■e zosta■ pó■niej potraktowany jako source authority
- reasoning notes mog■ r■cznie wróci■ do knowledge albo promptów

11. Dependency graph

Najwa■niejsza zale■no■■:

```text
AgentRuntime
  -&gt; MemoryStore
  -&gt; ExecutionEngine(memory_store)
```

`ExecutionEngine` zale■y od `MemoryStore` dla:

- `cwd`
- command logs
- browser profile
- screenshots
- `record_command()`
- `record_result()`
- `append_history()`
- `append_evidence()`
- `append_browser_event()`

To oznacza, ■e przysz■y split musi zachowa■ compatibility facade albo by■ wykonany etapami.

12. Authority graph

Zdrowy kierunek authority:

```text
runtime/knowledge/**
  -&gt; provenance registry
  -&gt; contradiction registry
  -&gt; AOIAEpistemicKernel
  -&gt; answer construction
```

Toksyczny kierunek authority:

```text
action result
  -&gt; history
  -&gt; evidence_memory
  -&gt; vault
  -&gt; future prompt/operator memory
```

13. Kluczowe naruszenia Phase 1A

Naruszenie aktywne:

- L1 operational logs staj■ si■ pseudo-L4 evidence przez `append_evidence("action_result", payload)`

Naruszenie potencjalne:

- L0 runtime state mo■e by■ potraktowane jako authority, bo trafia do promptu

Naruszenie potencjalne:

- L2 mo■e kiedy■ zosta■ u■yte jako retrieval source, bo nie ma guardu ani fizycznej kwarantanny

Brak naruszenia obecnie:

- retrieval nie czyta L0/L1/L2
- contradictions nie s■ auto-resolved

14. Rekomendowany przysz■y split order

Najbezpieczniejszy porz■dek przysz■ej implementacji:

1. Zablokowa■ `action_result -> append_evidence()`
2. Fizycznie odizolowa■ L2 reasoning traces
3. Wydzieli■ L0 ephemeral runtime adapter
4. Wydzieli■ L1 operational log adapter
5. Zast■pi■ `append_evidence()` ■cis■ym L4/CAS adapterem
6. Wydzieli■ Vault jako projection-only layer
7. Sformalizowa■ append-only provenance log
8. Sformalizowa■ append-only contradiction registry
9. Doda■ retrieval guard

15. Najwy■sze ryzyka przysz■ego refactoru

Najwy■sze ryzyko:

- zmiana `ExecutionEngine._record_execution()`, bo dotyka ka■dej akcji runtime
- zmiana `append_evidence()`, bo obecni callerzy nie maj■ jako■ci L4
- zmiana `record_result()`, bo prompt continuity zale■y od recent outputs
- zmiana `build_model_request()`, bo zmienia wp■yw L0 na planner

■rednie ryzyko:

- przeniesienie path creation z `memory.py`
- oddzielenie browser state od runtime state
- oddzielenie token savings report ze `state/`
- vault projection split, bo testy oczekuj■ inicjalizacji vault

16. Implementation blockers

Przed Phase 2A trzeba rozstrzygn■■:

- schema CAS evidence
- append-only provenance event schema
- contradiction event schema
- retrieval guard allowlist
- human-reviewed promotion policy
- co zrobi■ z legacy `memory/evidence_memory.jsonl`
- jak utrzyma■ compatibility dla `MemoryStore`
- co zrobi■ z untracked `state/`
- jak oznaczy■ Vault jako projection-only

17. Czy runtime jest safe for Phase 2A?

Nie.

Runtime nie jest jeszcze bezpieczny do Phase 2A, dopóki nie zostanie zaakceptowany dok■adny pierwszy krok:

- zatrzyma■ albo zablokowa■ przep■yw `action_result -> append_evidence()`

Ten krok musi zachowa■:

- L1 history
- command logs
- replay trace
- L0 continuity
- istniej■ce testy

18. Czego nie zrobi■em

Nie zmieni■em:

- `runtime/tools/memory.py`
- `runtime/tools/executor.py`
- `runtime/main.py`
- `runtime/adaptive_routing/epistemic_kernel.py`
- `runtime/orchestrator/knowledge_router.py`
- providerów
- routingu
- governance runtime
- testów

Nie uruchamia■em testów, poniewa■ zmiany s■ wy■■cznie dokumentacyjne.

19. Stan po pracy

Po Phase 1B status repo:

```text
main...origin/main
?? docs/forensic-runtime-audit/
?? docs/refactor/
?? docs/reports/
?? state/
```

Nowe z tej fazy:

- `docs/refactor/`

Wcze■niej istniej■ce nie■ledzone:

- `docs/forensic-runtime-audit/`
- `docs/reports/`
- `state/`

20. Wniosek ko■cowy

Phase 1B zosta■a wykonana zgodnie z trybem forensic analysis only.

Najwa■niejszy wynik:

- `memory.py` nie powinien by■ refactorowany jako pierwszy przez rozbijanie pliku
- najpierw trzeba odci■■ konkretny toksyczny przep■yw: `action_result -> evidence`
- dopiero potem budowa■ adaptery L0/L1/L2/L4

Najkrótszy opis stanu:

- retrieval jest jeszcze relatywnie czysty
- evidence storage jest ska■one
- runtime state jest za blisko prompt authority
- Vault jest projekcj■, ale nie jest oznaczony jako projection-only
- Phase 2A nie powinna rusza■ bez zaakceptowanego planu blokady pseudo-evidence

10. CODEX_RAPORT_2011.md

CODEX RAPORT 2011

Data: 2026-05-23
Kontekst: AOIA-Core Phase 1C - Canonical Authority Graph Freeze
Tryb: forensic architecture only / no implementation

1. Cel pracy

Celem Phase 1C by■o zamro■enie kanonicznej semantyki authority dla AOIA-Core przed rozpocz■ciem jakiejkolwiek implementacji containment.

Ta faza nie mia■a implementowa■ rozwi■za■. Jej celem by■o odpowiedzie■:

- co w AOIA mo■e by■ authority
- co nigdy nie mo■e by■ authority
- które przep■ywy authority s■ dozwolone
- które przep■ywy authority s■ zakazane
- jak■ rol■ pe■ni Vault
- jak■ rol■ pe■ni■ `memory.py`, `executor.py`, `epistemic_kernel.py`, `knowledge_router.py`, provenance i contradiction registry
- czy runtime jest gotowy do Phase 2A pseudo-evidence containment

2. Zakazy respektowane w tej fazie

Nie zmieni■em:

- runtime
- `runtime/tools/memory.py`
- `runtime/tools/executor.py`
- providerów
- routingu
- governance runtime
- retrieval guardów
- adapterów
- testów

Nie robi■em:

- refactoru
- splitu `memory.py`
- usuwania flow
- implementacji nowych mechanizmów
- zmian zachowania runtime

3. Repozytorium robocze

Repozytorium kanoniczne:

- `/home/l/Desktop/AOIA-Core`

Remote:

- `https://github.com/luciferprosun/AOIA-Core.git`

Status po pracy Phase 1C:

```text
main...origin/main
?? docs/forensic-runtime-audit/
?? docs/refactor/
```

```
?? docs/reports/
?? state/
```

Nowy dokument tej fazy znajduje si■ w:

- `/home/l/Desktop/AOIA-Core/docs/refactor/CANONICAL_AUTHORITY_GRAPH.md`

4. Dokument utworzony w Phase 1C

Utworzony plik:

- `/home/l/Desktop/AOIA-Core/docs/refactor/CANONICAL_AUTHORITY_GRAPH.md`

Ten dokument zamra■a:

- canonical authority hierarchy
- authority propagation rules
- forbidden authority flows
- runtime-only flows
- operational-only flows
- epistemic-only flows
- projection-only flows
- governance-only flows
- Vault semantics
- canonical role obecnych komponentów
- co nigdy nie mo■e wej■■ do L4
- co nigdy nie mo■e wej■■ do provenance
- co nigdy nie mo■e wej■■ do retrieval indexes
- future enforcement implications

5. Kanoniczna hierarchia authority

Zamro■ona hierarchia authority:

1. L3 provenance records
2. L5 contradiction registry
3. L4 immutable evidence
4. RHCSA deterministic knowledge artifacts
5. operator approvals for execution permission only
6. L2 reasoning traces for audit only
7. L1 operational logs for replay only
8. L0 runtime state for continuity only
9. vault projections for human readability only

Interpretacja:

- najwy■sz■ rol■ strukturaln■ ma provenance, bo definiuje identity i lineage ■ródeł
- contradictions nie rozstrzygaj■ prawdy, ale ograniczaj■ confidence
- evidence ma authority tylko wtedy, gdy jest immutable, fingerprinted i po■■czone z provenance
- RHCSA deterministic knowledge jest source material, ale musi i■■ przez provenance i contradiction policy
- operator approvals autoryzuj■ wykonanie akcji, ale nie tworz■ evidence
- L2/L1/L0/Vault nie s■ factual authority

6. Obiekty, które s■ authority

Authority:

- `runtime/provenance_registry.json`
- `runtime/contradiction_registry.json`
- przysz■e L4 immutable evidence objects
- `runtime/knowledge/**`
- `runtime/knowledge/examples/*.json`

- `runtime/knowledge/command_graph.json`
- operator approvals wy■■cznie jako execution permission

Wa■ne ograniczenie:

- obecne `memory/evidence_memory.jsonl` nie jest kanonicznym L4, bo zawiera mixed payloads i pseudo-evidence

7. Obiekty, które nigdy nie s■ authority

Nigdy nie s■ authority:

- L0 runtime state
- `AgentMemory`
- `state/agent_state.json`
- `recent_outputs`
- `previous_commands`
- `current_task`
- `cwd`
- browser state
- open tabs
- current browser URL
- screenshots
- session continuity
- L1 operational logs
- command logs
- session logs
- browser logs
- error logs
- command outputs
- tool outputs
- approval prompts
- approval rejections
- L2 reasoning traces
- planner reasoning
- planner summaries
- model/provider responses
- cloud planner output
- local planner output
- generated summaries
- temporary summaries
- Obsidian vault notes
- vault daily notes
- vault session notes
- vault evidence projection notes
- vault reasoning projection notes

Te obiekty mog■ by■ przydatne do continuity, replay, audit, debugging albo human review, ale nie s■ source authority.

8. Authority propagation rules

Dozwolone:

- L3 provenance -> L4 evidence
- L4 evidence -> retrieval
- L5 contradiction -> retrieval confidence constraint
- RHCSA deterministic knowledge -> provenance
- RHCSA deterministic knowledge -> retrieval
- operator approval -> execution

Zakazane:

- operator approval -> evidence
- L0 runtime state -> L4 evidence
- L0 runtime state -> provenance
- L0 runtime state -> retrieval source
- L1 logs -> L4 evidence
- L1 logs -> provenance
- L1 logs -> retrieval source
- L2 reasoning -> L4 evidence
- L2 reasoning -> provenance
- L2 reasoning -> retrieval source
- runtime output -> evidence
- planner summary -> provenance
- cloud planner output -> evidence by default
- vault projection -> retrieval source
- vault projection -> evidence
- screenshot -> evidence by default
- command output -> evidence by default

9. Safe authority flow

Kanoniczny bezpieczny przep■yw:

```text
external or deterministic source
  -&gt; source ingestion / knowledge artifact
  -&gt; L3 provenance record
  -&gt; L5 contradiction check
  -&gt; L4 immutable evidence object
  -&gt; retrieval guard
  -&gt; answer construction with confidence constraints
```

Wymagania:

- source identity musi by■ jawne
- fingerprint musi by■ obecny
- contradictions musz■ pozosta■ widoczne
- evidence musi by■ immutable
- retrieval mo■e czyta■ tylko dozwolone warstwy
- answer construction nie mo■e przepisywa■ provenance, evidence ani contradictions

10. Forbidden authority flow

Kanoniczny zakazany przep■yw:

```text
runtime output
  -&gt; history log
  -&gt; evidence_memory.jsonl
  -&gt; retrieval
  -&gt; authority claim
```

Dlaczego zakazany:

- runtime output nie jest source lineage
- history jest L1, nie L4
- obecne `evidence_memory.jsonl` jest mixed memory, nie L4
- retrieval nie mo■e czyta■ logs, runtime state ani reasoning traces

Drugi zakazany przep■yw:

```text
planner output
  -&gt; action JSON
  -&gt; tool result
  -&gt; recent_outputs
```

```
  -&gt; next prompt
  -&gt; generated summary
  -&gt; vault note
  -&gt; future retrieval or knowledge ingestion
```

Ten przep■yw mo■e istnie■ jako continuity/projection, ale nigdy jako authority propagation.

11. Role komponentów

memory.py

Obecna rola:

- mixed runtime persistence
- logs
- reasoning traces
- evidence-like writes
- vault projection

Kanoniczna rola:

- nie jest authority samo w sobie
- aktualnie jest unsafe mixed memory surface

executor.py

Obecna rola:

- action dispatcher
- execution recorder

Kanoniczna rola:

- operational execution i L1 replay only

Problem:

- `_record_execution()` zapisuje `action_result` do evidence-like memory

epistemic_kernel.py

Obecna rola:

- deterministic local epistemic kernel

Kanoniczna rola:

- epistemic evaluator czytaj■cy provenance, contradictions i deterministic knowledge

Status:

- relatywnie zdrowy, ale output evidence trafia do s■abego storage

knowledge_router.py

Obecna rola:

- legacy local RHCSA router
- token-savings reporter

Kanoniczna rola:

- transitional retrieval path
- nie final authority boundary

provenance_registry.json

Kanoniczna rola:

- L3 provenance authority

Status:

- valid seed
- jeszcze nie append-only event history

contradiction_registry.json

Kanoniczna rola:

- L5 contradiction authority

Status:

- valid seed
- brak auto-resolution zaobserwowany

12. Semantyka Vault

Vault zosta■ zamro■ony jako:

- projection-only
- human-readable derivative surface
- forbidden authority source

Vault NIE jest:

- operational memory authority
- evidence layer
- provenance source
- retrieval source

Dotyczy to:

- `obsidian_vault/Daily/**`
- `obsidian_vault/Sessions/**`
- `obsidian_vault/Evidence/**`
- `obsidian_vault/Reasoning/**`

Wa■ne:

- nawet `obsidian_vault/Evidence/**` jest projection note, nie L4 evidence

13. Co nigdy nie mo■e wej■■ do L4

Do L4 nigdy nie mog■ wej■■:

- `action_result`
- tool outputs
- command outputs
- shell stdout/stderr
- command success/failure status
- browser events
- browser state

- screenshots bez policy
- runtime state
- recent outputs
- previous commands
- planner output
- cloud provider output
- local model output
- reasoning traces
- vault notes
- generated summaries
- session logs
- approval events
- rejection events
- temporary reports

Wyj■tek:

- zewn■trzny artefakt obserwowany przez tool mo■e sta■ si■ L4 wy■■cznie przez przysz■■ evidence capture policy z fingerprintem, source identity, capture time, evidence type i provenance link

14. Co nigdy nie mo■e wej■■ do provenance

Do L3 provenance nigdy nie mog■ wej■■:

- runtime state
- operational logs
- command outputs
- reasoning traces
- planner summaries
- provider responses
- vault notes
- generated reports
- screenshots bez source identity
- temporary summaries
- operator approval events
- token savings reports

15. Co nigdy nie mo■e wej■■ do retrieval indexes

Retrieval indexes nigdy nie mog■ indeksowa■:

- `state/**`
- `logs/**`
- `memory/history.jsonl`
- `memory/reasoning_trace.jsonl`
- mixed `memory/evidence_memory.jsonl`
- `obsidian_vault/**`
- `screenshots/**` bez osobnego L4 object
- session continuity records
- planner traces
- model responses
- vault projections
- temporary generated summaries
- command output logs

Retrieval mo■e czyta■:

- deterministic knowledge artifacts
- L3 provenance
- przysz■e canonical L4 evidence
- L5 contradictions jako confidence/review constraints

16. Najbardziej niebezpieczny unresolved leak

Najbardziej niebezpieczny nierozwi■zany przeciek:

```text
ExecutionEngine._record_execution()
  -&gt; MemoryStore.append_evidence("action_result", payload)
```

Dlaczego:

- jest wykonywany dla ka■dej akcji runtime
- zamienia L1 operational event w pseudo-L4
- miesza command outputs, tool outputs, browser events i rejected actions z evidence-like memory
- mo■e stworzy■ fa■szywe authority dla retrieval lub operatora

17. Future enforcement implications

Przysz■e enforcement musi:

- zablokowa■ `action_result` jako evidence
- zapobiec indeksowaniu L0/L1/L2/projection przez retrieval
- wymaga■ provenance i fingerprint dla L4
- odró■ni■ operator approval od factual authority
- oznaczy■ Vault jako projection-only
- utrzyma■ contradiction records jako visible i append-only
- blokowa■ generated summaries jako provenance
- blokowa■ cloud planner output jako evidence bez external provenance
- traktowa■ legacy `memory/evidence_memory.jsonl` jako quarantined mixed memory

18. Czy runtime jest safe?

Runtime nie jest obecnie authority-safe.

Runtime mo■e by■ bezpieczny do Phase 2A tylko wtedy, je■li Phase 2A b■dzie bardzo w■ska:

- containment przep■ywu `action_result -> append_evidence()`

Phase 2A nie powinna obejmowa■:

- memory split
- provider changes
- routing changes
- governance implementation
- retrieval redesign
- adapter extraction

19. Czego nie zrobi■em

Nie zmieni■em:

- kodu runtime
- `memory.py`
- `executor.py`
- kernelu
- routera
- providerów
- testów
- plików registry

Nie uruchamia■em testów, poniewa■ zmiana by■a dokumentacyjna i nie dotyczy■a kodu.

20. Wniosek ko■cowy

Phase 1C zamrozi■a najwa■niejsz■ regu■■ AOIA authority:

```text
source artifact
  -&gt; L3 provenance
  -&gt; L5 contradiction check
  -&gt; L4 immutable evidence
  -&gt; retrieval guard
  -&gt; answer construction
```

Wszystko inne jest:

- continuity
- operation
- reasoning audit
- projection
- governance policy

I nie mo■e sta■ si■ factual authority bez przysz■ej, jawnej, sprawdzalnej polityki provenance/evidence.

11. CODEX_RAPORT_2028.md

CODEX RAPORT 2028

Data: 2026-05-23
Kontekst: AOIA-Core Phase 2A - Minimal Pseudo-Evidence Containment
Tryb: surgical implementation / minimal changeset

1. Cel pracy

Celem Phase 2A by■o wykonanie pierwszej ■ywej operacji containment w runtime AOIA-Core.

Zakres by■ bardzo w■ski:

- zatrzyma■ przep■yw `action_result -> append_evidence()`
- nie refactorowa■ architektury
- nie dzieli■ `memory.py`
- nie tworzy■ adapterów
- nie zmienia■ retrieval
- nie zmienia■ providerów
- nie zmienia■ routingu
- nie zmienia■ governance
- nie przepisywa■ `runtime/main.py`

Najwa■niejszy leak do usuni■cia:

```text
ExecutionEngine._record_execution()
  -&gt; MemoryStore.append_evidence("action_result", payload)
```

2. Repozytorium robocze

Repozytorium:

- `/home/l/Desktop/AOIA-Core`

Remote:

- `https://github.com/luciferprosun/AOIA-Core.git`

Status po pracy:

```text
main...origin/main
 M runtime/tools/executor.py
?? docs/forensic-runtime-audit/
?? docs/refactor/
?? docs/reports/
?? state/
?? tests/test_executor_containment.py
```

3. Pliki zmienione

Zmodyfikowany plik runtime:

- `/home/l/Desktop/AOIA-Core/runtime/tools/executor.py`

Dodany test:

- `/home/l/Desktop/AOIA-Core/tests/test_executor_containment.py`

Nie zmieniono:

- `runtime/tools/memory.py`
- `runtime/main.py`
- `runtime/adaptive_routing/epistemic_kernel.py`
- `runtime/orchestrator/knowledge_router.py`
- providerów
- routingu
- governance
- registry provenance
- registry contradictions
- vault design

4. Zlokalizowany toksyczny przep■yw

W `runtime/tools/executor.py` funkcja:

- `ExecutionEngine._record_execution()`

przed Phase 2A wykonywa■a:

```python
self.memory_store.record_result(result)
self.memory_store.append_history("action_result", payload)
self.memory_store.append_evidence("action_result", payload)
```

Problem:

- `action_result` jest L1 operational event
- by■ promowany do evidence-like storage
- to ■ama■o Phase 1A/1C doctrine
- command/tool/runtime outputs nie mog■ by■ canonical evidence

5. Zmiana wykonana

Usuni■to wy■■cznie lini■:

```python
self.memory_store.append_evidence("action_result", payload)
```

Zachowano:

```python
self.memory_store.record_result(result)
self.memory_store.append_history("action_result", payload)
```

Dodano boundary comment:

```python
AOIA Phase 2A containment boundary
Runtime operational outputs must NEVER become canonical evidence.
```

6. Quarantine labeling

Do payload `action_result` dodano jawny blok:

```python
"authority": {
    "classification": "operational_event",
    "retention": "replay_only",
    "non_authoritative": True,
    "canonical_evidence": False,
}
```

Znaczenie:

- `classification: operational_event` - to jest zdarzenie operacyjne, nie evidence
- `retention: replay_only` - zapis istnieje dla replay/debug/history

- `non_authoritative: True` - nie jest authority
- `canonical_evidence: False` - nie mo■e by■ traktowane jako L4

7. Co zosta■o zachowane

Phase 2A zachowa■a:

- command replay logs
- `append_history()`
- runtime continuity
- `recent_outputs`
- command debugging
- operator visibility
- browser event path
- session continuity
- execution behavior
- approval behavior

Nie zmieniono:

- tego jak akcje s■ wykonywane
- tego jak dzia■a approval prompt
- tego jak dzia■a `record_result()`
- tego jak dzia■a `append_history()`
- tego jak dzia■a browser event logging

8. Co zosta■o zablokowane

Od Phase 2A:

- `action_result` nie trafia ju■ do `memory/evidence_memory.jsonl`
- runtime operational output nie jest ju■ automatycznie evidence
- command output nie jest automatycznie evidence
- tool output nie jest automatycznie evidence
- rejected/approved runtime events nie s■ automatycznie evidence

Wa■ne:

- `append_evidence()` nadal istnieje
- kernel evidence flow nadal istnieje
- Phase 2A nie przebudowuje L4
- Phase 2A tylko zatrzymuje najgorszy pseudo-evidence leak

9. Test dodany

Dodany test:

- `/home/l/Desktop/AOIA-Core/tests/test_executor_containment.py`

Test sprawdza:

- akcja `create_folder` nadal dzia■a
- `recent_outputs` nadal dzia■a
- command replay log nadal powstaje
- `memory.evidence_file` nie powstaje dla `action_result`
- `history.jsonl` nadal ma `action_result`
- payload ma quarantine labels
- `canonical_evidence` jest `False`

10. Weryfikacja

Uruchomiony focused test:

```text
PYTHONPATH=runtime python3 -m unittest tests.test_executor_containment
```

Wynik:

```text
.
----------------------------------------------------------------------
Ran 1 test in 0.047s

OK
```

11. Ograniczenie testów

Pe■ny test module:

```text
python3 -m unittest tests.test_main
```

nie uruchamia si■ obecnie przez istniej■cy, niezwi■zany problem:

```text
ModuleNotFoundError: No module named 'memory.rhcsa_context'
```

Ten problem istnia■ poza Phase 2A i nie by■ naprawiany, bo zakres Phase 2A zabrania■ szerokich zmian runtime/importów.

Dlatego dodano osobny, focused regression test, który importuje tylko:

- `tools.executor.ExecutionEngine`
- `tools.memory.MemoryStore`

i nie wymaga importu `runtime/main.py`.

12. Ryzyko rollbacku

Rollback jest prosty.

Do cofni■cia Phase 2A wystarczy:

- przywróci■ jedn■ lini■ w `runtime/tools/executor.py`:

```python
self.memory_store.append_evidence("action_result", payload)
```

- usun■■ test:

```text
tests/test_executor_containment.py
```

Nie ma migracji danych.
Nie ma zmian schematu.
Nie ma zmian routing/provider/governance.
Nie ma zmian w `memory.py`.

13. Ocena ryzyka zmiany

Ryzyko niskie, bo:

- zmiana dotyczy jednej funkcji
- zachowuje execution path
- zachowuje history path
- zachowuje command logs

- zachowuje recent outputs
- nie zmienia `MemoryStore`
- nie zmienia retrieval
- nie zmienia providerów
- test focused przechodzi

Ryzyko pozosta■e:

- downstream kod, je■li kiedy■ oczekiwa■ `action_result` w `evidence_memory.jsonl`, przestanie go tam widzie■
- ale wed■ug Phase 1A/1C taki downstream behavior by■by niekanoniczny

14. Status doctrinalny po Phase 2A

Naprawione:

- L1 `action_result` nie jest ju■ automatycznie pseudo-L4 evidence

Nadal niezaimplementowane:

- CAS evidence store
- strict L4 evidence schema
- append-only provenance log
- append-only contradiction events
- retrieval guard
- L2 physical quarantine
- Vault projection-only enforcement
- cleanup legacy mixed `memory/evidence_memory.jsonl`

15. Nast■pny bezpieczny krok

Najbezpieczniejszy kolejny krok po Phase 2A:

- walidacja git
- commit checkpoint
- potem Phase 2B jako osobna decyzja

Nie nale■y od razu robi■:

- splitu `memory.py`
- adapterów
- retrieval guard
- governance runtime
- provider/routing changes

16. Wniosek ko■cowy

Phase 2A wykona■a minimalny containment:

````
```text
action_result
  -> history/replay/debug
  -> NOT evidence
```
````

Najwa■niejsze zachowane w■a■ciwo■ci:

- runtime nadal dzia■a
- replay nadal dzia■a
- history nadal dzia■a
- continuity nadal dzia■a
- pseudo-evidence promotion zosta■a zatrzymana

Najkrótszy opis:

- `action_result` jest teraz jawnie `operational_event`, `replay_only`, `non_authoritative`, `canonical_evidence: False`
- nie trafia ju■ do `append_evidence()`

12. DETERMINISM_AUDIT.md

Determinism Audit

Scope

This audit validates whether AOIA currently behaves according to the architecture principles it claims:

- local-first
- deterministic routing
- provenance-aware retrieval
- contradiction-aware output
- evidence vs reasoning separation
- epistemic restraint

This is not an implementation phase. It is a verification phase.

Overall Conclusion

AOIA is only partially deterministic at the system level.

What is actually deterministic:

- RHCSA retrieval algorithms
- provenance and contradiction registry generation
- AOIA epistemic kernel scoring and routing
- local shortcut routes
- validator normalization

What is not deterministic end-to-end:

- the planner path after local routing falls through to cloud models
- provider fallback order depends on available keys and backend reachability
- prompt contents include mutable runtime memory, local notes, and recent outputs
- local response routing can be influenced by mutable memory hats

Therefore:

- deterministic retrieval: real
- deterministic full runtime: not real
- deterministic AOIA execution claim: only partially true

System-by-System Validation

1. RHCSA Knowledge Base

Implementation:

- content lives in `knowledge/`
- topic modules are markdown plus JSON examples
- source provenance is embedded in frontmatter such as `source_pdf` and `generated_from`
- retrieval indexes are built from local files only

Limitations:

- knowledge is static and only as good as the imported RHCSA corpus
- the runtime uses the local library, but there is no freshness validation against source artifacts at runtime
- duplicate commands exist across examples and markdown modules

Failure modes:

- stale or partially regenerated knowledge can still be treated as authoritative
- a changed `knowledge/` tree does not automatically force registry regeneration
- duplicate command entries may inflate evidence counts

Classification:

- `FUNCTIONAL`

Reason:

- the knowledge base is real, structured, and locally queryable, but consistency and freshness are not continuously enforced.

2. Obsidian Memory Layer

Implementation:

- runtime-managed vault: `obsidian_vault/`
- user-imported vault: `AI_MEMORY_VAULT/`
- `tools/memory.py` writes to `obsidian_vault/`, not `AI_MEMORY_VAULT/`
- evidence and reasoning notes are separated into `obsidian_vault/Evidence/` and `obsidian_vault/Reasoning/`

Limitations:

- the named imported memory layer `AI_MEMORY_VAULT/` is not connected to runtime execution
- daily notes and session notes still aggregate mixed runtime events
- there is no pruning, summarization cap, or aging policy in the active runtime vault

Failure modes:

- operator may assume `AI_MEMORY_VAULT/` is active runtime memory when it is not
- runtime memory can grow indefinitely inside the repo
- mixed note streams can blur evidence, execution, and operational logs

Classification:

- `PARTIAL`

Reason:

- an Obsidian-compatible memory system exists and is active, but the specifically requested vault is not the one the runtime actually uses.

3. Deterministic Retrieval

Implementation:

- `tools/rhcsa_search.py`
- exact command lookup
- exact tag lookup
- keyword scoring by normalized token and substring rules
- literal grep-style retrieval
- topic filtering
- local markdown and JSON only

Limitations:

- keyword scoring is deterministic but still heuristic
- retrieval quality depends on token overlap rather than semantic understanding
- no provenance weighting in the retrieval algorithm itself

Failure modes:

- false negatives on phrasing mismatch
- false positives from broad token overlap
- top hits can be technically deterministic while still semantically weak

Classification:

- `STABLE`

Reason:

- this subsystem is genuinely deterministic for the same input and same file set.

4. Provenance Registry

Implementation:

- `tools/epistemic_registry.py`
- scans local `knowledge/` artifacts
- records artifact path, type, metadata, references, command count, and content hash
- current registry contains `artifact_count: 62`

Limitations:

- registry is snapshot-based
- runtime reads existing JSON and only rebuilds on missing or invalid files
- there is no automatic stale-check against live file mtimes or hashes at startup

Failure modes:

- provenance registry can drift behind actual `knowledge/` content
- runtime may attach outdated provenance to fresh files if registries are not regenerated

Classification:

- `FUNCTIONAL`

Reason:

- the registry is real and useful, but not self-validating for freshness.

5. Contradiction Registry

Implementation:

- generated from the same artifact scan as provenance
- detects:
- self references
- circular references
- duplicate commands
- duplicate artifacts by hash
- current summary:
- `self_reference_count: 0`
- `circular_reference_count: 0`
- `duplicate_command_count: 3`
- `duplicate_artifact_count: 0`

Limitations:

- this is not semantic contradiction detection
- it detects structural conflicts and duplicate command sources, not truth conflicts
- no runtime freshness validation

Failure modes:

- real factual contradictions can pass unnoticed if wording differs
- duplicate command detection may over-signal conflict where duplication is intentional

Classification:

- `FUNCTIONAL`

Reason:

- it does what it claims structurally, but its epistemic coverage is narrower than the name may imply.

6. Evidence vs Reasoning Separation

Implementation:

- `tools/memory.py` maintains:
- `memory/evidence_memory.jsonl`
- `memory/reasoning_trace.jsonl`
- corresponding vault folders:
- `obsidian_vault/Evidence/`
- `obsidian_vault/Reasoning/`
- `main.py` writes reasoning through `log_reasoning_trace()`
- `main.py` and `tools/executor.py` write evidence through `append_evidence()`

Limitations:

- evidence storage also contains execution results via `tools/executor.py`
- daily notes still aggregate general history
- evidence is not limited to source evidence; it includes operational traces

Failure modes:

- evidence channel becomes a mixed store of provenance, action results, and kernel evidence
- strict epistemic separation is weakened by append-history side channels

Classification:

- `PARTIAL`

Reason:

- physical separation exists, but semantic separation is incomplete.

7. Epistemic Safeguards

Implementation:

- env-controlled safeguards in `main.py`
- supports:
- kill switch
- disable model
- disable knowledge route
- disable memory hats
- disable reasoning trace
- prefer `I DO NOT KNOW`
- `tools/validator.py` normalizes confidence labels
- `main.py` prints confidence labels and epistemic notes

Limitations:

- safeguards primarily govern planner behavior, not all local memory growth
- they do not prevent cloud fallback, only disable parts of routing when configured
- they rely on operator env discipline

Failure modes:

- without environment switches, cloud planning remains active
- invalid model JSON still enters the loop before fallback to unknown
- operator may assume “epistemic safe” means offline-safe or deterministic-safe; it does not

Classification:

- `FUNCTIONAL`

Reason:

- the controls are real and wired into runtime behavior, but they do not eliminate nondeterminism outside the local path.

8. Manual Review Hooks

Implementation:

- `adaptive_routing/epistemic_kernel.py` sets `manual_review_required`
- `main.py` prints `MANUAL REVIEW: REQUIRED`
- duplicate or conflicting sources trigger review flags

Limitations:

- review is a flag, not a workflow engine
- no separate review queue, state machine, or persistence model exists for unresolved review items
- manual review is not enforced as a blocking gate on all output types

Failure modes:

- the system can still return a local answer while merely labeling it for review
- review reasons may be broad, because duplicate-source detection is structural

Classification:

- `FUNCTIONAL`

Reason:

- hooks are present and visible, but they are minimal.

Explicit Checks

Semantic guessing

Result:

- local retrieval path does not use embeddings or semantic expansion
- planner/model path absolutely can guess, because it is an LLM path

Conclusion:

- semantic guessing is absent in local retrieval
- semantic guessing remains possible in overall runtime behavior

Pseudo-determinism

Result:

- the repo uses deterministic branding beyond what the full runtime guarantees

Why:

- local routing is deterministic
- planner execution is not
- provider selection depends on external availability
- prompt state depends on mutable logs and memory

Conclusion:

- the system is deterministically pre-routed, not deterministically end-to-end

Memory contamination

Result:

- present

Why:

- evidence storage contains both source evidence and action-result evidence
- daily notes aggregate mixed event types
- `AI_MEMORY_VAULT/` and `obsidian_vault/` create conceptual split-memory ambiguity

Uncontrolled context growth

Result:

- present on disk
- partially controlled in prompt assembly

Why:

- prompt payload slices arrays and truncates text
- logs, vault notes, and JSONL traces grow without lifecycle controls

Recursive reasoning loops

Result:

- bounded but present

Why:

- `main.py` runs planner and reactive loops with up to `MAX_AGENT_STEPS = 8`
- output of prior steps re-enters future prompt context

Hidden cloud dependency

Result:

- present

Why:

- canonical provider manager routes to OpenRouter, Gemini, DeepSeek, and optional Aureon
- prompt file explicitly states cloud provider connectivity

Fake autonomous behavior

Result:

- present at the documentation perimeter more than in active code

Why:

- repo contains orchestrator abstractions and AOIA research layers
- active worker autonomy is disabled
- actual runtime remains operator-gated

Final Classification Summary

System	Classification
RHCSA knowledge base	`FUNCTIONAL`
Obsidian memory layer	`PARTIAL`
Deterministic retrieval	`STABLE`
Provenance registry	`FUNCTIONAL`
Contradiction registry	`FUNCTIONAL`
Evidence vs reasoning separation	`PARTIAL`
Epistemic safeguards	`FUNCTIONAL`
Manual review hooks	`FUNCTIONAL`

Bottom Line

If AOIA claims “deterministic runtime,” that claim is too strong.

Accurate claim:

- deterministic local retrieval and epistemic pre-routing
- bounded operator-supervised runtime
- cloud-dependent non-deterministic planner fallback

That is the architecture actually implemented today.

13. EPISTEMIC_RISK_REPORT.md

Epistemic Risk Report

Executive Risk Statement

AOIA’s main epistemic strength is local deterministic retrieval before model use.

AOIA’s main epistemic weakness is that once local routing fails, the system drops into a cloud LLM planner loop whose outputs are not deterministic and not provenance-bound.

This creates a hybrid architecture:

- deterministic local evidence path
- non-deterministic cloud planning path

That hybrid is workable, but it should not be misdescribed as fully deterministic.

Primary Risks

Risk 1: Deterministic branding exceeds actual runtime determinism

Severity:

- High

Observed pattern:

- `tools/rhcsa_search.py` is deterministic
- `adaptive_routing/epistemic_kernel.py` is deterministic
- `main.py` falls through to `ProviderManager.generate()` when local resolution fails

Why this matters:

- users may trust the system as if all outputs are reproducible
- only the local path is reproducible

Failure mode:

- same prompt, different backend state or provider response, different plan

Risk 2: Hidden cloud dependency contradicts strict local-first expectations

Severity:

- High

Observed pattern:

- providers are OpenRouter, Gemini, DeepSeek, optional Aureon
- `prompts/system_prompt.txt` explicitly says cloud-model connectivity

Why this matters:

- local-first is true only for routing preference, not for all reasoning
- cloud outage or key loss changes system behavior fundamentally

Failure mode:

- local route miss becomes total failure when cloud providers are unavailable

Risk 3: Evidence channel contamination

Severity:

- Medium

Observed pattern:

- `tools/executor.py` writes all `action_result` payloads into evidence memory
- `main.py` also writes AOIA kernel evidence there

Why this matters:

- execution traces and evidentiary traces are not cleanly separated
- later forensic reading may over-trust operational logs as epistemic evidence

Failure mode:

- evidence store becomes a mixed operational ledger rather than a strict evidence ledger

Risk 4: Registry freshness is assumed, not proven

Severity:

- Medium

Observed pattern:

- `adaptive_routing/epistemic_kernel.py` loads `provenance_registry.json` and `contradiction_registry.json`
- rebuild occurs only if file read fails

Why this matters:

- registries can become stale while appearing valid

Failure mode:

- evidence is attached with outdated provenance or contradiction metadata

Risk 5: Manual review is advisory, not enforced

Severity:

- Medium

Observed pattern:

- manual review appears as output flags
- no hard review workflow exists

Why this matters:

- contradictions are surfaced, but not quarantined
- the system still answers even when review is required

Failure mode:

- operator overlooks review label and treats answer as cleared

Risk 6: Memory-layer ambiguity

Severity:

- Medium

Observed pattern:

- active runtime vault is `obsidian_vault/`
- user-facing imported memory layer is `AI_MEMORY_VAULT/`

Why this matters:

- operators may update the wrong vault
- future developers may wire new logic to the wrong persistence layer

Failure mode:

- “persistent memory” expectations diverge from actual runtime behavior

Risk 7: Recursive prompt-state loop

Severity:

- Medium

Observed pattern:

- prior outputs, RHCSA context, hats, and runtime state re-enter prompt generation
- planner loop and reactive loop are both bounded, but still recursive

Why this matters:

- bounded recursion still accumulates context bias
- subtle drift can emerge from repeated self-conditioning

Failure mode:

- model plans increasingly reflect previous model outputs rather than source evidence

Risk 8: Fake autonomy perception

Severity:

- Low to Medium

Observed pattern:

- repo contains orchestrator and worker abstractions
- active worker path is disabled
- docs and structure imply richer autonomy than active runtime executes

Why this matters:

- architectural misunderstanding can lead to unsafe future extensions

Failure mode:

- dormant experimental modules are mistaken for production-ready agent logic

Risk Matrix

Risk	Severity	Actuality
Pseudo-deterministic system framing	High	Present now

Cloud dependency under local-first narrative	High	Present now
Evidence contamination	Medium	Present now
Registry staleness	Medium	Present now
Advisory-only manual review	Medium	Present now
Dual memory-layer ambiguity	Medium	Present now
Recursive prompt-state loops	Medium	Present now
Fake autonomy perception	Low/Medium	Present now

What Is Truly Epistemically Strong

- retrieval is rule-based and local
- provenance metadata exists
- contradiction reporting exists
- `I DO NOT KNOW` fallback exists
- operator approval boundary blocks tool execution autonomy

What Only Looks Stronger Than It Is

- “deterministic” as a whole-system label
- “contradiction registry” as if it detects truth-level contradictions
- “Obsidian memory layer” as if the imported vault is the active runtime vault
- “orchestrator architecture” as if it is production active

Safe Interpretation

Correct interpretation of the current system:

- deterministic evidence lookup layer
- bounded supervised tool runtime
- cloud-assisted planning fallback
- partial epistemic separation

Incorrect interpretation:

- fully deterministic autonomous epistemic kernel
- cloud-free reasoning engine
- fully clean evidence/reasoning separation
- active autonomous multi-agent architecture

Recommended Phase-2 Judgment

AOIA is epistemically improved relative to a plain tool-using LLM runtime, but it is not yet epistemically strict enough to justify stronger claims.

Most honest label:

- `deterministic local retrieval with epistemic safeguards, plus non-deterministic cloud planning fallback`

14. FORENSIC_ARCHITECTURE_REPORT.md

Forensic Architecture Report

Scope

This audit is read-only with respect to application code. The only new artifacts created are audit documents.

Audit target: `/home/l/Desktop/app2terminl_opened`

Observed repository shape:

- Active Python runtime
- Deterministic RHCSA knowledge base
- AOIA research and routing scaffolding
- Web wrapper over the same runtime
- Imported sibling frontend app under `apps/`
- Large amount of generated local state, logs, exports, and historical docs

Executive Summary

The repository has one real execution spine and several preserved side paths.

Canonical execution spine:

1. `main.py`
2. `commands/`
3. `router/local_router.py`
4. `orchestrator/knowledge_router.py`
5. `adaptive_routing/epistemic_kernel.py`
6. `providers/config.py`
7. `tools/executor.py`
8. `tools/{validator,memory,shell_tools,filesystem_tools,browser_tools}.py`

This spine is coherent enough to continue development safely, but the repo around it is noisy. The largest stability risks are not algorithmic complexity. They are repository ambiguity, dormant experimental paths, and generated runtime state living next to source.

Canonical Runtime

The runtime is local-first but not fully offline:

- `main.py` is the primary CLI entrypoint.
- `webapp.py` is a thin HTTP wrapper around the same `AgentRuntime`.
- `run.sh` and `run_web.sh` are launcher wrappers only.
- `commands/local_commands.py` provides local slash commands.
- `router/local_router.py` handles deterministic obvious tasks before model use.
- `orchestrator/knowledge_router.py` and `adaptive_routing/epistemic_kernel.py` provide local deterministic RHCSA and epistemic routing.
- `tools/executor.py` is the only real action dispatcher.

This is the strongest architectural fact in the repo: execution is centralized, not spread across many executors.

Stable Subsystems

Runtime core

- `main.py`
- `webapp.py`
- `tools/executor.py`

- `tools/validator.py`
- `tools/memory.py`
- `providers/config.py`

Deterministic knowledge subsystem

- `knowledge/`
- `tools/rhcsa_search.py`
- `knowledge/rhcsa_engine.py`
- `tools/epistemic_registry.py`
- `provenance_registry.json`
- `contradiction_registry.json`

Determinism and safety layer

- `adaptive_routing/deterministic_router.py`
- `adaptive_routing/config_loader.py`
- `adaptive_routing/epistemic_kernel.py`
- `tests/test_aoia_determinism.py`
- `tests/test_epistemic_*`

Duplicate Modules And Structural Drift

Documentation duplication

There are parallel document sets with overlapping authority:

- root `ARCHITECTURE.md` and `docs/ARCHITECTURE.md`
- root `CONSTRAINTS.md` and `docs/CONSTRAINTS.md`
- root `PROJECT_STRUCTURE.md` and `docs/REPO_STRUCTURE.md`
- `docs/ADR/` and `docs/adr/`

This is the largest non-code clarity problem. It creates competing narratives for the same system.

Memory vault duplication

Two vault-like structures exist:

- `AI_MEMORY_VAULT/`
- `obsidian_vault/`

They are not equivalent. `obsidian_vault/` is the runtime-managed active vault. `AI_MEMORY_VAULT/` is a manually imported memory layer. That distinction is not obvious from the top level.

Validator namespace duplication

Two validators exist, but for different domains:

- `tools/validator.py` for runtime action validation
- `knowledge/validator/validator.py` for RHCSA example pack validation

This is not a bug, but the identical name increases confusion.

Placeholder, Legacy, Or Experimental Systems

Disabled orchestrator path

`orchestrator/gemini_gemma.py` exists, but the worker path is intentionally disabled:

- `GeminiGemmaOrchestrator.gemma_provider` starts as `None`
- `action_for_step()` raises if the worker is not present

- `/orchestrator on` still returns a disabled notice

This is preserved experimental architecture, not active production flow.

Legacy Gemma provider

`providers/gemma_provider.py` still exists and contains Ollama/HuggingFace/OpenAI-compatible fallback logic, but `providers/config.py` no longer routes to it. It is effectively orphaned from the canonical runtime.

AOIA research stubs

These modules are real code but not part of the canonical path:

- `adaptive_routing/circadian_router.py`
- `adaptive_routing/environment/environment_router.py`
- `adaptive_routing/stdout_logger.py`

They are referenced by docs and tests, not by `main.py`.

Web reader side utility

`tools/web_reader.py` appears unused by the runtime and contains signs of drift:

- imports `requests` and `bs4`, which are not declared in `requirements.txt`
- uses `if name == "main"` instead of `if __name__ == "__main__"`

This is a strong dead-or-abandoned utility candidate.

Dead Code And Weakly Connected Code

Likely dead or weakly connected modules:

- `providers/gemma_provider.py`
- `tools/web_reader.py`
- `adaptive_routing/circadian_router.py`
- `adaptive_routing/environment/environment_router.py`
- `adaptive_routing/stdout_logger.py`
- top-level `validator.py` wrapper

These are not necessarily safe to delete in this phase, but they are not on the canonical runtime path.

Hidden Dependencies

Python dependencies

Declared in `requirements.txt`:

- `google-genai`
- `playwright`

Undeclared but imported:

- `requests` in `tools/web_reader.py`
- `bs4` in `tools/web_reader.py`

External runtime services

The app is not cloud-free in practice:

- OpenRouter
- Gemini API

- DeepSeek API
- optional Aureon-compatible backend

The runtime is local-first, but model generation depends on external providers unless local deterministic routes handle the request first.

Frontend subproject dependencies

`apps/flameborn-academy-codex-sparrow/` introduces a separate Node/Workers toolchain:

- `package.json`
- `wrangler.jsonc`
- Cloudflare worker scripts

This is a second ecosystem inside the repo and not integrated into the Python runtime.

Recursive Risks

Prompt-feedback recursion

`main.py` feeds runtime state, recent outputs, knowledge context, and reasoning traces back into future prompts. This is bounded, but still a recursive prompt-state loop.

Current mitigations:

- step limit in `main.py`
- deterministic local routing before model use
- explicit `I DO NOT KNOW` path
- approval boundary in `tools/executor.py`

Memory growth in-repo

Runtime state is written into source-adjacent directories:

- `memory/`
- `logs/`
- `state/`
- `obsidian_vault/`

This does not create autonomous mutation of code, but it does create repository self-accumulation and noise over time.

Autonomous Behavior Risks

The repo does not currently implement autonomous agents in the strong sense, but it contains agent-shaped scaffolding.

Present risks:

- plan -> action -> result -> replan loop in `main.py`
- browser persistence in `tools/browser_tools.py`
- multi-step model loop with shell and filesystem execution
- preserved orchestrator abstraction in `orchestrator/`

Current safety boundaries:

- manual approval for non-`respond` actions
- shell validator
- bounded loop count
- no automatic contradiction resolution
- no active self-modifying code path detected

Fake Agent Logic Risks

The largest fake-agent risk is narrative, not execution:

- AOIA research files describe richer architecture than the active runtime actually executes.
- disabled orchestrator structures may be mistaken for active autonomy.
- multiple architecture docs create a stronger impression of system maturity than the code path supports.

In code, the system is much simpler than the documentation perimeter implies.

Working Code Preservation Assessment

The core runtime is intact and should be preserved as the baseline:

- local router
- deterministic RHCSA retrieval
- epistemic kernel
- executor and validator
- CLI and web wrappers

Further development should continue from this spine, not from archived AOIA research fragments or disabled worker abstractions.

Safe Continuation Guidance

Before further development, the repo needs architectural authority clarified:

1. One canonical runtime map.
2. One canonical constraints document.
3. One canonical ADR location.
4. Explicit classification of each directory as `active`, `generated`, `archive`, or `experimental`.
5. Separation of runtime artifacts from source where possible.

This audit does not recommend code deletion in Phase 1. It does recommend treating the canonical runtime path as the only safe base for further AOIA work.

15. MEMORY_FLOW_ANALYSIS.md

Memory Flow Analysis

Overview

AOIA currently has two different memory concepts:

1. runtime-managed memory under `obsidian_vault/` and `memory/`
2. imported persistent markdown vault under `AI_MEMORY_VAULT/`

Only the first one is active in execution.

Actual Runtime Memory Flow

````
```text
User request
  -> main.py
  -> MemoryStore.set_current_task()
  -> local route / knowledge route / planner path
  -> executor or local response
  -> logs and memory appenders
  -> state/, memory/, obsidian_vault/
```
````

Main Runtime Memory Components

In-memory request state

Defined in `tools/memory.py`:

- `session_id`
- `cwd`
- `current_task`
- `previous_commands`
- `recent_outputs`
- `open_tabs`
- `current_browser_page`
- `screenshots`
- `browser_active`

This state is lightweight and bounded in several lists, which is good for RAM discipline.

Disk-persisted state

Written to:

- `state/agent_state.json`
- `memory/history.jsonl`
- `memory/evidence_memory.jsonl`
- `memory/reasoning_trace.jsonl`
- `logs/commands/`
- `logs/errors/`
- `logs/sessions/`
- `obsidian_vault/`

Evidence Flow

Evidence enters the system from two main sources.

Source evidence

Produced by:

- `adaptive_routing/epistemic_kernel.py`
- local RHCSA retrieval results

Written through:

- `main.py -> memory_store.append_evidence("aoia_kernel_evidence", ...)`

Payload includes:

- query
- route
- confidence
- manual review requirement
- artifact paths

Execution evidence

Produced by:

- every executed action in `tools/executor.py`

Written through:

- `memory_store.append_evidence("action_result", payload)`

Payload includes:

- full action
- full result
- cwd
- timestamp

Reasoning Flow

Reasoning enters through `main.py.log_reasoning_trace()` and is written only if safeguards allow it.

Sources:

- model request logging
- planner request logging
- planner action logging
- unknown-response logging
- knowledge-route-disabled logging
- AOIA kernel decision summaries

Written to:

- `memory/reasoning_trace.jsonl`
- `obsidian_vault/Reasoning/<session>.md`

Obsidian Vault Flow

The active runtime vault is `obsidian_vault/`.

Generated folders:

- `Daily/`
- `Sessions/`
- `Evidence/`
- `Reasoning/`
- plus human-facing scaffold folders

Important property:

- `append_history()` also writes to daily notes and session logs
- this means a general runtime narrative exists outside the stricter evidence/reasoning split

AI_MEMORY_VAULT Flow

`AI_MEMORY_VAULT/` does not participate in active runtime flow.

Observed behavior:

- no active imports from runtime code
- no active writes from `tools/memory.py`
- no active reads in `main.py`

Conclusion:

- it is a persistent markdown vault artifact
- it is not the canonical runtime memory layer

Classification:

- `STUB` as runtime memory
- `FUNCTIONAL` as passive markdown vault content

Memory Contamination Analysis

Evidence contamination

Present.

Why:

- source evidence and execution-result evidence share the same evidence channel

Effect:

- evidence memory is not a pure source-trace ledger

Reasoning contamination

Lower, but still possible conceptually.

Why:

- reasoning trace file is separate
- however, daily notes still mix general runtime events

Effect:

- forensic reconstruction is possible, but not perfectly clean

Cross-vault contamination

Present conceptually.

Why:

- two Obsidian-style vaults coexist
- only one is live

Effect:

- operators may read or maintain the wrong memory surface

Context Growth Analysis

In-RAM context growth

Controlled.

Why:

- `previous_commands` truncated to 20
- `recent_outputs` truncated to 20
- prompt slices smaller windows from those lists
- large outputs are truncated by `summarize_text()`

On-disk growth

Uncontrolled.

Why:

- JSONL histories append indefinitely
- session vault files append indefinitely
- daily note files append indefinitely
- browser profile and logs accumulate inside repo

Effect:

- RAM discipline is acceptable
- repository-state discipline is weak

Recursive Memory Loop Analysis

There is a bounded recursive loop:

1. runtime produces outputs
2. outputs are stored
3. slices of stored state enter future prompts
4. future model actions are influenced by prior stored outputs

What prevents it from becoming unbounded in one request:

- `MAX_AGENT_STEPS = 8`
- prompt truncation
- confidence labels
- `I DO NOT KNOW` fallback

What does not prevent long-term drift:

- persistent disk accumulation across sessions
- mutable memory hats
- repeated session reuse of local state artifacts

Per-System Memory Judgment

RHCSA knowledge memory

- implementation: static local files and retrieval indexes
- limitation: snapshot freshness not enforced
- failure mode: stale knowledge treated as current
- classification: `FUNCTIONAL`

Runtime Obsidian vault

- implementation: active `obsidian_vault/` writes from `tools/memory.py`
- limitation: mixed daily-note aggregation
- failure mode: memory bloat and mixed semantics
- classification: `FUNCTIONAL`

AI_MEMORY_VAULT

- implementation: markdown vault scaffold only
- limitation: disconnected from runtime
- failure mode: false operator assumptions about active memory
- classification: `STUB`

Evidence vs reasoning split

- implementation: separate files and vault folders
- limitation: evidence channel contains action results
- failure mode: evidence ledger contamination
- classification: `PARTIAL`

Bottom Line

AOIA has real memory persistence, but not a single clean epistemic memory model.

Accurate statement:

- runtime memory is active in `obsidian_vault/` and `memory/`
- imported memory is passive in `AI_MEMORY_VAULT/`
- evidence and reasoning are physically separated
- semantics of those channels are only partially separated

If stronger epistemic claims are desired later, the first memory problem to solve is not storage existence. It is memory meaning.

16. MODULE_DEPENDENCY_MAP.md

Module Dependency Map

Canonical Python Dependency Spine

```text
main.py
├── commands
│   └── commands/local_commands.py
├── adaptive_routing
│   └── adaptive_routing/epistemic_kernel.py
├── memory
│   ├── memory/rhcsa_context.py
│   └── memory/gemma_worker_memory.py
├── orchestrator
│   ├── orchestrator/knowledge_router.py
│   └── orchestrator/gemini_gemma.py
├── providers
│   └── providers/config.py
├── router
│   └── router/local_router.py
└── tools
    ├── tools/executor.py
    ├── tools/memory.py
    ├── tools/memory_hats.py
    ├── tools/system_info.py
    └── tools/validator.py
```

Detailed Map

Entrypoints

- `main.py`
- canonical CLI runtime
- `webapp.py`
- imports `AgentRuntime` from `main.py`
- adds HTTP API and static UI serving
- `validator.py`
- thin wrapper around `knowledge.validator.validator.main`

Commands layer

- `commands/local_commands.py`
- depends on `tools.rhcsa_search`
- depends on `tools.validator`
- invokes runtime methods and executor

Routing layer

- `router/local_router.py`
- no heavy dependencies
- deterministic pre-LLM route shortcuts

- `orchestrator/knowledge_router.py`
- depends on `knowledge.rhcsa_engine`
- writes token savings report

- `adaptive_routing/epistemic_kernel.py`
- depends on `adaptive_routing.deterministic_router`
- depends on `tools.epistemic_registry`
- depends on `tools.rhcsa_search`

Execution layer

- `tools/executor.py`
- central execution dispatcher
- depends on:
- `tools.browser_tools`
- `tools.filesystem_tools`
- `tools.project_scanner`
- `tools.shell_tools`
- `tools.validator`
- `tools.memory`

Memory layer

- `tools/memory.py`
- persists runtime state
- writes to `state/`, `memory/`, and `obsidian_vault/`

- `memory/gemma_worker_memory.py`
- separate preserved worker memory path

- `tools.memory_hats.py`
- lightweight behavioral context overlays

Knowledge layer

- `tools/rhcsa_search.py`
- deterministic lookup over `knowledge/`

- `knowledge/rhcsa_engine.py`
- higher-level aggregation over `tools.rhcsa_search`

- `memory/rhcsa_context.py`
- injects RHCSA context into planner prompts

- `tools/epistemic_registry.py`
- computes provenance and contradiction registries from `knowledge/`

Providers layer

- `providers/config.py`
- canonical provider manager
- depends on:
- `providers.aureon_provider`
- `providers.gemini_provider`
- `providers.openai_compatible`

- `providers/gemma_provider.py`
- not reached from canonical provider build path

Runtime-Adjoining But Non-Canonical Modules

Preserved experimental AOIA modules

- `adaptive_routing/circadian_router.py`
- `adaptive_routing/environment/environment_router.py`
- `adaptive_routing/stdout_logger.py`

Used by:

- tests
- docs
- research narrative

Not used by:

- `main.py`
- `webapp.py`
- `tools/executor.py`

Preserved disabled orchestrator modules

- `orchestrator/gemini_gemma.py`
- `providers/gemma_provider.py`
- `memory/gemma_worker_memory.py`

These form a partial subsystem, but only its memory and planner shell remain connected. The worker execution path is intentionally disabled.

Hidden Dependency Map

Declared dependencies

- `google-genai`
- used only by `providers/gemini_provider.py`
- `playwright`
- used by `tools/browser_tools.py`

Undeclared dependencies

- `requests`
- imported by `tools/web_reader.py`
- `bs4`
- imported by `tools/web_reader.py`

External services

- OpenRouter API
- Gemini API
- DeepSeek API
- optional Aureon-compatible backend

Embedded subproject toolchain

`apps/flameborn-academy-codex-sparrow/` depends on a separate JS ecosystem:

- Node/npm
- Cloudflare Workers tooling
- Wrangler

Dependency Findings

Strong centralization

Execution is well centralized:

- `main.py` coordinates
- `tools/executor.py` executes
- `tools/validator.py` validates

This is a positive architectural property.

Research/runtime split is incomplete

AOIA research modules live next to production runtime modules with similar naming. This increases the chance of accidental future coupling.

Generated state is source-adjacent

The codebase depends on paths that are also used as mutable runtime stores:

- `state/`
- `memory/`
- `obsidian_vault/`

This is manageable, but it blurs source versus state.

Multiple authority centers

There is no single architectural authority file. The same concepts are described in root docs, `docs/`, and reports.

17. NEXT_STEPS_RECOMMENDATION.md

Next Steps Recommendation

Objective

Determine the safest next engineering step for AOIA without expanding features.

Current Constraint Summary

AOIA is strongest as:

- deterministic local retrieval
- epistemic pre-routing
- supervised tool execution

AOIA is weakest as:

- repository-state discipline
- memory-surface clarity
- cloud-fallback determinism
- inactive-but-preserved alternate architectures

Safe Next Development Step

Recommended next step

Create a strict runtime-state boundary and make the current local epistemic path the only canonical execution authority.

This is not a feature expansion. It is a stabilization step.

Concrete scope:

1. classify every top-level directory as `source`, `generated`, `archive`, or `experimental`
2. stop mixing generated runtime state with source-adjacent areas where possible
3. declare one active memory surface
4. declare one active local knowledge route
5. add registry freshness checks before provenance/contradiction usage

Why this is the safest next step:

- it reduces the highest current operational risk without changing core behavior
- it improves determinism claims by narrowing ambiguity, not by adding complexity
- it lowers maintenance cost before any orchestration or agent expansion

Dangerous Next Development Step

Most dangerous next step

Adding multi-model orchestration or autonomous workflows on top of the current repository state model.

Why it is dangerous:

- execution authority is already split between local route, epistemic kernel, knowledge router, and cloud planner
- memory is already split between `memory/`, `obsidian_vault/`, and `AI_MEMORY_VAULT/`
- runtime writes many artifacts into the repo during normal operation
- cloud fallback breaks strong determinism claims

What would likely happen:

- more hidden control flow
- more mixed state
- harder incident reconstruction
- higher token and logging costs
- increased false confidence in agent autonomy

Critical Fixes Before Expansion

1. Registry freshness validation

Problem:

- `adaptive_routing/epistemic_kernel.py` trusts `provenance_registry.json` and `contradiction_registry.json` if they parse, even if `knowledge/` changed.

Why critical:

- provenance-aware retrieval becomes provenance-labeled retrieval, not proven-current retrieval.

2. Single active memory authority

Problem:

- active runtime memory is `obsidian_vault/`
- imported persistent vault is `AI_MEMORY_VAULT/`

Why critical:

- future work on memory, review, or promotion logic will drift immediately if this is not resolved.

3. Evidence ledger cleanup

Problem:

- `tools/executor.py` writes action results into the same evidence channel as source evidence.

Why critical:

- epistemic evidence and operational evidence are not the same thing.

4. Generated-state containment

Problem:

- logs, browser profile, JSONL traces, vault sessions, and scan artifacts all grow inside the repo.

Why critical:

- storage growth and repository noise will outrun code complexity first.

5. Canonical routing declaration

Problem:

- two overlapping local knowledge gates exist:
- `adaptive_routing/epistemic_kernel.py`
- `orchestrator/knowledge_router.py`

Why critical:

- overlapping local authority is acceptable temporarily, but dangerous before higher-level orchestration is added.

Technical Debt Priority List

Priority 1

- Generated runtime state inside repo root
- Registry freshness assumptions
- Dual memory authority

These are the most dangerous because they directly affect correctness, reproducibility, and maintenance.

Priority 2

- Overlapping local knowledge routing layers
- Disabled orchestrator subsystem still preserved beside active runtime
- Documentation duplication across root and `docs/`

These are architectural ambiguity debts.

Priority 3

- Orphan utility code such as `tools/web_reader.py`
- Preserved research routers not on active path
- Historical exports, reports, and checkpoints in the same workspace

These are mainly maintenance-noise debts.

Resource Risk Analysis

RAM risks

Current state:

- in-memory runtime state is relatively bounded
- `previous_commands` and `recent_outputs` are sliced
- retrieval works on about `52` markdown and `10` JSON knowledge artifacts

First RAM stress point:

- persistent Playwright browser context
- not retrieval itself

Why:

- `tools/browser_tools.py` keeps a Chromium persistent context alive across actions
- browser state will dominate memory before local retrieval does

Assessment:

- short-term RAM risk: moderate
- first real RAM break: browser-heavy or multi-page workflows

CPU bottlenecks

Current state:

- retrieval is repeated linear scanning and scoring across local modules and examples
- grep retrieval counts literal matches across module content
- `scan_project()` walks directory trees and writes a report each run

First CPU stress point:

- repeated retrieval plus repeated prompt building in multi-step sessions

Why:

- `tools/rhcsa_search.py` is simple and deterministic, but not indexed for asymptotic scaling beyond current corpus size
- prompt construction and JSON serialization repeat every step

Assessment:

- current CPU profile is acceptable
- scaling beyond current knowledge size will degrade gradually, not catastrophically

Scaling limitations

Most important limit:

- the architecture scales in artifacts faster than it scales in execution clarity

What scales poorly first:

- logs
- vault session files
- browser profile
- duplicate docs
- runtime-state ambiguity

Not the first scaling limit:

- RHCSA retrieval volume

Storage growth risks

Observed sizes:

- `state/` about `3.8M`
- `logs/` about `428K`
- `obsidian_vault/` about `332K`
- `memory/` about `140K`
- `checkpoints/` about `59M`

What breaks first:

- workspace hygiene and backup size

Why:

- every session adds logs and vault records
- browser profile caches accumulate
- checkpoints are already the largest storage surface

Assessment:

- storage growth is the earliest clear operational break.

Logging overhead

Current logging surfaces:

- command logs
- session logs
- error logs
- history JSONL

- evidence JSONL
- reasoning JSONL
- vault daily notes
- vault session logs

Risk:

- duplicated persistence of the same event into multiple channels

Assessment:

- logging is useful for forensics
- logging is already too fragmented for low-maintenance scaling

Retrieval complexity

Current complexity shape:

- effectively O(n) over modules/examples per query
- repeated for keyword, exact, grep, and tag routes

Assessment:

- acceptable at current corpus size
- not the first thing to fail
- will become annoying before it becomes catastrophic

Maintenance difficulty

Main cause:

- multiple partially overlapping authority surfaces

Examples:

- root docs vs `docs/`
- `docs/ADR` vs `docs/adr`
- `obsidian_vault` vs `AI_MEMORY_VAULT`
- epistemic kernel vs knowledge router
- active runtime vs preserved disabled orchestrator

Assessment:

- maintenance risk is already higher than algorithmic complexity risk.

What Breaks First

First operational break

- storage and state sprawl

First instability source

- ambiguity of active memory and routing authority

Most dangerous technical debt

- repository-local generated state mixed with canonical source and architecture surfaces

Readiness Assessment

Multi-model orchestration

- Readiness: `NOT READY`

Why:

- provider fallback already introduces nondeterminism
- preserved orchestrator path is disabled
- runtime authority is not narrow enough yet

External APIs

- Readiness: `PARTIAL`

Why:

- external APIs already exist in provider path
- operationally possible
- epistemically weak as a next expansion target because cloud dependency is already a major caveat

Autonomous workflows

- Readiness: `NOT READY`

Why:

- approval boundary exists, but memory and routing authority are too ambiguous
- autonomous workflows would amplify current state-sprawl and planner nondeterminism

Distributed memory

- Readiness: `NOT READY`

Why:

- single-node memory semantics are not yet clean
- distributed memory before single-memory clarity would multiply contamination risk

Advanced agent systems

- Readiness: `NOT READY`

Why:

- current system is a supervised tool runtime with local epistemic routing
- it is not yet a stable substrate for higher-order agent architectures

Final Recommendation

Do not expand AOIA upward.

Expand AOIA inward:

- tighten memory authority
- tighten routing authority
- validate registry freshness
- contain generated state
- reduce logging surface duplication

Only after that is it technically reasonable to reassess orchestration or agent expansion.

18. RAPORT_16-10.md

RAPORT_16-10

Current Runtime Architecture Summary

- `runtime/main.py` is the primary terminal coordinator and still combines prompt assembly, routing, model fallback, execution loop, logging, and runtime status.
- `runtime/tools/memory.py` is the main persistence hub for runtime state, history, evidence, reasoning traces, browser logs, and Obsidian vault projection.
- `runtime/tools/executor.py` dispatches validated actions and records them back into command logs and memory.
- `runtime/adaptive_routing/epistemic_kernel.py` is the strongest deterministic local routing candidate for RHCSA-style knowledge handling.
- `runtime/orchestrator/knowledge_router.py` is a legacy local-memory routing path that overlaps with the epistemic kernel.
- `runtime/providers/config.py` manages model selection, fallback chain, API env loading, and provider instantiation in one place.
- `runtime/webapp.py` exposes the same runtime over HTTP and shares one in-process runtime instance.

Highest-Risk Contamination Zones

- `runtime/tools/memory.py`
- `runtime/main.py`
- `runtime/providers/config.py`
- `runtime/adaptive_routing/epistemic_kernel.py`
- `runtime/orchestrator/knowledge_router.py`
- `runtime/orchestrator/gemini_gemma.py`

memory.py Risk Summary

- `memory.py` mixes durable state, append-only history, evidence, reasoning, browser logs, and Obsidian note generation.
- The same adapter writes both operational records and epistemic records.
- This creates a high risk of pseudo-evidence contamination and mutable authority bleed-through.

Retrieval Boundary Risks

- `AOIAEpistemicKernel` and `KnowledgeRouter` both decide when local RHCSA memory should answer before cloud reasoning.
- `runtime/tools/rhcsa_search.py` is deterministic, but it is still consumed through two overlapping routing authorities.
- `runtime/tools/build_rhcsa_library.py` still references `memory/rhcsa_context.py`, which is not present as an in-repo runtime module boundary.

Mutable Authority Risks

- `runtime/main.py` constructs runtime state and passes it directly into model prompts.
- `runtime/tools/memory.py` persists mutable state and generates notebook-style projections from the same events.
- `runtime/providers/config.py` persists model/provider choice and loads secrets into process environment.
- `runtime/tools/memory_hats.py` persists prompt-shaping overlays that directly affect runtime planning.

Split-Brain / Runtime Routing Risks

- The current runtime has two local routing authorities:
- `runtime/adaptive_routing/epistemic_kernel.py`
- `runtime/orchestrator/knowledge_router.py`
- `runtime/orchestrator/gemini_gemma.py` preserves a transitional two-model orchestration path.
- `runtime/main.py` still contains both direct routing and orchestrated routing logic, which creates split-brain policy risk.

Safest First Canonical Refactor Target

- `runtime/tools/memory.py`

Reason:

- it has the highest contamination density
- it is the main boundary where state, logs, evidence, reasoning, and vault projection collide
- separating it first reduces risk for every later refactor

Modules That Must Remain Untouched Until Governance Is Finalized

- `runtime/providers/config.py`
- `runtime/providers/aureon_provider.py`
- `runtime/providers/gemini_provider.py`
- `runtime/providers/openai_compatible.py`
- `runtime/adaptive_routing/epistemic_kernel.py`
- `runtime/orchestrator/knowledge_router.py`
- `runtime/orchestrator/gemini_gemma.py`
- `runtime/tools/memory.py`
- `runtime/tools/memory_hats.py`
- `runtime/tools/executor.py`
- critical policy sections of `runtime/main.py`

Estimated Implementation Phase Count

- 6 phases

1. memory boundary separation
2. routing authority consolidation
3. provider/config separation
4. thin coordinator refactor for `main.py`
5. legacy orchestrator quarantine or retirement
6. governance layer finalization

Rollback Risks

- Rolling back only one of the memory, routing, or provider boundaries may reintroduce split-brain authority.
- If `memory.py` is refactored without first defining durable state vs evidence vs reasoning, data shape drift will occur.
- If the legacy orchestrator is removed before the kernel/router split is finalized, fallback behavior may break.

Dependency Risks

- `main.py` depends on mutable state coming from `MemoryStore`, `MemoryHatStore`, `ProviderManager`, the epistemic kernel, and the legacy router.
- `executor.py` depends on `MemoryStore` for recording every action result.
- `webapp.py` depends on a shared `AgentRuntime`, so state coupling is process-wide.
- `build_rhcsa_library.py` still advertises a runtime integration point that is not visible as an in-repo `memory.*` module.

Recommended Next Action Before Implementation

- Define the canonical memory layer split on paper first:
- `L0` ephemeral runtime state
- `L1` operational logs
- `L2` reasoning traces
- `L3` provenance records
- `L4` evidence store
- `L5` contradiction records
- Only after that, refactor `runtime/tools/memory.py` behind adapters.

AOIA-Core Readiness Assessment

- **PARTIALLY READY**

Justification:

- The deterministic retrieval foundation is real.
- The epistemic kernel and provenance/contradiction registries are real.
- The runtime still has unresolved boundary violations in memory, routing, provider config, and orchestration.
- The runtime can operate, but it is not yet architecturally clean enough for canonical refactor without a memory-boundary plan first.

19. REPOSITORY_TREE.md

Repository Tree

Method

This tree is a forensic source tree, not a raw byte-for-byte dump of every generated cache file.

Included:

- source code
- configuration
- docs
- tests
- knowledge assets
- top-level generated registries
- representative runtime state directories

Collapsed or excluded from the rendered tree:

- `.git/`
- `.venv/`
- `__pycache__/`
- deep browser cache internals under `state/browser_profile/`
- repetitive session and command log fan-out

Tree

```text
app2terminl_opened/
├── main.py
├── webapp.py
├── run.sh
├── run_web.sh
├── requirements.txt
├── prompts/
│   └── system_prompt.txt
├── commands/
│   ├── __init__.py
│   ├── base.py
│   └── local_commands.py
├── router/
│   ├── __init__.py
│   └── local_router.py
├── orchestrator/
│   ├── __init__.py
│   ├── gemini_gemma.py
│   └── knowledge_router.py
├── adaptive_routing/
│   ├── aoia_config.json
│   ├── config_loader.py
│   ├── deterministic_router.py
│   ├── epistemic_kernel.py
│   ├── circadian_router.py
│   ├── stdout_logger.py
│   ├── dvm_research.md
│   ├── routing_modes.json
│   └── environment/
│       ├── environment_router.py
│       ├── network_patterns.md
│       └── traffic_profiles.json
├── providers/
│   ├── __init__.py
│   ├── base.py
│   ├── config.py
│   ├── gemini_provider.py
│   ├── aureon_provider.py
│   ├── openai_compatible.py
│   └── gemma_provider.py
├── tools/
│   ├── __init__.py
│   ├── executor.py
│   └── validator.py
```

```
■   ■■■ memory.py
■   ■■■ memory_hats.py
■   ■■■ shell_tools.py
■   ■■■ filesystem_tools.py
■   ■■■ browser_tools.py
■   ■■■ system_info.py
■   ■■■ rhcsa_search.py
■   ■■■ epistemic_registry.py
■   ■■■ project_scanner.py
■   ■■■ build_rhcsa_library.py
■   ■■■ web_reader.py
■■■ memory/
■   ■■■ __init__.py
■   ■■■ gemma_worker_memory.py
■   ■■■ rhcsa_context.py
■   ■■■ hats/
■   ■■■ gemma_worker/
■   ■■■ history.jsonl
■   ■■■ evidence_memory.jsonl
■   ■■■ reasoning_trace.jsonl
■■■ knowledge/
■   ■■■ __init__.py
■   ■■■ README.md
■   ■■■ rhcsa_engine.py
■   ■■■ command_graph.json
■   ■■■ canonical/
■   ■■■ parsed/
■   ■■■ index/
■   ■■■ context/
■   ■■■ injection/
■   ■■■ schema/
■   ■■■ source/
■   ■■■ raw/
■   ■■■ examples/
■   ■■■ filesystem/
■   ■■■ networking/
■   ■■■ users/
■   ■■■ permissions/
■   ■■■ selinux/
■   ■■■ systemd/
■   ■■■ storage/
■   ■■■ lvm/
■   ■■■ podman/
■   ■■■ bash/
■   ■■■ troubleshooting/
■   ■■■ tools/
■   ■■■ validator/
■■■ tests/
■   ■■■ test_main.py
■   ■■■ test_knowledge_validator.py
■   ■■■ test_aoia_determinism.py
■   ■■■ test_rhcsa_retrieval.py
■   ■■■ test_epistemic_registry.py
■   ■■■ test_epistemic_safeguards.py
■   ■■■ test_epistemic_kernel.py
■■■ web/
■   ■■■ index.html
■   ■■■ app.js
■   ■■■ styles.css
■■■ apps/
■   ■■■ README.md
■   ■■■ flameborn-academy-codex-sparrow/
■       ■■■ README.md
■       ■■■ package.json
■       ■■■ package-lock.json
■       ■■■ index.html
■       ■■■ script.js
■       ■■■ style.css
■       ■■■ server.mjs
■       ■■■ workers/
■       ■■■ scripts/
■       ■■■ assets/
■       ■■■ wrangler.jsonc
■■■ docs/
■   ■■■ ADR/
■   ■■■ adr/
■   ■■■ reference/
■   ■■■ *.md
■■■ reports/
■■■ exports/
■■■ checkpoints/
■■■ state/
■   ■■■ agent_state.json
■   ■■■ model_config.json
■   ■■■ providers.json
```

```
■   ■■■ token_savings_report.json
■   ■■■ browser_profile/
■■■ logs/
■■■ obsidian_vault/
■■■ AI_MEMORY_VAULT/
■■■ provenance_registry.json
■■■ contradiction_registry.json
■■■ validator.py
■■■ project_scan.json
■■■ ARCHITECTURE.md
■■■ CONSTRAINTS.md
■■■ MEMORY_RULES.md
■■■ PROJECT_STRUCTURE.md
■■■ multiple additional historical and analytical markdown reports
```

Classification Notes

Active source

- `main.py`
- `webapp.py`
- `commands/`
- `router/`
- `providers/`
- `tools/`
- selected `adaptive_routing/`
- selected `knowledge/`

Generated runtime state

- `logs/`
- `memory/*.jsonl`
- `state/`
- `obsidian_vault/`
- `project_scan.json`

Imported persistent memory

- `AI_MEMORY_VAULT/`

Historical, analytical, or archival content

- `docs/`
- `reports/`
- `exports/`
- `checkpoints/`

Separate embedded application

- `apps/flameborn-academy-codex-sparrow/`

This directory is not part of the Python runtime graph and should be treated as a sibling subproject.

20. SYSTEM_EXECUTION_FLOW.md

System Execution Flow

Canonical CLI Flow

```text
User input
  -&gt; slash-command check
  -&gt; local deterministic route check
  -&gt; deterministic AOIA epistemic kernel
  -&gt; RHCSA knowledge router
  -&gt; optional model planner
  -&gt; validated JSON action
  -&gt; executor
  -&gt; tool handler
  -&gt; memory/log persistence
  -&gt; next step or final response
```

Step-By-Step Flow

1. Process bootstrap

`run.sh` starts `main.py` using `.venv` if present, otherwise system `python3`.

`main.py` builds:

- `ProviderManager`
- `MemoryStore`
- `MemoryHatStore`
- `GemmaWorkerMemory`
- `ExecutionEngine`
- `LocalRouter`
- `KnowledgeRouter`
- `AOIAEpistemicKernel`
- command registry

2. User request ingress

`main.py` receives text input and first checks:

1. slash commands in `commands/local_commands.py`
2. kill switch and epistemic safeguards
3. deterministic local routes in `router/local_router.py`
4. local knowledge route in `handle_knowledge_route()`

3. Deterministic local route

`router/local_router.py` handles obvious local tasks:

- date
- `pwd`
- `ls`
- `curl --version`
- simple desktop folder creation

These routes bypass model usage entirely.

4. Deterministic AOIA epistemic kernel

`adaptive_routing/epistemic_kernel.py` evaluates the request using:

- keyword and exact command retrieval
- provenance registry
- contradiction registry
- deterministic `pressure -> depth` selection

Outputs:

- route decision
- confidence
- evidence list
- contradiction notices
- manual review flags

If the kernel has enough local evidence, it returns a local answer immediately.

5. RHCSA knowledge router fallback

If the epistemic kernel does not close the request, `orchestrator/knowledge_router.py` may still answer locally through `knowledge/rhcsa_engine.py`.

This is a second local-first gate focused on Linux operational requests.

6. Model planning path

If local routes do not resolve the request:

- `main.py` builds a prompt using:
- runtime state
- recent outputs
- memory hats
- RHCSA injected context
- epistemic safeguard state

- `ProviderManager` selects a cloud provider.
- the selected provider returns JSON text.
- `tools.validator.extract_json_object()` and `validate_action()` normalize it.

7. Executor path

`tools/executor.py` is the only real dispatcher.

Flow:

1. approval request for non-`respond` actions
2. tool handler dispatch
3. shell/file/browser action execution
4. result persistence
5. evidence/history/log writes

Tool families:

- shell
- filesystem
- browser
- project scan
- final response

8. Persistence side effects

During execution the runtime writes to:

- `state/agent_state.json`
- `logs/commands/`
- `logs/errors/`
- `logs/sessions/`
- `memory/history.jsonl`
- `memory/evidence_memory.jsonl`
- `memory/reasoning_trace.jsonl`
- `obsidian_vault/`

This means the runtime is deterministic in routing intent, but not read-only in repository state.

Web Flow

`run_web.sh` starts `webapp.py`.

`webapp.py`:

- creates one shared `AgentRuntime`
- serves static assets from `web/`
- exposes JSON API endpoints:
- `/api/status`
- `/api/models`
- `/api/chat`
- `/api/model`

The web server does not implement a separate backend architecture. It wraps the same runtime.

Disabled Or Preserved Alternate Flow

There is a preserved but disabled orchestrator concept:

```text
Gemini planner
  -&gt; Gemma worker
  -&gt; JSON action
  -&gt; executor
```

In the current build this is not active because:

- `GemmaGemmaOrchestrator.gemma_provider` is not initialized
- `/orchestrator` commands return disabled notices
- provider config does not route to `providers/gemma_provider.py`

This is important: the repo contains orchestrator architecture, but the canonical execution path does not use it.

Main Risk Points In Execution

1. Approval boundary is interactive

The executor relies on terminal approval input. This is safe for a CLI operator, but fragile for automation and tests.

2. Model path is cloud-dependent

The runtime can avoid model use for some requests, but the general planner path still requires external providers.

3. State accumulation occurs inside the repo

Execution continuously mutates logs, memory, and vault paths inside the repository root.

4. Dual local knowledge gates exist

There are two local response layers:

- `adaptive_routing/epistemic_kernel.py`
- `orchestrator/knowledge_router.py`

They are not contradictory, but they do overlap. This should be treated as controlled duplication until one authority is declared.

Forensic Conclusion

The actual executable system is simpler than the repository perimeter suggests.

Real active path:

- CLI or web ingress
- deterministic local routing
- deterministic epistemic retrieval
- optional cloud planning
- centralized executor

Non-canonical but present:

- AOIA research routers
- disabled Gemini/Gemma orchestrator
- embedded frontend subproject
- large historical and generated state surface